PYTHON

CHEAT SHEETS

A Complete Reference for Modern Python 3

Syntax · Strings · Control Flow · match/case · Functions · Data Structures · OOP · Files & JSON · Standard Library · Testing · Async & Concurrency · GUI & Web Frameworks

93 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–04 Introduction & Setup
05–06 Output & Comments
07–13 Variables & Data Types
14–18 Strings
19–24 Operators
25–33 Control Flow

CORE PYTHON

34–36 Error Handling
37–45 Functions
46–56 Data Structures
57–67 OOP
68–72 Files & I/O

LIBRARIES & ADVANCED

73–79 Modules & Standard Library
80–82 Testing & Logging
83–89 Advanced Topics
90–91 Frameworks Overview
92–93 Quick Reference

PY

Table of Contents

93 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

- 01 Python History & Philo…
- 02 Installation & Setup
- 03 Running Python
- 04 Program Structure & E…

OUTPUT & COMMENTS 05 – 06

- 05 Print & Output
- 06 Comments & Docstrin…

VARIABLES & DATA TYPES 07 – 13

- 07 Variables & Assignment
- 08 Integers
- 09 Floats
- 10 Booleans & Truthiness
- 11 None & Type Checking
- 12 Type Conversion & Ca…
- 13 User Input & Parsing

STRINGS 14 – 18

- 14 String Basics
- 15 String Formatting
- 16 String Methods — Sear…
- 17 String Methods — Tran…
- 18 String Slicing & Indexing

OPERATORS 19 – 24

- 19 Arithmetic Operators
- 20 Comparison Operators
- 21 Logical Operators
- 22 Bitwise Operators
- 23 Assignment & Compo…
- 24 Membership & Identit…

CONTROL FLOW 25 – 33

- 25 if/elif/else Statements
- 26 match/case Pattern M…
- 27 Ternary & Conditional …
- 28 for Loops
- 29 while Loops
- 30 Loop Controls
- 31 List Comprehensions
- 32 Dict & Set Comprehen…
- 33 Generator Expressions

ERROR HANDLING 34 – 36

- 34 try/except/finally Blocks
- 35 Raising & Re-raising Ex…
- 36 Custom Exceptions

FUNCTIONS 37 – 45

- 37 Defining & Calling Func…
- 38 Parameters & Argume…
- 39 Default & Keyword Arg…
- 40 Variable Scope
- 41 Closures & Nested Fun…
- 42 Lambda Functions
- 43 Higher-Order Functions
- 44 Decorators
- 45 Generators & yield

DATA STRUCTURES 46 – 56

- 46 Lists — Basics
- 47 Lists — Methods
- 48 Lists — Advanced Ope…
- 49 Tuples
- 50 Dictionaries — Basics
- 51 Dictionaries — Methods
- 52 Dictionaries — Advanc…
- 53 Sets
- 54 Collections Module
- 55 itertools Module
- 56 Sorting & Searching

OOP 57 – 67

- 57 Classes & Objects
- 58 Constructors & Initializ…
- 59 Instance, Class & Stati…
- 60 Properties & Access C…
- 61 Inheritance
- 62 Multiple Inheritance & …
- 63 Polymorphism & Duck …
- 64 Abstract Classes & Int…
- 65 Dunder Methods — Ba…
- 66 Dunder Methods — Op…
- 67 Context Managers

FILES & I/O 68 – 72

- 68 Reading Files
- 69 Writing & Appending F…
- 70 File Paths & Operations
- 71 JSON
- 72 CSV Files

MODULES & STANDARD LIBRARY 73 – 79

- 73 Importing & Modules
- 74 Creating Packages
- 75 pip & Package Manage…
- 76 Math & Random Modul…
- 77 Dates, Times & Timezo…
- 78 OS & System Modules
- 79 functools Module

TESTING & LOGGING 80 – 82

- 80 unittest Testing Frame…
- 81 pytest Testing Framew…
- 82 Logging

ADVANCED TOPICS 83 – 89

- 83 Regular Expressions
- 84 Async/await & Corouti…
- 85 Threading & Concurre…
- 86 Multiprocessing
- 87 SQLite Database
- 88 Serialization & Pickling
- 89 Type Hints & Typing M…

FRAMEWORKS OVERVIEW 90 – 91

- 90 GUI Frameworks Overv…
- 91 Web Frameworks Over…

QUICK REFERENCE 92 – 93

- 92 Quick Reference — Sy…
- 93 Quick Reference — Col…

Python History & Philosophy

WHAT IS PYTHON?

Python is a general-purpose, high-level language created by **Guido van Rossum** and first released in 1991. It puts readability first: indentation defines code blocks, and the language favors one obvious way to do things. The reference implementation, **CPython**, compiles source to bytecode and runs it on a virtual machine. Python 2 reached end of life on January 1, 2020, so all modern code targets **Python 3**.

KEY FACTS

- Named after Monty Python, not the snake
- **Dynamically typed** — types belong to values, not variables
- **Strongly typed** — `"2" + 2` raises `TypeError`
- Memory is managed by reference counting plus a cycle-detecting garbage collector
- Language changes are proposed and documented as **PEPs**

UNDER THE HOOD

- Source compiles to bytecode, cached in `__pycache__/*.pyc`
- The eval loop runs that bytecode one instruction at a time
- See it yourself: `import dis; dis.dis(f)`

HELLO, WORLD & VERSION CHECK

```
# hello.py
print("Hello, World!")          # Hello, World!

# The Zen of Python (PEP 20)
import this                     # prints 19 aphorisms

# Inspect the running interpreter
import sys
print(sys.version_info)         # sys.version_info(major=3, minor=12, ...)
print(sys.version_info >= (3, 10))  # True on 3.10 and newer
print(sys.implementation.name)  # cpython
```

CODE EXPLAINED

- → `print()` — built-in; writes text to standard output
- → `import this` — Easter egg that prints the Zen of Python
- → `sys.version_info` — named tuple; compare it to a tuple like `(3, 10)`
- → `sys.implementation.name` — `cpython`, `pypy`, etc.

PYTHON VERSIONS

- **3.0** — 2008 — `print()` is a function, `str` is Unicode
- **2.7** — end of life in January 2020
- **3.13** — experimental free-threaded build and JIT

THE PYTHONIC WAY

- ✗ ~~`python` (could be Python 2)~~ → ✓ `python3` **or** `py`
- ✗ ~~Clever one-liners~~ → ✓ **readable code (Zen: readability counts)**
- ✗ ~~Hand-formatting style debates~~ → ✓ **PEP 8 +** `ruff format`

TIMELINE

YEAR	MILESTONE
1991	Python 0.9.0 released
2000	2.0: list comprehensions, cycle GC
2008	3.0: `print()` function, Unicode `str`
2020	Python 2 end of life
2021	3.10: `match` statement
2023	3.12: f-string grammar relaxed (PEP 701)
2024	3.13: new REPL, experimental free-threading
2025	3.14: template strings (t-strings)

IMPLEMENTATIONS

NAME	USE
CPython	Reference implementation, written in C
PyPy	JIT compiler; faster long-running code
MicroPython	Microcontrollers and embedded boards
GraalPy	Runs on GraalVM; Java interop
IronPython	.NET integration

FILE TYPES

EXT	MEANING
`.py`	Source file
`.pyw`	Windows script with no console window
`.pyc`	Cached bytecode in __pycache__/
`.pyi`	Type stub (annotations only)
`.ipynb`	Jupyter notebook
`.whl`	Wheel: built package for pip

⚠ COMMON MISTAKES

- Following Python 2 tutorials — `print "x"` is a `SyntaxError` in Python 3
- Mixing tabs and spaces — raises `TabError`; use 4 spaces
- Assuming `python` means Python 3 — some systems map it to 2 or nothing; use `python3` or `py`
- Thinking interpreted means no compile step — CPython compiles to `.pyc` bytecode first

✓ TIPS & BEST PRACTICES

- Run `import this` once and read it; idiomatic code follows it
- Learn PEP 8 style, then let `ruff format` or `black` enforce it
- Check `sys.version_info` before relying on newer syntax
- Read the docs at docs.python.org for the exact version you run

QUICK SUMMARY

- `CPython` — source → bytecode → virtual machine
- `Python 3` — the only supported line since 2020
- `PEP 20` — the Zen of Python; readability counts
- `.py` — source; `.pyc` — cached bytecode

Installation & Setup

GETTING PYTHON RUNNING

Install Python 3 from **python.org** (Windows, macOS), your OS package manager (Linux), or a version manager. Give every project its own **virtual environment** so its packages never collide with other projects or the system Python. The `venv` module is built in, and `pip` installs packages into whichever environment is active.

PLATFORM NOTES

- Windows installer: tick **Add python.exe to PATH**
- Windows `py` launcher selects versions: `py -3.12`
- macOS `/usr/bin/python3` comes with Xcode tools; prefer python.org or Homebrew
- Linux system Python runs OS tools; never `pip install` into it
- `uv` is a fast newer tool that handles venvs and packages

VERIFY, CREATE A VENV, INSTALL

```
# Verify the install
python3 --version              # Python 3.12.x
python3 -m pip --version       # pip 24.x from ...

# Create and activate a virtual environment
python3 -m venv .venv          # creates ./.venv
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate         # Windows

# Work inside the env
pip install requests           # installs into .venv only
pip freeze > requirements.txt # record exact versions
deactivate                     # leave the env
```

CODE EXPLAINED

- → `-m venv` — runs the venv module as a script
- → `.venv` — conventional folder name; add it to `.gitignore`
- → `activate` — puts the env's `bin/` or `Scripts\` first on PATH
- → `python -m pip` — guarantees pip matches that interpreter
- → `deactivate` — restores the previous PATH

PYTHON VERSIONS

- **3.3** — `venv` module and the Windows `py` launcher
- **3.4** — `pip` bundled through `ensurepip`
- **pip 23** — refuses installs into PEP 668 system Pythons

THE PYTHONIC WAY

- ✗ ~~`pip install` into the system Python~~ → ✓ `python -m venv .venv` **first**
- ✗ ~~`pip install x`~~ → ✓ `python -m pip install x`
- ✗ ~~Global installs of CLI tools~~ → ✓ `pipx install ruff`

INSTALL BY PLATFORM

PLATFORM	COMMAND
Windows	python.org installer or `winget install Python.Python.3.12`
macOS	`brew install python`
Debian/Ubuntu	`sudo apt install python3 python3-venv`
Fedora	`sudo dnf install python3`
Any (Unix)	`pyenv install 3.12`

ENVIRONMENT TOOLS

TOOL	PURPOSE
venv	Built-in isolated environments
pyenv	Install and switch Python versions
conda	Envs plus non-Python dependencies
uv	Fast pip + venv replacement
pipx	Install CLI tools in isolation

DIAGNOSTICS

COMMAND	SHOWS
`which python`	Interpreter path (Unix)
`where python`	Interpreter paths (Windows)
`pip list`	Packages in the active env
`py -0`	Installed versions (Windows)
`python -m site`	sys.path and site-packages

⚠ COMMON MISTAKES

- `pip install` with no active venv — pollutes the global site-packages
- `sudo pip install` on Linux — can break OS tools; use a venv or `pipx`
- Committing `.venv/` to git — it's machine-specific; commit `requirements.txt`
- `pip` and `python` pointing at different installs — use `python -m pip`

✓ TIPS & BEST PRACTICES

- One venv per project, stored in the project folder as `.venv`
- VS Code and PyCharm detect `.venv`; select it as the interpreter
- Restore an env with `pip install -r requirements.txt`
- Use `pipx` for global tools like `ruff` or `httpie`

QUICK SUMMARY

- `python3 --version` — confirm the install
- `python -m venv .venv` — create an isolated env
- `activate` / `deactivate` — enter and leave it
- `python -m pip install` — install into the active env

Running Python

THREE WAYS TO RUN CODE

Python runs code interactively in the **REPL**, from a **script file**, or as a **module** with `-m`. The REPL evaluates each line immediately and echoes expression results, which makes it ideal for testing ideas. A script runs top to bottom once. The `-m` flag runs a module by its import name, which is how you launch tools like `pip`, `venv`, and `http.server`.

REPL BASICS

- `>>>` is the main prompt, `...` a continuation line
- `_` holds the last expression result
- Exit with `exit()`, Ctrl+D (Unix), or Ctrl+Z Enter (Windows)
- 3.13+ REPL adds color, multiline editing, and bare `exit`
- IPython and Jupyter are richer third-party REPLs

RUNNING PYTHON

```
python3                          # start the REPL
>>> 2 + 3
5
>>> _ * 10                       # 50  (last result)

python3 hello.py                 # run a script
python3 hello.py a b             # sys.argv == ['hello.py', 'a', 'b']
python3 -m http.server 8000      # run a module as a script
python3 -c "print(2**10)"        # 1024
python3 -i hello.py              # run, then stay in the REPL

# Unix: make a script executable
#!/usr/bin/env python3          # first line of hello.py
chmod +x hello.py && ./hello.py
```

CODE EXPLAINED

- → `-m module` — finds the module on `sys.path` and runs it as `__main__`
- → `-c "code"` — executes a string of code
- → `-i` — keeps the REPL open so you can inspect variables
- → `#!/usr/bin/env python3` — shebang; locates python3 on PATH
- → `sys.argv[0]` — script name; arguments follow

PYTHON VERSIONS

- **3.7** — `breakpoint()` and `-X dev` mode
- **3.13** — new REPL: color, multiline editing, bare `exit`

THE PYTHONIC WAY

- ✗ ~~Copy-pasting code to test a snippet~~ → ✓ `python -i script.py`
- ✗ ~~`python path/to/pkg/mod.py`~~ → ✓ `python -m pkg.mod`
- ✗ ~~`print()` everywhere to debug~~ → ✓ `breakpoint()`

COMMAND-LINE FLAGS

FLAG	EFFECT
`-m mod`	Run a module
`-c cmd`	Run a code string
`-i`	Interactive after script
`-u`	Unbuffered stdout/stderr
`-O`	Strip `assert` statements
`-X dev`	Development mode (extra checks)
`-V`	Print version

REPL HELPERS

CALL	SHOWS
`help(obj)`	Docs for an object
`dir(obj)`	Its attributes
`type(obj)`	Its type
`_`	Last result
`help()`	Interactive help; `q` quits

OTHER WAYS TO RUN

TOOL	HOW
IDLE	`python -m idlelib`
VS Code	Run ▶ or F5
Jupyter	`jupyter lab`
py launcher	`py script.py` (Windows)

⚠ COMMON MISTAKES

- Typing `python hello.py` inside the REPL — it's a shell command; exit first
- Naming a script `random.py` or `json.py` — it shadows the stdlib module
- `python -m mymodule.py` — drop `.py`; `-m` takes an import name
- Shebang file saved with CRLF — Unix reports `python3\r` not found; save with LF

✓ TIPS & BEST PRACTICES

- `python -i script.py` lets you inspect state after a crash
- `python -m http.server` serves the current folder instantly
- Keep a REPL open while coding to test snippets
- `breakpoint()` drops into the pdb debugger mid-script

QUICK SUMMARY

- `python3` — interactive REPL
- `python3 file.py` — run a script
- `python3 -m pkg` — run a module by import name
- `#!/usr/bin/env python3` — executable Unix scripts

Program Structure & Entry Point

HOW A PYTHON FILE IS ORGANIZED

A Python file is a **module** that runs top to bottom when executed or imported. Blocks are defined by **indentation** instead of braces, and every block header ends with a colon. The `if __name__ == "__main__":` guard lets one file work both as an importable module and a runnable script, because the code inside it runs only when the file is executed directly.

RULES

- `__name__` is `"__main__"` when run directly, the module name when imported
- PEP 8: 4 spaces per indent level, no tabs
- One statement per line; no semicolons needed
- Lines inside `()`, `[]`, `{}` continue automatically
- A trailing `\` continues a line but breaks easily

ANNOTATED SCRIPT TEMPLATE

```
#!/usr/bin/env python3
"""Greet a user by name."""          # 1. module docstring

import sys                           # 2. imports: stdlib, 3rd-party, local

DEFAULT_NAME = "World"               # 3. module-level constants

def greet(name: str) -> str:         # 4. functions and classes
    """Return a greeting."""
    return f"Hello, {name}!"

def main() -> int:                   # 5. entry-point function
    name = sys.argv[1] if len(sys.argv) > 1 else DEFAULT_NAME
    print(greet(name))               # Hello, World!
    return 0                         # exit status 0 = success

if __name__ == "__main__":           # 6. runs only when executed
    sys.exit(main())
```

CODE EXPLAINED

→ docstring — first string in the file; shown by `help()`

→ imports — grouped stdlib → third-party → local, one blank line between

→ `main()` — keeps logic out of module scope and makes it testable

→ `sys.exit(main())` — passes the return code to the shell

→ `__name__` guard — importing the file doesn't call `main()`

THE PYTHONIC WAY

✗ ~~Top-level script code~~ → ✓ `def main():` + `if __name__ == "__main__":`

✗ ~~Trailing `\` line continuation~~ → ✓ **wrap in** `( )`

✗ ~~`import os, sys`~~ → ✓ **one import per line**

INDENTATION & BLOCKS

SYNTAX	MEANING
`if x:`	Colon opens a block
`4 spaces`	One indent level
`pass`	Empty-block placeholder
`dedent`	Closes the block
`tab + spaces`	`TabError`

LINE CONTINUATION

SYNTAX	MEANING
`( ... )`	Implicit continuation in () [] {}
`\`	Explicit continuation (avoid)
`a = 1; b = 2`	Two statements on one line (discouraged)
`"""..."""`	String spanning lines

MODULE ATTRIBUTES

NAME	VALUE
`__name__`	`"__main__"` or module name
`__file__`	Path to the module file
`__doc__`	Module docstring
`__all__`	Names exported by `import *`
`__package__`	Parent package name

⚠ COMMON MISTAKES

- Script logic at module level — it runs on every import; put it in `main()`
- `if __name__ == "main":` — missing underscores, so it's never true
- Mixing tabs and spaces — `TabError` or `IndentationError`
- Space after a trailing `\` — breaks the continuation; use parentheses

✓ TIPS & BEST PRACTICES

- Return an `int` from `main()` and hand it to `sys.exit()`
- Wrap long expressions in parentheses instead of using `\`
- Set your editor to insert 4 spaces on Tab
- Let `ruff` or `isort` order your imports

QUICK SUMMARY

- `:` + indent — defines a block
- `__name__ == "__main__"` — true only when run directly
- `main()` — single, testable entry point
- `( )` — implicit line continuation

Print & Output

HOW PRINT() WORKS

`print()` converts each argument with `str()`, joins them with **sep** (a space by default), appends **end** (a newline by default), and writes the result to `sys.stdout`. Its keyword arguments cover most formatting needs without string concatenation. Send errors and diagnostics to `sys.stderr` so they stay separate from normal output.

NOTES

- Signature: `print(*objects, sep=' ', end='\n', file=None, flush=False)`
- Non-strings convert automatically: `print(42)` works
- `print()` returns `None`
- stdout is line-buffered in a terminal, block-buffered when piped
- Use the `logging` module (Sheet 82) for applications

PRINT() VARIANTS

```
import sys
print("Hello")                          # Hello
print("a", "b", "c")                    # a b c
print("a", "b", "c", sep="-")           # a-b-c
print("a", "b", sep="")                 # ab
print("Loading", end="...")             # Loading...  (no newline)
print()                                 # blank line
print(3, 4.5, True, None)               # 3 4.5 True None
print(*[1, 2, 3])                       # 1 2 3
print(*[1, 2, 3], sep="\n")             # one per line
print(f"{2 + 3 = }")                    # 2 + 3 = 5
print(repr("hi"))                       # 'hi'
print("failed!", file=sys.stderr)       # goes to stderr
print("50%", end="\r", flush=True)      # overwrite same line
```

CODE EXPLAINED

- → `sep` — string placed between arguments
- → `end` — string placed after the last argument
- → `*list` — unpacks items into separate arguments
- → `f"{expr = }"` — prints the expression and its value (3.8+)
- → `file=sys.stderr` — separate stream for errors
- → `flush=True` — write immediately (progress output)

PYTHON VERSIONS

- **3.0** — `print` became a function
- **3.3** — `flush=` argument

THE PYTHONIC WAY

- ✗ ~~`print("x=" + str(x))`~~ → ✓ `print(f"{x=}")`
- ✗ ~~`print(a + " " + b)`~~ → ✓ `print(a, b)`
- ✗ ~~`print()` for errors~~ → ✓ `print(..., file=sys.stderr)`

ESCAPE SEQUENCES

SEQ	OUTPUT
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\' \"`	Quote marks
`\r`	Carriage return
`\u00e9`	Unicode: é

OUTPUT TARGETS

TARGET	USE
`sys.stdout`	Default destination
`sys.stderr`	Errors and warnings
`file=f`	An open file object
`io.StringIO()`	Capture output as a string
`pprint.pprint()`	Pretty-print nested data

STR VS REPR

CODE	PRINTS
`print("hi")`	hi
`print(repr("hi"))`	'hi'
`print(["hi"])`	['hi'] (items use repr)
`f"{x!r}"`	repr inside an f-string

⚠ COMMON MISTAKES

- `print "x"` — Python 2 syntax; `SyntaxError` in Python 3
- `print("Total: " + 5)` — `TypeError`; use a comma or an f-string
- `x = print("hi")` — `x` is `None`, not the text
- Progress not appearing when piped — add `flush=True`

✓ TIPS & BEST PRACTICES

- `print(f"{var=}")` is the fastest debug print
- `pprint.pprint()` makes nested dicts readable
- Send errors to `sys.stderr` so `> out.txt` captures only results
- `print(*items, sep="\n")` prints one item per line

QUICK SUMMARY

- `print(a, b)` — space-separated, newline-terminated
- `sep=` / `end=` — control separators
- `file=` — redirect to stderr or a file
- `flush=True` — force an immediate write

Comments & Docstrings

COMMENTS VS DOCSTRINGS

Comments start with # and run to the end of the line; Python ignores them completely. **Docstrings** are string literals placed first in a module, class, or function. Unlike comments, they are kept at runtime in `__doc__`, where `help()`, IDE hover text, and documentation generators read them. Python has no block-comment syntax, so use consecutive # lines.

NOTES

- Comments explain **why**; the code already shows what
- A triple-quoted string that isn't first in a body is a discarded expression, not a comment
- PEP 257 defines docstring conventions
- Common styles: Google, NumPy, reStructuredText (Sphinx)
- `# noqa` and `# type: ignore` are tool directives

COMMENTS & A GOOGLE-STYLE DOCSTRING

```python
# Full-line comment
x = 10  # inline comment: two spaces before #

def area(w: float, h: float) -> float:
    """Return the area of a rectangle.

    Args:
        w: Width in meters.
        h: Height in meters.

    Returns:
        Area in square meters.

    >>> area(2, 3)
    6
    """
    return w * h

print(area.__doc__.splitlines()[0])  # Return the area of a rectangle.
help(area)                           # shows the full docstring
```

CODE EXPLAINED

- → `#` — ignored to the end of the line
- → `"""..."""` — first statement in the body, stored in `__doc__`
- → summary line — one sentence, then a blank line
- → `Args:` / `Returns:` — Google-style sections
- → `>>>` — doctest example; run `python -m doctest file.py -v`

THE PYTHONIC WAY

- ✗ ~~`# This function adds two numbers`~~ → ✓ **a docstring saying why / what it returns**
- ✗ ~~`'''blocks to disable code`~~ → ✓ `#` **or delete it (git remembers)**
- ✗ ~~Multi-paragraph first line~~ → ✓ **one-line summary, blank line, details**

COMMENT FORMS

SYNTAX	USE
`# text`	Full-line comment
`x = 1 # text`	Inline comment
`# TODO:`	Marker editors can list
`#!`	Shebang (line 1 only)
`# noqa`	Silence a linter warning

DOCSTRING LOCATIONS

WHERE	HOW
Module	First statement in the file
Class	First line of the class body
Function	First line after `def`
Method	First line after `def` in a class
Access	`obj.__doc__` or `help(obj)`

TOOLS THAT READ THEM

TOOL	DOES
`help()`	Shows docs in the REPL
pydoc	`python -m pydoc module`
doctest	Runs >>> examples as tests
Sphinx	Builds HTML documentation
IDEs	Hover tooltips

⚠ COMMON MISTAKES

- Using `'''` blocks as comments — they're string expressions and may be mistaken for docstrings
- Comments that restate code: `i += 1 # add 1 to i`
- Docstring placed after other code — it's no longer the docstring
- Stale comments — a wrong comment is worse than none

✓ TIPS & BEST PRACTICES

- Document every public module, class, and function
- Write the first line as a short imperative summary
- Add doctest examples to small, pure functions
- Toggle comments with Ctrl+/ (Cmd+/ on macOS) in most editors

QUICK SUMMARY

- `#` — comment, discarded at runtime
- `"""docstring"""` — first statement, kept in `__doc__`
- `help(obj)` — reads docstrings
- `python -m doctest` — tests docstring examples

Variables & Assignment

NAMES BOUND TO OBJECTS

A Python variable is a **name bound to an object**; assignment never copies data. = creates the name or rebinds it, with no declaration keyword and no type required. The same name can later refer to an object of any type. Constants exist only as a naming convention (ALL_CAPS); nothing stops reassignment.

RULES

- Names are case-sensitive: `total` ≠ `Total`
- Letters, digits, `_`; can't start with a digit
- Keywords are reserved: see `keyword.kwlist`
- `del x` removes the name, not necessarily the object
- Two names can share one object; mutations show through both

UNDER THE HOOD

- A name is a reference; `b = a` never copies the object
- Every object counts its references: `sys.getrefcount(x)`
- The count hitting 0 frees the object immediately

ASSIGNMENT FORMS

```
count = 10                      # bind a name to an int
count = "ten"                   # rebind to a str (allowed)
name, age = "Ana", 30           # multiple assignment
x = y = 0                       # chained: both refer to 0
a, b = 1, 2
a, b = b, a                     # swap: a=2, b=1
first, *rest = [1, 2, 3, 4]     # first=1, rest=[2, 3, 4]
MAX_RETRIES = 3                 # constant by convention
total: int = 0                  # annotation, not enforced
nums = [1, 2]
alias = nums                    # same list, two names
alias.append(3); print(nums)    # [1, 2, 3]
del count; print(count)         # NameError
```

CODE EXPLAINED

- → `a, b = 1, 2` — tuple packing on the right, unpacking on the left
- → `*rest` — collects leftover items into a list
- → `x: int = 0` — type hint for tools; runtime ignores it
- → `alias = nums` — copies the reference, not the list
- → `del` — unbinds the name; later use raises `NameError`

THE PYTHONIC WAY

- ✗ ~~`temp = a; a = b; b = temp`~~ → ✓ `a, b = b, a`
- ✗ ~~`x = point[0]; y = point[1]`~~ → ✓ `x, y = point`
- ✗ ~~`for i in range(3): ...`~~ ~~(unused i)~~ → ✓ `for _ in range(3):`

PEP 8 NAMING

STYLE	USED FOR
`snake_case`	Variables, functions
`UPPER_CASE`	Constants
`PascalCase`	Classes
`_name`	Internal by convention
`__name`	Name-mangled class attribute
`class_`	Avoids keyword clash

VALID & INVALID NAMES

NAME	RESULT
`total_2`	✔ valid
`_temp`	✔ valid
`café`	✔ Unicode allowed
`2total`	✖ starts with digit
`my-var`	✖ – is minus
`class`	✖ reserved keyword

ASSIGNMENT FORMS

SYNTAX	MEANING
`x = 1`	Simple
`a, b = 1, 2`	Unpacking
`x = y = 0`	Chained
`a, *b = seq`	Starred
`x: int = 1`	Annotated
`(n := 10)`	Walrus (expression)

⚠ COMMON MISTAKES

- `a = b = []` then appending — both names share one list
- Naming variables `list`, `str`, `id` — shadows the built-ins
- Expecting `b = a` to copy a list — use `a.copy()` or `list(a)`
- Reading a name before assigning it — `NameError` or `UnboundLocalError`

✓ TIPS & BEST PRACTICES

- Pick descriptive names instead of explaining them in comments
- Swap with `a, b = b, a` — no temp variable needed
- Use `_` for throwaway values: `for _ in range(3):`
- Check sharing with `a is b` or `id(a) == id(b)`

QUICK SUMMARY

- `=` — binds a name, never copies
- `a, b = b, a` — unpack and swap
- `UPPER_CASE` — constant by convention only
- `del x` — removes the name

Integers

ARBITRARY-PRECISION INTEGERS

`int` stores whole numbers with **arbitrary precision**: there is no overflow, only memory limits. Literals can be decimal, hex, octal, or binary, and underscores improve readability. `int()` converts strings and floats. `/` always returns a float, while `//` and `%` keep integer results.

NOTES

- No `MAX_INT`; `sys.maxsize` is the largest container index
- Leading zeros like `012` are a `SyntaxError`
- `bool` is a subclass of `int`: `True == 1`
- int → str conversion is capped at 4300 digits by default (3.11+)
- `//` floors toward negative infinity, not toward zero

UNDER THE HOOD

- Ints from -5 to 256 are cached singletons
- Big ints are arrays of 30-bit digits — no overflow
- `sys.getsizeof(2**100)` grows with the value

INTEGER OPERATIONS

```
n = 1_000_000                   # 1000000 (underscores ignored)
0xFF, 0o17, 0b1010              # (255, 15, 10)
2 ** 100                        # 1267650600228229401496703205376
int("42"), int("  -7 ")         # (42, -7)  whitespace ok
int("ff", 16)                   # 255
int("0b101", 0)                 # 5  (base 0 reads the prefix)
int(3.99), int(-3.99)           # (3, -3)  truncates toward zero
hex(255), oct(8), bin(5)        # ('0xff', '0o10', '0b101')
(255).bit_length()              # 8
abs(-5), pow(2, 10, 1000)       # (5, 24)
7 // 2, -7 // 2, 7 % 3          # (3, -4, 1)
divmod(17, 5)                   # (3, 2)
f"{n:,}"                        # '1,000,000'
```

CODE EXPLAINED

- → `1_000_000` — underscores are visual only
- → `int(s, base)` — parse bases 2–36; base `0` uses the prefix
- → `int(float)` — truncates; use `round()` to round
- → `pow(b, e, m)` — fast modular exponent
- → `(255).bit_length()` — parentheses needed on a literal

PYTHON VERSIONS

- **3.6** — `_` digit separators: `1_000_000`
- **3.10** — `int.bit_count()`
- **3.11** — int ↔ str conversion capped at 4300 digits

THE PYTHONIC WAY

- ✗ ~~`1000000`~~ → ✓ `1_000_000`
- ✗ ~~`int(math.sqrt(n))`~~ → ✓ `math.isqrt(n)`
- ✗ ~~`hex(n)[2:]`~~ → ✓ `f"{n:x}"`

LITERALS

LITERAL	BASE
`42`	Decimal
`0x2A`	Hexadecimal
`0o52`	Octal
`0b101010`	Binary
`1_000`	Grouped digits

CONVERSIONS

CALL	RESULT
`hex(42)`	'0x2a'
`bin(5)`	'0b101'
`f"{5:08b}"`	'00000101'
`f"{255:X}"`	'FF'
`f"{1234567:,}"`	'1,234,567'

INT METHODS

METHOD	RETURNS
`bit_length()`	Bits needed
`bit_count()`	Count of 1 bits (3.10+)
`to_bytes(2, 'big')`	Bytes object
`int.from_bytes(b, 'big')`	Int from bytes
`math.isqrt(n)`	Exact integer sqrt

⚠ COMMON MISTAKES

- `int("3.5")` — `ValueError`; use `int(float(s))`
- Expecting `int()` to round — it truncates: `int(2.9)` is `2`
- `-7 // 2` is `-4`, not `-3` — floor division rounds down
- Writing `010` for octal — `SyntaxError`; use `0o10`

✓ TIPS & BEST PRACTICES

- Group digits in big literals: `10_000_000`
- Use `math.isqrt()` instead of `int(math.sqrt())` for exact roots
- `f"{n:,}"` adds thousands separators
- `int(s, 0)` accepts `0x`, `0o`, `0b` input automatically

QUICK SUMMARY

- `int` — unbounded whole numbers
- `0x 0o 0b` — hex, octal, binary literals
- `int(s, base)` — parse from text
- `// % divmod` — integer division tools

Floats

IEEE 754 DOUBLES

`float` is a 64-bit **IEEE 754 double** with about 15–17 significant digits and a range near ±1.8e308. Many decimals such as `0.1` have no exact binary form, so arithmetic produces tiny rounding errors. Compare floats with `math.isclose()` and use `decimal.Decimal` when exact decimal results matter, as with money.

NOTES

- `/` always returns a float: `4 / 2` is `2.0`
- `round()` uses banker's rounding (half to even)
- `nan` is never equal to anything, including itself
- Limits live in `sys.float_info`
- `float("inf")` and `math.inf` are equivalent

UNDER THE HOOD

- A `float` is an IEEE 754 double: 53 bits of precision
- `0.1` is stored as `0.1000000000000000055…`
- `(0.1).hex()` shows the exact bits

FLOAT BEHAVIOR

```
3.14, 2., .5, 1e3, 2.5e-4       # (3.14, 2.0, 0.5, 1000.0, 0.00025)
0.1 + 0.2                       # 0.30000000000000004
0.1 + 0.2 == 0.3                # False
import math
math.isclose(0.1 + 0.2, 0.3)    # True
round(2.5), round(3.5)          # (2, 4)  banker's rounding
round(2.675, 2)                 # 2.67  (stored as 2.67499...)
float("inf") > 1e308            # True
nan = float("nan")
nan == nan, math.isnan(nan)     # (False, True)
(7.0).is_integer()              # True
from decimal import Decimal
Decimal("0.1") + Decimal("0.2")  # Decimal('0.3')
```

CODE EXPLAINED

- → `1e3` — scientific notation; always a float
- → `math.isclose(a, b)` — relative tolerance 1e-09 by default
- → `round(x, n)` — rounds to n decimals, half to even
- → `math.isnan()` — the only reliable `nan` test
- → `Decimal("0.1")` — build from a string to stay exact

PYTHON VERSIONS

- **3.9** — `math.nextafter()` and `math.ulp()`
- **3.12** — `sum()` of floats uses compensated summation

THE PYTHONIC WAY

- ✗ ~~`a == b` for floats~~ → ✓ `math.isclose(a, b)`
- ✗ ~~`float` for money~~ → ✓ `Decimal("19.99")`
- ✗ ~~`sum(floats)` for exact totals~~ → ✓ `math.fsum(floats)`

SPECIAL VALUES

EXPRESSION	VALUE
`float("inf")`	Positive infinity
`float("-inf")`	Negative infinity
`float("nan")`	Not a number
`sys.float_info.max`	1.7976931348623157e+308
`sys.float_info.epsilon`	2.220446049250313e-16

ROUNDING

CALL	RESULT
`round(2.567, 2)`	2.57
`math.floor(-2.5)`	-3
`math.ceil(2.1)`	3
`math.trunc(-2.7)`	-2
`f"{2.5:.0f}"`	'2' (half to even)

EXACT ALTERNATIVES

TOOL	USE
`Decimal("1.10")`	Exact decimal math
`Fraction(1, 3)`	Exact rationals
`d.quantize(cents)`	Round money to 0.01
`math.fsum(xs)`	Accurate float sums

⚠ COMMON MISTAKES

- Comparing floats with `==` — use `math.isclose()`
- `Decimal(0.1)` — inherits the float error; pass `"0.1"`
- Expecting `round(2.5)` to be `3` — it's `2`
- Storing currency as float — use `Decimal` or integer cents

✓ TIPS & BEST PRACTICES

- Format for display with `f"{x:.2f}"`; keep full precision internally
- Pass `abs_tol=1e-9` to `isclose()` when comparing to zero
- `math.fsum([0.1] * 10)` gives `1.0`; `sum()` gives `0.9999999999999999`
- Read big numbers with `float("1e6")` or `1e6`

QUICK SUMMARY

- `float` — 64-bit IEEE 754 double
- `math.isclose()` — safe equality
- `inf` / `nan` — special values
- `Decimal` — exact decimal arithmetic

Booleans & Truthiness

TRUE, FALSE, AND TRUTH VALUES

`bool` has exactly two values, `True` and `False`, and is a subclass of `int` (`True == 1`). Every object also has a **truth value** used by `if`, `while`, `and`, `or`, and `not`. Zero, `None`, and empty containers are **falsy**; almost everything else is **truthy**. `bool(x)` shows an object's truth value.

NOTES

- Capitalized: `True`, `False` (`true` is a `NameError`)
- `and` / `or` return one of their operands, not always a bool
- `not` always returns a real bool
- Classes set truthiness with `__bool__` or `__len__`
- `sum()` over bools counts the `True` values

UNDER THE HOOD

- `True` and `False` are the only two `bool` objects
- `bool` subclasses `int`: `True + True` is `2`
- Truthiness tries `__bool__`, then `__len__`

TRUTHINESS IN PRACTICE

```
bool(0), bool(0.0), bool("")   # (False, False, False)
bool([]), bool({}), bool(None) # (False, False, False)
bool(42), bool("0"), bool([0]) # (True, True, True)
True + True                    # 2
sum([True, False, True])       # 2  (count Trues)
isinstance(True, int)          # True
items = []
if not items:                  # idiomatic empty check
    print("empty")             # empty
name = "" or "Guest"           # 'Guest'
5 > 3 and "yes"                # 'yes'
1 < 5 < 10                     # True  (chained)
```

CODE EXPLAINED

→ `bool(x)` — the truth value Python uses in conditions

→ `if not items:` — true for any empty container

→ `a or b` — returns `a` if truthy, else `b`

→ `a and b` — returns `a` if falsy, else `b`

→ `1 < 5 < 10` — means `1 < 5 and 5 < 10`

THE PYTHONIC WAY

✗ ~~`if flag == True:`~~ → ✓ `if flag:`

✗ ~~`if len(items) > 0:`~~ → ✓ `if items:`

✗ ~~`count = 0; for x in xs: if x > 0: count += 1`~~ → ✓ `sum(x > 0 for x in xs)`

FALSY VALUES

VALUE	TYPE
`False`	bool
`None`	NoneType
`0, 0.0, 0j`	Numeric zeros
`""`	Empty str
`[] () {} set()`	Empty containers
`range(0)`	Empty range

TRUTHY SURPRISES

VALUE	BOOL()
`"0"`	True
`"False"`	True
`" "`	True (a space)
`[0]`	True
`-1`	True
`float("nan")`	True

BOOLEAN HELPERS

CALL	RESULT
`any([0, 1])`	True
`all([1, 0])`	False
`any([])`	False
`all([])`	True (vacuous)
`not 0`	True

⚠ COMMON MISTAKES

- `if x == True:` — just write `if x:`
- `bool("False")` is `True` — compare the text instead
- `if not x:` when `0` is valid — use `if x is None:`
- Writing `true` or `TRUE` — `NameError`

✓ TIPS & BEST PRACTICES

- Test emptiness with `if not seq:`, not `len(seq) == 0`
- Use `any()` / `all()` with a generator for checks over items
- `x or default` works only when falsy values are invalid input
- Give custom classes `__bool__` for meaningful truthiness

QUICK SUMMARY

- `True` / `False` — subclass of `int`
- falsy — `0`, `None`, empty containers
- `and` / `or` — return an operand
- `any()` / `all()` — test many values

None & Type Checking

THE ABSENCE OF A VALUE

`None` is the single instance of `NoneType` and means "no value". Functions without a `return` give `None`. Because there is exactly one `None`, test it with `is None`. For types, `type()` returns the exact class, `isinstance()` also accepts subclasses and is usually preferred, and `hasattr()` checks for a capability.

NOTES

- `None` is falsy but not equal to `0`, `""`, or `False`
- `None < 1` raises `TypeError`
- Hint optional values as `int | None` (3.10+)
- `type(None)` is `NoneType`
- Python favors duck typing over type checks

UNDER THE HOOD

- `None` is one shared object, so `is None` is safe
- A function with no `return` returns that object
- `type(None)()` returns the same `None`

NONE & TYPE CHECKS

```
def find(items, target):
    for i, x in enumerate(items):
        if x == target:
            return i
    # falls off the end → returns None
result = find([5, 7], 9)
result is None                  # True
result == 0                     # False
type(3), type("a")              # (<class 'int'>, <class 'str'>)
type(True) is int               # False (it's bool)
isinstance(True, int)           # True  (bool subclasses int)
isinstance(3.0, (int, float))   # True  (tuple of types)
hasattr("abc", "upper")         # True
callable(len)                   # True
```

CODE EXPLAINED

→ `is None` — identity test; fastest and lint-clean

→ implicit return — any function without `return` gives `None`

→ `type(x) is T` — exact class match only

→ `isinstance(x, (A, B))` — any of several types

→ `hasattr()` / `callable()` — check capabilities

PYTHON VERSIONS

- **3.10** — `int | None` in hints and `isinstance(x, int | str)`

THE PYTHONIC WAY

✗ ~~`if x == None:`~~ → ✓ `if x is None:`

✗ ~~`type(x) == int`~~ → ✓ `isinstance(x, int)`

✗ ~~`Optional[int]`~~ → ✓ `int | None`

NONE CHECKS

CODE	NOTE
`x is None`	Preferred
`x is not None`	Preferred negation
`x == None`	Works; linters flag it
`if not x:`	Also catches 0 and empty
`x if x is not None else d`	Default when None

TYPE VS ISINSTANCE

CALL	RESULT
`type(True) is int`	False
`isinstance(True, int)`	True
`issubclass(bool, int)`	True
`type(3).__name__`	'int'

WHERE NONE APPEARS

SOURCE	RETURNS
`no return`	None
`d.get(missing)`	None
`lst.sort()`	None (in place)
`print()`	None
`re.match() fails`	None

⚠ COMMON MISTAKES

- `lst = lst.sort()` — `lst` becomes `None`; use `sorted()`
- `if x == None:` — use `is None`
- `if not x:` when `0` or `""` are valid values
- `type(x) == int` — misses subclasses; use `isinstance`

✓ TIPS & BEST PRACTICES

- Use `None` as the default for mutable args: `def f(items=None):`
- Pass a tuple to `isinstance` to accept several types
- Prefer trying an operation (EAFP) over checking types first
- Annotate optional returns as `-> int | None`

QUICK SUMMARY

- `None` — single no-value object
- `is None` — the correct test
- `isinstance()` — type check with inheritance
- `hasattr()` — capability check

Type Conversion & Casting

EXPLICIT CONVERSION

Python converts values by **calling the target type**: `int()`, `float()`, `str()`, `bool()`, `list()`, and so on. Implicit conversion only happens for numeric widening in arithmetic (`int` → `float` → `complex`). Python never joins `str` and `int` automatically. Invalid conversions raise `ValueError` (bad content) or `TypeError` (wrong type).

NOTES

- `int(float)` truncates toward zero
- `str(x)` calls `x.__str__()`
- `bool(s)` checks emptiness, not the text
- `list("abc")` splits a string into characters
- `ord()` / `chr()` convert characters ↔ code points

CONVERSIONS

```
int("42"), float("3.5")          # (42, 3.5)
int(3.9), float(7)               # (3, 7.0)
str(42) + "!"                    # '42!'
int("3.5")                       # ValueError
int(float("3.5"))                # 3
1 + 2.0                          # 3.0  (int widened)
"1" + 1                          # TypeError
list("abc")                      # ['a', 'b', 'c']
tuple([1, 2]), set([1, 1, 2])    # ((1, 2), {1, 2})
dict([("a", 1), ("b", 2)])       # {'a': 1, 'b': 2}
ord("A"), chr(66)                # (65, 'B')
"5" * 3, int("5") * 3            # ('555', 15)
```

CODE EXPLAINED

→ `int("3.5")` — fails; the string must be an integer literal

→ `1 + 2.0` — the only automatic conversion is numeric widening

→ `set([...])` — conversion also removes duplicates

→ `dict(pairs)` — needs an iterable of 2-item pairs

→ `"5" * 3` — string repetition, not math

THE PYTHONIC WAY

✗ ~~`if s.isdigit(): n = int(s)`~~ → ✓ `try: n = int(s)` + `except ValueError`

✗ ~~`"Age: " + str(age)`~~ → ✓ `f"Age: {age}"`

✗ ~~`bool("False")`~~ → ✓ `s.lower() == "true"`

SCALAR CONVERSIONS

CALL	RESULT
`int("1f", 16)`	31
`float("1e3")`	1000.0
`str(3.0)`	'3.0'
`round(3.7)`	4
`bool("")`	False
`complex("1+2j")`	(1+2j)

CONTAINER CONVERSIONS

CALL	RESULT
`list((1, 2))`	[1, 2]
`"a,b".split(",")`	['a', 'b']
`"-".join(["a","b"])`	'a-b'
`list(d)`	Dict keys
`list(d.items())`	Key-value tuples

CONVERSION ERRORS

CALL	RAISES
`int("abc")`	ValueError
`float("")`	ValueError
`int(None)`	TypeError
`int([1])`	TypeError
`int("1,000")`	ValueError

⚠ COMMON MISTAKES

- `"Total: " + 5` — `TypeError`; use `str(5)` or an f-string
- `int("3.5")` — `ValueError`; go through `float()` first
- `bool("False")` is `True` — compare `s.lower() == "true"`
- `int("1,000")` fails — remove commas: `s.replace(",", "")`

✓ TIPS & BEST PRACTICES

- Wrap conversions of outside data in `try/except ValueError`
- `str.isdigit()` rejects `"-5"` and `"3.5"`; prefer try/except
- `int(round(x))` rounds before converting
- `repr(x)` shows the exact value, including quotes

QUICK SUMMARY

- `T(x)` — convert by calling the type
- `ValueError` — bad content
- `TypeError` — wrong type
- numeric widening — the only implicit conversion

User Input & Parsing

READING FROM THE KEYBOARD

`input(prompt)` shows the prompt, reads one line from standard input, and returns it as a **str** without the trailing newline. Always convert and validate, because users type unexpected things. Wrap conversion in `try/except` and loop until the value is valid. `EOFError` signals that input ended (Ctrl+D, or a piped file ran out).

NOTES

- `input()` always returns `str`
- The prompt prints without a trailing newline
- Ctrl+C raises `KeyboardInterrupt`
- `getpass.getpass()` hides typed passwords
- For script options use `sys.argv` or `argparse`

VALIDATED INPUT

```
name = input("Name: ").strip()          # 'Ana'
age = int(input("Age: "))               # ValueError on "abc"

def ask_int(prompt, lo, hi):
    while True:
        raw = input(prompt).strip()
        try:
            n = int(raw)
        except ValueError:
            print(f"{raw!r} is not a number")
            continue
        if lo <= n <= hi:
            return n
        print(f"Enter {lo}-{hi}")

qty = ask_int("Qty (1-10): ", 1, 10)
x, y = map(int, input("x y: ").split())  # "3 4" → 3, 4
```

CODE EXPLAINED

- → `.strip()` — removes stray spaces and tabs
- → `while True` + `return` — loop until valid
- → `continue` — re-prompt after a bad value
- → `map(int, s.split())` — convert several numbers at once

THE PYTHONIC WAY

- ✗ ~~Repeated `input()` + `int()` blocks~~ → ✓ **one** `ask_int(prompt)` **helper**
- ✗ ~~`raw = input(); raw = raw.strip()`~~ → ✓ `input(">").strip()`
- ✗ ~~Parsing `sys.argv` by hand~~ → ✓ `argparse`

PARSING PATTERNS

CODE	GETS
`int(input())`	One integer
`float(input())`	One float
`input().split()`	List of words
`input().lower() in ("y", "yes")`	Yes/no as bool
`[int(v) for v in s.split(",")]`	CSV numbers

EXCEPTIONS

EXCEPTION	CAUSE
`ValueError`	Text won't convert
`EOFError`	Input stream ended
`KeyboardInterrupt`	User pressed Ctrl+C
`ValueError (unpack)`	Wrong number of values

STRING CHECKS

METHOD	TRUE FOR
`isdigit()`	Digits only
`isalpha()`	Letters only
`isalnum()`	Letters or digits
`lstrip("-").isdigit()`	Negative ints too
`strip() == ""`	Blank entry

⚠ COMMON MISTAKES

- `input() + 1` — `TypeError`; convert first
- Comparing `age > 18` while `age` is still a str
- Relying on `isdigit()` — rejects `"-5"` and `"3.5"`
- `a, b = input().split()` with three words — `ValueError`

✓ TIPS & BEST PRACTICES

- Show the expected format in the prompt: `"Date (YYYY-MM-DD): "`
- Put validation in a reusable function like `ask_int()`
- Use `getpass.getpass()` for passwords
- Catch `EOFError` when your script may read piped input

QUICK SUMMARY

- `input()` — returns a str
- `int(input())` — convert immediately
- `try/except ValueError` — handle bad input
- `while True` — re-prompt until valid

String Basics

quotes · triple quotes · raw strings · concatenation · len() · immutability

IMMUTABLE UNICODE TEXT

A `str` is an **immutable sequence of Unicode code points**. Single and double quotes are interchangeable, triple quotes span lines, and prefixes change meaning: `r` for raw, `f` for formatted, `b` for bytes. Because strings are immutable, every method that seems to modify a string actually returns a new one.

NOTES

- No separate char type: `"a"` is a `str` of length 1
- Adjacent literals join: `"a" "b"` → `"ab"`
- Strings are hashable, so they work as dict keys
- `str` holds text; `bytes` holds raw data — `encode()` / `decode()` convert
- `len()` counts code points, not bytes

UNDER THE HOOD

- Strings use 1, 2, or 4 bytes per char (PEP 393)
- Identifier-like literals are interned: `sys.intern()`
- A `str` caches its hash after the first `hash()`

STRING LITERALS & BASICS

```
s1 = 'single'; s2 = "double"        # same type
quote = "It's fine"                 # mix quotes to skip escapes
path = r"C:\new\table"              # raw: backslashes kept
text = """Line one
Line two"""                         # newline is part of the string
len("héllo")                        # 5
"ab" + "cd", "ab" * 3               # ('abcd', 'ababab')
"py" "thon"                         # 'python'
s = "cat"
s[0] = "b"                          # TypeError: immutable
s = "b" + s[1:]                     # 'bat'  (new string)
"ell" in "hello"                    # True
"café".encode("utf-8")              # b'caf\xc3\xa9'
b"caf\xc3\xa9".decode()             # 'café'
```

CODE EXPLAINED

- → `r"..."` — backslashes are not escapes (paths, regex)
- → `"""..."""` — multi-line literal; also used for docstrings
- → `s[0] = ...` — fails; build a new string instead
- → `encode()` — str → bytes; `decode()` — bytes → str

PYTHON VERSIONS

- **3.0** — `str` is Unicode; `bytes` is a separate type
- **3.3** — compact strings (PEP 393); `u""` prefix allowed again

THE PYTHONIC WAY

- ✗ ~~`s = s + part` in a loop~~ → ✓ `"".join(parts)`
- ✗ ~~`"C:\\temp\\new"`~~ → ✓ `r"C:\temp\new"` or `Path`
- ✗ ~~`a.lower() == b.lower()`~~ → ✓ `a.casefold() == b.casefold()`

LITERAL FORMS

LITERAL	MEANING
`'...'` / `"..."`	Plain string
`"""..."""`	Multi-line
`r"..."`	Raw (no escapes)
`f"..."`	Formatted
`b"..."`	Bytes
`rf"..."`	Raw + formatted

BASIC OPERATIONS

CODE	RESULT
`len(s)`	Length
`s + t`	Concatenate
`s * 3`	Repeat
`x in s`	Substring test
`"Apple" < "apple"`	True (code point order)

UNICODE

CODE	RESULT
`ord("é")`	233
`chr(9731)`	'☃'
`"\u00e9"`	'é'
`"\N{BULLET}"`	'•'
`len("👍")`	1

⚠ COMMON MISTAKES

- `s[0] = "x"` — strings are immutable; slice and rebuild
- `+=` in a loop to build big text — slow; collect parts and `"".join()`
- `r"C:\"` — a raw string can't end in one backslash
- Sorting mixed-case words — uppercase sorts before lowercase

✓ TIPS & BEST PRACTICES

- Use `textwrap.dedent()` to clean indented triple-quoted text
- Compare case-insensitively with `a.casefold() == b.casefold()`
- Prefer `pathlib.Path` over raw path strings
- Always pass `encoding="utf-8"` when crossing to bytes or files

QUICK SUMMARY

- `str` — immutable Unicode text
- `r""` / `f""` / `b""` — raw, formatted, bytes
- `"".join(parts)` — efficient building
- `encode()` / `decode()` — str ↔ bytes

String Formatting

F-STRINGS & FORMAT SPECS

f-strings (3.6+) embed expressions inside {} and are the standard way to format text. After a colon, a **format spec** controls fill, alignment, width, grouping, precision, and type: `{value:fill align width , .precision type}`. `str.format()` uses the same spec language; % formatting is legacy but still common in older code and `logging`.

NOTES

- `{x!r}` uses `repr()`, `{x!s}` uses `str()`
- `{x=}` prints the expression and value (3.8+)
- `{{` and `}}` produce literal braces
- Nested specs work: `f"{x:{width}}"`
- 3.12+ allows the same quote type inside f-string braces

FORMATTING EXAMPLES

```
name, n, pi = "Ana", 1234.5, 3.14159
f"Hi {name}!"                    # 'Hi Ana!'
f"{pi:.2f}"                      # '3.14'
f"{n:,.2f}"                      # '1,234.50'
f"{0.256:.1%}"                   # '25.6%'
f"[{name:<6}]"                   # '[Ana   ]'
f"[{name:>6}]"                   # '[   Ana]'
f"[{name:*^7}]"                  # '[**Ana**]'
f"{42:05d}"                      # '00042'
f"{255:#x}, {5:08b}"             # '0xff, 00000101'
f"{1e6:.2e}"                     # '1.00e+06'
f"{name!r}", f"{n = }"           # ("'Ana'", 'n = 1234.5')
"{} is {}".format("x", 3)        # 'x is 3'
"%s has %d" % ("Ana", 3)         # 'Ana has 3'
```

CODE EXPLAINED

- → `.2f` — fixed point, 2 decimals
- → `,` — thousands separator (`_` also works)
- → `< > ^` — left, right, center within the width
- → `*^7` — fill with `*`, center, width 7
- → `#x` — hex with `0x` prefix; `08b` — zero-padded binary

PYTHON VERSIONS

- **3.6** — f-strings (PEP 498)
- **3.8** — `f"{x=}"` debug form
- **3.12** — same quotes and comments inside braces (PEP 701)
- **3.14** — t-strings: `t"..."` templates (PEP 750)

THE PYTHONIC WAY

- ✗ ~~`"%s is %d" % (name, age)`~~ → ✓ `f"{name} is {age}"`
- ✗ ~~`"{} {}".format(a, b)`~~ ~~for simple text~~ → ✓ `f"{a} {b}"`
- ✗ ~~`str(round(x, 2))`~~ → ✓ `f"{x:.2f}"`

SPEC PARTS (IN ORDER)

PART	OPTIONS
`fill`	Any character
`align`	< > ^ =
`sign`	+ - space
`width`	Minimum width
`grouping`	, or _
`.precision`	Digits after point
`type`	See next table

TYPE CODES

CODE	MEANING
`d`	Integer
`f`	Fixed-point
`e`	Scientific
`%`	Percent (×100)
`x / X`	Hex
`b`	Binary
`g`	General (shortest)

DATES IN F-STRINGS

SPEC	OUTPUT
`f"{d:%Y-%m-%d}"`	2026-10-01
`f"{d:%b %d, %Y}"`	Oct 01, 2026
`f"{d:%H:%M}"`	14:05
`f"{d:%A}"`	Thursday

⚠ COMMON MISTAKES

- Forgetting the `f` prefix — braces print literally
- `f"{name:.2f}"` on a str — `ValueError`
- `"%s" % (x)` with a tuple `x` — wrap it: `(x,)`
- Formatting to a string, then doing math with it

✓ TIPS & BEST PRACTICES

- Debug with `f"{total=}"`
- Build tables with fixed widths: `f"{item:<20} {price:>8.2f}"`
- Use `string.Template` for user-supplied templates
- `logging` uses `%` style lazily: `log.info("%s", x)`

QUICK SUMMARY

- `f"{x}"` — embed expressions
- `{x:.2f}` — precision
- `{x:>10}` — width and alignment
- `{x:,}` — thousands separators

String Methods — Search

FINDING SUBSTRINGS

Search methods locate substrings without regular expressions. `in` gives a yes/no answer, `find()` returns an index or −1, and `index()` raises `ValueError` when nothing matches. `startswith()` and `endswith()` accept a tuple of options. All of them are case-sensitive, and most accept optional `start` and `end` positions.

NOTES

- `rfind()` / `rindex()` search from the right
- `count()` counts non-overlapping matches
- For patterns, use the `re` module (Sheet 83)
- Case-insensitive search: lower both sides first
- Index results are 0-based

SEARCH METHODS

```
s = "banana split"
"nan" in s                      # True
s.find("an")                    # 1
s.find("an", 2)                 # 3  (search from index 2)
s.rfind("an")                   # 3
s.find("xyz")                   # -1
s.index("xyz")                  # ValueError
s.count("a")                    # 3
s.count("ana")                  # 1  (non-overlapping)
s.startswith("ban")             # True
s.endswith((".txt", "split"))   # True  (tuple of options)
"Hello".lower().find("h")       # 0  (case-insensitive)
s.split().index("split")        # 1  (list.index)
```

CODE EXPLAINED

→ `in` — best for a plain existence check

→ `find()` — safe position lookup, `-1` if missing

→ `index()` — same, but raises on a miss

→ tuple argument — `endswith()` tries each option

THE PYTHONIC WAY

✗ ~~`if s.find("x") != -1:`~~ → ✓ `if "x" in s:`

✗ ~~`s.endswith(".jpg") or s.endswith(".png")`~~ → ✓ `s.endswith((".jpg", ".png"))`

✗ ~~`s[:4] == "http"`~~ → ✓ `s.startswith("http")`

SEARCH METHODS

METHOD	RETURNS
`s.find(sub)`	First index or -1
`s.rfind(sub)`	Last index or -1
`s.index(sub)`	First index or ValueError
`s.rindex(sub)`	Last index or ValueError
`s.count(sub)`	Number of matches
`s.startswith(p)`	bool
`s.endswith(p)`	bool

CHARACTER TESTS

METHOD	TRUE WHEN
`isdigit()`	All digits
`isalpha()`	All letters
`isalnum()`	Letters or digits
`isspace()`	All whitespace
`isupper()`	All cased chars upper
`islower()`	All cased chars lower
`istitle()`	Title Case

RANGE ARGUMENTS

CALL	SEARCHES
`s.find(x, 5)`	From index 5
`s.find(x, 2, 8)`	Indexes 2–7
`s.count(x, 3)`	From index 3
`s.startswith(p, 4)`	Starting at 4

⚠ COMMON MISTAKES

- `if s.find(x):` — `0` is falsy and `-1` is truthy; use `in`
- Calling `index()` without `try` — crashes on a miss
- Forgetting case: `"Py" in "python"` is `False`
- Expecting `count()` to find overlapping matches

✓ TIPS & BEST PRACTICES

- Use `in` for yes/no, `find()` when you need the position
- Pass a tuple to `endswith()` to match several extensions
- Normalize with `casefold()` before searching user text
- Switch to `re.search()` once you need wildcards

QUICK SUMMARY

- `in` — contains test
- `find()` — index or -1
- `index()` — index or ValueError
- `startswith()` / `endswith()` — prefix/suffix tests

String Methods — Transform

CHANGING TEXT

Transform methods return a **new string** and leave the original untouched. They cover case changes, trimming whitespace, replacing text, splitting a string into a list, and joining a list back into a string. `split()` and `join()` are opposites and appear in nearly every text-processing task.

NOTES

- Assign the result: `s = s.strip()`
- `split()` with no argument splits on any whitespace run
- `strip(chars)` removes characters, not a substring
- `removeprefix()` / `removesuffix()` (3.9+) remove exact text
- `join()` needs every item to be a str

TRANSFORM METHODS

```
s = "  Hello, World  "
s.strip()                       # 'Hello, World'
"xxhixx".strip("x")             # 'hi'
"Hello".upper(), "Hello".lower()  # ('HELLO', 'hello')
"hello world".title()           # 'Hello World'
"a-b-c".replace("-", "+")       # 'a+b+c'
"a-b-c".replace("-", "", 1)     # 'ab-c'  (first only)
"a,b,,c".split(",")             # ['a', 'b', '', 'c']
"  a  b ".split()               # ['a', 'b']
"k=v=w".split("=", 1)           # ['k', 'v=w']
", ".join(["x", "y", "z"])      # 'x, y, z'
"one\ntwo".splitlines()         # ['one', 'two']
"report.pdf".removesuffix(".pdf")  # 'report'
"7".zfill(3), "ab".center(6, "*")  # ('007', '**ab**')
```

CODE EXPLAINED

- → `strip()` — trims both ends; `lstrip()` / `rstrip()` one end
- → `replace(old, new, count)` — optional max replacements
- → `split(sep, maxsplit)` — limit the number of splits
- → `sep.join(items)` — the separator calls `join`

PYTHON VERSIONS

- **3.9** — `removeprefix()` and `removesuffix()`

THE PYTHONIC WAY

- ✗ ~~`name.strip(".txt")`~~ → ✓ `name.removesuffix(".txt")`
- ✗ ~~`s.split(" ")`~~ → ✓ `s.split()`
- ✗ ~~`", ".join([str(n) for n in nums])`~~ → ✓ `", ".join(map(str, nums))`

CASE

METHOD	"HELLO WORLD" →
`upper()`	HELLO WORLD
`lower()`	hello world
`title()`	Hello World
`capitalize()`	Hello world
`swapcase()`	HELLO wORLD
`casefold()`	hello world (aggressive)

TRIM & PAD

METHOD	EFFECT
`strip(chars)`	Trim both ends
`ljust(w)`	Pad right to width w
`rjust(w)`	Pad left to width w
`center(w, c)`	Center, fill with c
`zfill(w)`	Pad with zeros
`expandtabs(4)`	Tabs → spaces

SPLIT & JOIN

METHOD	RETURNS
`split(sep)`	List of parts
`rsplit(sep, 1)`	Split from the right
`splitlines()`	List of lines
`partition("=")`	(head, sep, tail)
`sep.join(xs)`	One string
`removeprefix(p)`	Without prefix

⚠ COMMON MISTAKES

- `s.upper()` without assigning — `s` is unchanged
- `"file.txt".strip(".txt")` → `'file'` by luck; it strips characters — use `removesuffix()`
- `",".join([1, 2])` — `TypeError`; use `map(str, items)`
- `split(" ")` keeps empty strings on double spaces; `split()` doesn't

✓ TIPS & BEST PRACTICES

- Chain calls: `s.strip().lower().replace(" ", "_")`
- `partition()` is cleaner than `split(sep, 1)` for one cut
- Use `str.translate(str.maketrans(...))` for many single-char swaps
- `textwrap.fill(s, 60)` wraps long text

QUICK SUMMARY

- `strip()` — trim whitespace
- `replace()` — swap substrings
- `split()` / `join()` — str ↔ list
- `upper()` `lower()` `title()` — change case

String Slicing & Indexing

INDEXES AND SLICES

Strings are sequences: `s[i]` returns one character, and **negative indexes** count from the end. A **slice** `s[start:stop:step]` returns a new string from `start` up to, but not including, `stop`. Omitted bounds default to the ends. Out-of-range slices are clamped silently, while an out-of-range index raises `IndexError`.

NOTES

- `s[0]` is first, `s[-1]` is last
- `len(s[a:b]) == b - a` when both are in range
- A negative step walks right to left
- Slicing never raises an error
- The same syntax works on lists and tuples

INDEX MAP & SLICES

```
s = "PYTHON"
#     P  Y  T  H  O  N
#     0  1  2  3  4  5
#    -6 -5 -4 -3 -2 -1
s[0], s[-1]                  # ('P', 'N')
s[1:4]                       # 'YTH'
s[:2], s[2:]                 # ('PY', 'THON')
s[-3:]                       # 'HON'
s[::2]                       # 'PTO'
s[::-1]                      # 'NOHTYP'
s[4:1:-1]                    # 'OHT'
s[10:]                       # ''  (clamped, no error)
s[10]                        # IndexError
```

CODE EXPLAINED

- → `s[1:4]` — indexes 1, 2, 3; stop is excluded
- → `s[-3:]` — the last three characters
- → `s[::2]` — every second character
- → `s[::-1]` — reversed copy
- → `s[4:1:-1]` — 4, 3, 2 walking backward

THE PYTHONIC WAY

- ✗ ~~`s[len(s) - 1]`~~ → ✓ `s[-1]`
- ✗ ~~`s[0:3]`~~ → ✓ `s[:3]`
- ✗ ~~`"".join(reversed(s))`~~ → ✓ `s[::-1]`

COMMON SLICES

SLICE	GETS
`s[:n]`	First n
`s[-n:]`	Last n
`s[n:]`	Drop first n
`s[:-n]`	Drop last n
`s[::-1]`	Reversed
`s[:]`	Full copy

SLICE RULES

PART	DEFAULT
`start`	0 (or end if step < 0)
`stop`	len(s) (or start if step < 0)
`step`	1
`stop`	Always excluded
`step = 0`	ValueError

SLICE OBJECTS

CODE	RESULT
`slice(1, 4)`	Reusable slice
`s[slice(1, 4)]`	'YTH'
`CODE = slice(0, 3)`	Named field
`s == s[::-1]`	Palindrome test

⚠ COMMON MISTAKES

- Expecting `stop` to be included — `s[0:3]` has 3 chars, not 4
- `s[len(s)]` — `IndexError`; the last index is `len(s) - 1`
- `s[1:4:-1]` — empty; direction and bounds disagree
- Assigning to a slice of a str — `TypeError` (immutable)

✓ TIPS & BEST PRACTICES

- Name slices for fixed-width records: `DATE = slice(0, 10)`
- `s[-1:]` returns `''` on empty strings instead of raising
- Reverse with `s[::-1]`; check palindromes with `s == s[::-1]`
- Practice slices on lists too — the rules are identical

QUICK SUMMARY

- `s[i]` — one character
- `s[a:b]` — a up to b (exclusive)
- `s[::-1]` — reverse
- negative indexes — count from the end

Arithmetic Operators

MATH OPERATORS

Python has seven arithmetic operators. `/` is **true division** and always returns a float. `//` is **floor division** and rounds toward negative infinity. `%` returns a remainder with the sign of the divisor. `**` raises to a power, groups right to left, and binds tighter than unary minus.

NOTES

- int op int → int, except `/`
- Any float operand makes the result a float
- `a == (a // b) * b + a % b` always holds
- Division by zero raises `ZeroDivisionError`
- `@` is matrix multiplication (NumPy), unused by built-ins

ARITHMETIC IN ACTION

```
7 + 2, 7 - 2, 7 * 2              # (9, 5, 14)
7 / 2, 6 / 3                     # (3.5, 2.0)  always float
7 // 2, -7 // 2                  # (3, -4)  floors
7 % 3, -7 % 3, 7 % -3            # (1, 2, -2)  sign of divisor
2 ** 10, 2 ** -1                 # (1024, 0.5)
2 ** 3 ** 2                      # 512  (2 ** 9, right to left)
-2 ** 2                          # -4   (** before unary -)
(-2) ** 2                        # 4
divmod(17, 5)                    # (3, 2)
7.5 // 2                         # 3.0  (float in, float out)
1 / 0                            # ZeroDivisionError
abs(-3), round(2.567, 1)         # (3, 2.6)
```

CODE EXPLAINED

- → `/` — use when you want a fraction
- → `//` — use for whole counts and indexes
- → `%` — remainder; great for even/odd and wrap-around
- → `-2 ** 2` — parsed as `-(2 ** 2)`
- → `divmod(a, b)` — quotient and remainder together

PYTHON VERSIONS

- **3.0** — `/` is true division; `//` floors
- **3.5** — `@` matrix-multiply operator

THE PYTHONIC WAY

✗ ~~`int(a / b)`~~ → ✓ `a // b`

✗ ~~`q = a // b; r = a % b`~~ → ✓ `q, r = divmod(a, b)`

✗ ~~`math.pow(2, 10)`~~ ~~for ints~~ → ✓ `2 ** 10`

OPERATORS (A = 7, B = 2)

OP	NAME	RESULT
`a + b`	Add	9
`a - b`	Subtract	5
`a * b`	Multiply	14
`a / b`	True divide	3.5
`a // b`	Floor divide	3
`a % b`	Modulo	1
`a ** b`	Power	49

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	**
2	+x −x ~x (unary)
3	* / // % @
4	+ −
5	Comparisons, then `not` and `or`

MATH BUILT-INS

CALL	RESULT
`abs(-4.5)`	4.5
`pow(2, 10, 1000)`	24
`round(7.456, 2)`	7.46
`sum([1, 2, 3])`	6
`max(3, 9, 4)`	9
`min([3, 9, 4])`	3

⚠ COMMON MISTAKES

- Expecting `7 / 2` to be `3` — use `//` for integer division
- Assuming `-7 // 2` is `-3` — it floors to `-4`
- `-2 ** 2` is `-4` — wrap the base: `(-2) ** 2`
- `-7 % 3` is `2`, not `-1` — use `math.fmod()` for C-style

✓ TIPS & BEST PRACTICES

- Even test: `n % 2 == 0`
- Wrap an index: `items[i % len(items)]`
- Split seconds: `mins, secs = divmod(total, 60)`
- Add parentheses whenever precedence isn't obvious

QUICK SUMMARY

- `/` — float division
- `//` — floor division
- `%` — remainder, divisor's sign
- `**` — power, right-associative

Comparison Operators

COMPARING VALUES

Comparison operators return a bool. == compares **values** by calling __eq__, not identity. Ordering operators work between compatible types; comparing an `int` with a `str` raises `TypeError`. Comparisons can be **chained**: `a < b < c` means `a < b and b < c`, with b evaluated only once.

NOTES

- Strings compare by Unicode code point
- Sequences compare item by item; the first difference decides
- `nan` is unequal to everything, itself included
- Sets use `<` / `>` for subset / superset
- `=` assigns; `==` compares

COMPARISON EXAMPLES

```
3 == 3.0                        # True  (value, not type)
"a" == "A"                      # False (case-sensitive)
[1, 2] == [1, 2]                # True  (element-wise)
3 != 4                          # True
"apple" < "banana"              # True  (lexicographic)
"Z" < "a"                       # True  (90 < 97)
[1, 2, 3] < [1, 3]              # True  (2 < 3 decides)
(1, "b") > (1, "a")             # True
1 < 5 < 10                      # True  (chained)
1 < 5 > 3                       # True  (1 < 5 and 5 > 3)
1 < "2"                         # TypeError
float("nan") == float("nan")    # False
max("pear", "fig", key=len)     # 'pear'
```

CODE EXPLAINED

→ `3 == 3.0` — numbers compare by value across types

→ string order — uppercase letters sort before lowercase

→ `[1, 2, 3] < [1, 3]` — stops at the first unequal item

→ chained — reads like math: `0 <= x < 10`

→ `key=` — compare by a derived value

THE PYTHONIC WAY

✗ ~~`x > 0 and x < 10`~~ → ✓ `0 < x < 10`

✗ ~~`x == 1 or x == 2`~~ → ✓ `x in (1, 2)`

✗ ~~`a if a > b else b`~~ → ✓ `max(a, b)`

OPERATORS (A = 5, B = 3)

OP	MEANING	RESULT
`a == b`	Equal	False
`a != b`	Not equal	True
`a < b`	Less than	False
`a > b`	Greater than	True
`a <= 5`	Less or equal	True
`a >= 6`	Greater or equal	False

ACROSS TYPES

EXPRESSION	RESULT
`1 == 1.0`	True
`1 == "1"`	False
`True == 1`	True
`None == 0`	False
`1 < "1"`	TypeError
`{1} < {1, 2}`	True (subset)

SEQUENCE RULES

EXPRESSION	RESULT
`[1, 2] < [1, 2, 0]`	True (prefix smaller)
`"ab" < "abc"`	True
`(2, 1) > (1, 9)`	True
`"10" < "9"`	True (text, not numbers)

⚠ COMMON MISTAKES

- `if x = 5:` — `SyntaxError`; use `==`
- `x == 1 or 2` — always truthy; use `x in (1, 2)`
- Comparing floats with `==` — use `math.isclose()`
- Sorting numeric strings — `"10" < "9"`; convert to int first

✓ TIPS & BEST PRACTICES

- Range checks read naturally: `0 <= x < 10`
- Use `key=` with `max`, `min`, `sorted` for custom order
- `functools.total_ordering` fills in missing comparison methods
- Case-insensitive compare: `a.casefold() == b.casefold()`

QUICK SUMMARY

- `==` / `!=` — value equality
- `<` `>` `<=` `>=` — ordering
- chaining — `a < b < c`
- mixed types — `TypeError` for ordering

Logical Operators

and · or · not · short-circuit · return values

COMBINING CONDITIONS

`and`, `or`, and `not` combine conditions. `and` and `or` **short-circuit**: they stop as soon as the result is known and return the **last operand evaluated**, which may not be a bool. `not` always returns `True` or `False`. Precedence runs `not` → `and` → `or`.

NOTES

- Words, not symbols: no `&&` or `||`
- `&` and `|` are bitwise, with different precedence
- Right-hand side is skipped when the left decides
- Works on truthiness (Sheet 10), not just bools
- De Morgan: `not (a and b)` == `not a or not b`

LOGIC & SHORT-CIRCUITING

```
True and False, True or False  # (False, True)
not True, not 0                # (False, True)
0 and 1 / 0                    # 0  (right side never runs)
1 or 1 / 0                     # 1
"" or "default"                # 'default'
"Ana" and "Bo"                 # 'Bo'  (last evaluated)
None or [] or 0                # 0  (all falsy → last one)
True or False and False        # True  (and binds first)
(True or False) and False      # False
x = 7
not (x > 5 or x < 0)           # False
user = {"name": "Ana"}
user and user.get("name")      # 'Ana'  (guard)
```

CODE EXPLAINED

→ `0 and ...` — `and` stops at the first falsy value

→ `1 or ...` — `or` stops at the first truthy value

→ `"" or "default"` — classic fallback pattern

→ `user and user.get(...)` — skips the call when `user` is empty

THE PYTHONIC WAY

✗ ~~`if not x == y:`~~ → ✓ `if x != y:`

✗ ~~`if a == 1 or a == 2 or a == 3:`~~ → ✓ `if a in {1, 2, 3}:`

✗ ~~for loop with an ok flag~~ → ✓ `all(check(x) for x in xs)`

TRUTH TABLE

A	B	A AND B	A OR B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

WHAT THEY RETURN

EXPRESSION	RETURNS
`a and b`	a if a is falsy, else b
`a or b`	a if a is truthy, else b
`not a`	True or False
`any(xs)`	True if any truthy
`all(xs)`	True if all truthy

PRECEDENCE

EXPRESSION	MEANS
`not a and b`	(not a) and b
`a or b and c`	a or (b and c)
`not a == b`	not (a == b)
`a or b == c`	a or (b == c)

⚠ COMMON MISTAKES

- `if x == "a" or "b":` — always true; use `x in ("a", "b")`
- Using `&` / `|` instead of `and` / `or` — bitwise, binds tighter
- `count or 10` replaces a valid `0` with `10`
- Mixing `and` / `or` without parentheses — easy to misread

✓ TIPS & BEST PRACTICES

- Guard against `None`: `if obj and obj.items:`
- Use `any()` / `all()` for conditions over a collection
- Put the cheap or most-likely-false check first in `and`
- Parenthesize mixed `and` / `or` even when not required

QUICK SUMMARY

- `and` — both truthy; returns an operand
- `or` — either truthy; returns an operand
- `not` — always a bool
- short-circuit — skips the right side

Bitwise Operators

WORKING WITH BITS

Bitwise operators work on the **binary digits** of integers. &, |, and ^ combine bits pair by pair, ~ flips every bit, and << / >> shift bits left or right. They're used for flags, masks, permissions, and binary protocols. Because Python ints are unbounded, ~x equals −x − 1.

NOTES

- Integers only; floats raise `TypeError`
- `x << n` = `x * 2**n`; `x >> n` = `x // 2**n`
- Sets reuse `&` `|` `^` `-` for set operations
- Precedence: looser than `+` and `<<`, tighter than `==`
- `enum.IntFlag` gives flags readable names

BITS & FLAGS

```
a, b = 12, 10                   # 0b1100, 0b1010
a & b                           # 8    0b1000  (both 1)
a | b                           # 14   0b1110  (either 1)
a ^ b                           # 6    0b0110  (differ)
~a                              # -13  (-(a + 1))
a << 2                          # 48   0b110000 (× 4)
a >> 2                          # 3    0b11     (÷ 4)
bin(a), f"{a:08b}"              # ('0b1100', '00001100')
READ, WRITE, EXEC = 4, 2, 1
perm = READ | WRITE             # 6   set flags
bool(perm & WRITE)              # True  test a flag
perm &= ~WRITE                  # 4   clear a flag
perm ^= EXEC                    # 5   toggle a flag
```

CODE EXPLAINED

- → `&` — keeps bits set in both
- → `|` — keeps bits set in either
- → `^` — keeps bits set in exactly one
- → `~` — flips all bits (two's complement)
- → `perm & WRITE` — nonzero means the flag is on

PYTHON VERSIONS

- **3.6** — `enum.IntFlag`
- **3.10** — `int.bit_count()`

THE PYTHONIC WAY

- ✗ ~~`bin(x).count("1")`~~ → ✓ `x.bit_count()`
- ✗ ~~`FLAG_A = 1; FLAG_B = 2`~~ → ✓ `class Perm(IntFlag)`
- ✗ ~~Bitwise flags for a few booleans~~ → ✓ **a `set` or named fields**

WORKED EXAMPLE

EXPR	BITS	DEC
`a`	1100	12
`b`	1010	10
`a & b`	1000	8
`a | b`	1110	14
`a ^ b`	0110	6
`a << 1`	11000	24

OPERATORS

OP	NAME
`&`	AND
`|`	OR
`^`	XOR
`~`	NOT (invert)
`<<`	Left shift
`>>`	Right shift

IDIOMS

CODE	DOES
`x |= F`	Set flag
`x &= ~F`	Clear flag
`x ^= F`	Toggle flag
`x & 1`	Odd test
`n & (n - 1) == 0`	Power of two (n > 0)
`n.bit_count()`	Count 1 bits

⚠ COMMON MISTAKES

- `&` instead of `and` in conditions — works on bools but breaks on ints
- `2 ^ 3` for powers — that's XOR (`1`); use `2 ** 3`
- Expecting `~5` to be `2` — it's `-6`
- Shifting a float — `TypeError`

✓ TIPS & BEST PRACTICES

- Inspect values with `f"{x:08b}"`
- Use `enum.IntFlag` instead of bare constants
- Mask the low byte with `x & 0xFF`
- Prefer sets when you just need membership flags

QUICK SUMMARY

- `&` `|` `^` — AND, OR, XOR
- `~x` — equals `-x - 1`
- `<<` `>>` — multiply/divide by powers of 2
- flags — set `|=`, clear `&= ~`, test `&`

Assignment & Compound Assignment

ASSIGN AND UPDATE

Compound operators combine an operation with assignment: `x += 1` means `x = x + 1`. For **mutable** objects such as lists, `+=` updates the object in place, so other references see the change. For immutable objects it creates a new object and rebinds the name. The **walrus** `:=` (3.8+) assigns inside an expression.

NOTES

- Python has no `++` or `--`
- `/=` turns an int into a float
- `list += iterable` behaves like `extend()`
- `:=` must be parenthesized as a statement
- Every arithmetic and bitwise operator has a compound form

COMPOUND ASSIGNMENT & WALRUS

```
x = 10
x += 5; x -= 3; x *= 2          # 24
x /= 4                          # 6.0  (now a float)
n = 17; n %= 5                  # 2
s = "ab"; s += "c"              # 'abc'  (new string)
a = [1, 2]; b = a
a += [3]                        # in place
b                               # [1, 2, 3]  (b sees it)
a = a + [4]                     # builds a new list
b                               # [1, 2, 3]  (b doesn't)
if (n := len("hello")) > 3:
    print(n)                    # 5
while (cmd := input("> ")) != "q":
    print(cmd)
```

CODE EXPLAINED

- → `x op= y` — shorthand for `x = x op y`
- → `a += [3]` — mutates the shared list
- → `a = a + [4]` — new list; `b` keeps the old one
- → `(n := ...)` — assign and use in one step
- → walrus loop — read and test without repeating the call

PYTHON VERSIONS

- **3.8** — walrus `:=` assignment expressions (PEP 572)

THE PYTHONIC WAY

- ✗ ~~`m = re.match(p, s)` then `if m:`~~ → ✓ `if m := re.match(p, s):`
- ✗ ~~`count = count + 1`~~ → ✓ `count += 1`
- ✗ ~~`d[k] = d[k] + 1` with a check first~~ → ✓ `Counter` or `d.get(k, 0) + 1`

COMPOUND OPERATORS

OP	EQUIVALENT
`x += y`	x = x + y
`x -= y`	x = x - y
`x *= y`	x = x * y
`x /= y`	x = x / y
`x //= y`	x = x // y
`x %= y`	x = x % y
`x **= y`	x = x ** y

+= BY TYPE

TYPE	EFFECT
`int / float`	Rebinds a new value
`str`	New string
`tuple`	New tuple
`list`	Extends in place
`set (|=)`	Updates in place
`dict (|=)`	Updates in place (3.9+)

WALRUS USES

PATTERN	WHY
`if (m := re.match(p, s)):`	Test and keep match
`while (b := f.read(4096)):`	Read until empty
`[y for x in xs if (y := f(x))]`	Call f once
`print(y := 5)`	Assign inside call

⚠ COMMON MISTAKES

- `x++` — `SyntaxError`; `++x` is just `+(+x)`
- `items += "ab"` adds `'a'` and `'b'` as separate items
- `x = x.append(1)` — `append` returns `None`
- Forgetting `/=` makes floats — use `//=` to stay integer

✓ TIPS & BEST PRACTICES

- Use `+=` on lists when you want shared references updated
- Use `a = a + b` when you need a fresh list
- Reach for `:=` only when it removes a duplicated call
- Counter pattern: `counts[k] = counts.get(k, 0) + 1`

QUICK SUMMARY

- `x += 1` — increment
- mutable `+=` — in place
- immutable `+=` — new object
- `:=` — assign inside an expression

Membership & Identity Operators

IN, NOT IN, IS, IS NOT

`in` and `not in` test **membership** in strings (substring), lists, tuples, sets, and dicts (keys). `is` and `is not` test **identity**: whether two names point to the same object, the same `id()`. Use == to compare values; use `is` only for singletons such as `None`, `True`, `False`, and sentinel objects.

NOTES

- `in` on a dict checks keys, not values
- Set and dict lookups are O(1) on average
- List, tuple, and str lookups scan: O(n)
- `is` can't be overridden; `==` can (`__eq__`)
- `x is 5` triggers a `SyntaxWarning` (3.8+)

UNDER THE HOOD

- `is` compares identity: `id()` is the address in CPython
- `in` calls `__contains__`, else iterates
- List `in` checks `is` before `==`: `nan in [nan]` is True

MEMBERSHIP & IDENTITY

```
3 in [1, 2, 3]                  # True
"py" in "python"                # True  (substring)
"a" in {"a": 1}                 # True  (dict keys)
1 in {"a": 1}.values()          # True
4 not in (1, 2, 3)              # True
a = [1, 2]; b = [1, 2]; c = a
a == b, a is b                  # (True, False)
a is c                          # True  (same object)
id(a) == id(c)                  # True
x = None
x is None                       # True  (preferred test)
5 in range(0, 10)               # True  (fast for ints)
```

CODE EXPLAINED

- → `in str` — substring search
- → `in dict` — key lookup; use `.values()` for values
- → `a == b` — equal contents
- → `a is b` — literally the same object
- → `in range` — computed, not scanned

PYTHON VERSIONS

- **3.8** — `is` with a literal raises a `SyntaxWarning`

THE PYTHONIC WAY

- ✗ ~~`x == None`~~ → ✓ `x is None`
- ✗ ~~`not x in items`~~ → ✓ `x not in items`
- ✗ ~~`in` on a big list in a loop~~ → ✓ **build a `set` once**

MEMBERSHIP COST

CONTAINER	LOOKUP
`list`	O(n) scan
`tuple`	O(n) scan
`str`	O(n) scan
`set`	O(1) average
`dict`	O(1) on keys
`range`	O(1) for ints

== VS IS

EXPRESSION	USE
`a == b`	Same value
`a is b`	Same object
`x is None`	✔ None check
`x == None`	Works; avoid
`x is 5`	✖ unreliable

CUSTOMIZING

METHOD	CONTROLS
`__contains__`	`in`
`__eq__`	`==`
`__ne__`	`!=`
`__iter__`	`in` fallback
`(none)`	`is` is fixed

⚠ COMMON MISTAKES

- `x is 1000` — identity of numbers is an implementation detail
- `"Ana" in users_dict.values()` vs keys — know which you need
- Checking `in` against a big list inside a loop — convert to a set
- `not x in y` — works, but `x not in y` is clearer

✓ TIPS & BEST PRACTICES

- Build a set once for repeated lookups: `seen = set(items)`
- Use `is` for `None` and sentinel objects only
- Use `d.get(key)` instead of `if key in d: d[key]`
- Test small option lists inline: `if ext in {".jpg", ".png"}:`

QUICK SUMMARY

- `in` / `not in` — membership
- `is` / `is not` — identity
- `==` — value equality
- sets & dicts — fast lookups

if/elif/else Statements

BRANCHING

`if` runs its block when the condition is truthy. `elif` checks further conditions in order, and **only the first true branch runs**. `else` catches everything left over. A condition can be any expression, judged by truthiness. Every block needs at least one statement, so use `pass` as a placeholder.

NOTES

- The keyword is `elif`, not `else if`
- Each header ends with a colon
- `elif` and `else` are optional; `else` must be last
- Conditions short-circuit with `and` / `or`
- For many fixed values, see `match` (Sheet 26) or a dict lookup

IF / ELIF / ELSE

```
score = 82
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"                 # this branch runs
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)                    # B

if not items: ...               # empty check
if 0 <= x < 10: ...             # range check
if role in {"admin", "root"}:
    pass                        # placeholder body
if user is not None and user.active:
    print("welcome")
```

CODE EXPLAINED

→ order matters — `>= 90` must come before `>= 80`

→ `...` — Ellipsis; a valid one-line body like `pass`

→ `in {...}` — compare against several values

→ `is not None and ...` — guard before using attributes

THE PYTHONIC WAY

✗ ~~`if x: return True` / `else: return False`~~ → ✓ `return bool(x)`

✗ ~~Deeply nested `if` blocks~~ → ✓ **guard clauses with early** `return`

✗ ~~Long `elif` chain mapping values~~ → ✓ **a** `dict` **lookup**

CONDITION FORMS

CONDITION	TESTS
`if x:`	Truthy
`if x is None:`	None exactly
`if x in opts:`	Membership
`if a <= x < b:`	Range
`if isinstance(x, int):`	Type
`if (m := f()):`	Assign and test

STRUCTURE RULES

RULE	DETAIL
First match wins	Later branches are skipped
`elif`	Zero or more
`else`	Zero or one, last
Empty block	Needs `pass` or `...`
Nesting	Indent another level

ALTERNATIVES TO LONG CHAINS

TOOL	USE
`dict.get(k, d)`	Map value → result
`match / case`	Patterns (Sheet 26)
`guard + return`	Exit early in functions
`x if c else y`	Two-way choice

⚠ COMMON MISTAKES

- `else if` — `SyntaxError`; use `elif`
- Ordering ranges wrong — `>= 70` first catches every grade above it
- `if x == True:` — just `if x:`
- Deep nesting — flatten with `and` or early returns

✓ TIPS & BEST PRACTICES

- Use guard clauses: `if not valid: return` at the top
- Put the most specific condition first
- Replace value-to-value chains with a dict
- Keep nesting to two levels where possible

QUICK SUMMARY

- `if` — run when truthy
- `elif` — next check; first true wins
- `else` — fallback
- `pass` — empty-block placeholder

match/case Pattern Matching

STRUCTURAL PATTERN MATCHING

`match` (3.10+) compares a **subject** against `case` patterns from top to bottom and runs the **first** one that matches. Patterns can be literals, alternatives joined with `|`, names that **capture** values, sequences, mappings, and class patterns, and any case can add an `if` **guard**. `case _:` is the catch-all. Unlike a C `switch`, there's no fall-through and no `break`.

NOTES

- Requires Python 3.10 or newer
- If nothing matches, nothing happens — no error
- A bare name always matches and **binds**; it doesn't compare
- Compare against constants with dotted names: `Color.RED`

LITERAL, GUARD, SEQUENCE, MAPPING

```
def http_status(code):
    match code:
        case 200 | 201: return "OK"           # or-pattern
        case 404: return "Not found"
        case n if 500 <= n < 600:             # capture + guard
            return f"Server error {n}"
        case _: return "Other"                # wildcard
http_status(503)                              # 'Server error 503'
def handle(cmd):
    match cmd.split():
        case ["go", direction]: return f"move {direction}"
        case ["drop", *items]: return f"drop {len(items)}"
        case []: return "empty"
handle("go north"), handle("drop a b")        # ('move north', 'drop 2')
event = {"type": "click", "pos": (3, 4)}
match event:
    case {"type": "click", "pos": (x, y)}:    # mapping pattern
        print(x, y)                           # 3 4
```

CODE EXPLAINED

- → `200 | 201` — matches either value
- → `case n if ...` — bind to `n`, then test the guard
- → `["go", direction]` — exactly two items, first is `"go"`
- → mapping pattern — checks listed keys; extra keys are ignored

PYTHON VERSIONS

- **3.10** — `match` / `case` structural pattern matching (PEP 634)

THE PYTHONIC WAY

- ✗ ~~`isinstance` + indexing to unpack data~~ → ✓ `case {"type": "move", "x": x}:`
- ✗ ~~`match` for a simple value → value map~~ → ✓ a `dict`
- ✗ ~~`case _: pass` hiding bad input~~ → ✓ `case _: raise ValueError(...)`

PATTERN KINDS

PATTERN	MATCHES
`case 404:`	Literal value
`case 1 | 2:`	Either alternative
`case n:`	Anything; binds n
`case _:`	Anything; no binding
`case [a, *rest]:`	Sequence of 1+ items
`case {"k": v}:`	Mapping with key k
`case Point(x=0):`	Class + attribute
`case n if n > 0:`	Pattern + guard

CLASS PATTERNS

PATTERN	MEANS
`case int():`	Any int
`case str(s):`	A str, bound to s
`case Point(x=0, y=y):`	x is 0; bind y
`case Point(0, y):`	Uses `__match_args__`
`@dataclass`	Sets `__match_args__`

GOTCHAS

SITUATION	RESULT
`case RED:`	Captures anything
`case Color.RED:`	Compares (dotted)
`Subject is str`	Not a sequence match
`Extra dict keys`	Still matches
`No case matches`	Silently skipped

⚠ COMMON MISTAKES

- `case MAX:` to compare a constant — it captures every value; use `Config.MAX`
- Leaving out `case _:` — unmatched values pass silently
- Expecting C-style fall-through — only one case ever runs
- Using `match` for a plain value → value lookup — a dict is simpler

✓ TIPS & BEST PRACTICES

- Order cases from most specific to most general
- Use `match` for command parsing and JSON-shaped data
- Pair it with `Enum` members for readable constant cases
- Raise in `case _:` when unknown input is a bug

QUICK SUMMARY

- `match x:` — test x against cases
- `|` / guards — alternatives and conditions
- sequence / mapping / class — destructure
- `case _:` — default; first match wins

Ternary & Conditional Expressions

CONDITIONAL EXPRESSIONS

The conditional expression `a if condition else b` evaluates to `a` or `b`. Because it's an **expression**, it fits inside assignments, `return` statements, function arguments, f-strings, lambdas, and comprehensions. Only the chosen branch is evaluated, and the `else` part is always required.

NOTES

- Order: value-if-true, condition, value-if-false
- No `?:` operator in Python
- Lowest precedence of all operators except `lambda`
- Nesting chains to the right; keep it short
- `x or default` is a shortcut for falsy fallbacks

TERNARY EXAMPLES

```
age = 20
status = "adult" if age >= 18 else "minor"   # 'adult'
n = 3
f"{n} file{'s' if n != 1 else ''}"           # '3 files'
a, b = 4, 9
bigger = a if a > b else b                   # 9 (or max(a, b))
x = -5
sign = "pos" if x > 0 else "neg" if x < 0 else "zero"  # 'neg'
["even" if i % 2 == 0 else "odd" for i in range(3)]
# ['even', 'odd', 'even']
user_name = ""
name = user_name or "Guest"                  # 'Guest'
fee = lambda member: 0 if member else 5
1 if True else 1 / 0                         # 1 (other side skipped)
```

CODE EXPLAINED

→ `a if c else b` — reads like English

→ inside an f-string — handy for plurals

→ nested — evaluated left to right; two levels max

→ in a comprehension — the ternary goes before `for`

→ skipped branch — never evaluated, so no error

THE PYTHONIC WAY

✗ ~~`if c: x = a / else: x = b`~~ → ✓ `x = a if c else b`

✗ ~~Nested ternaries~~ → ✓ `if` / `elif` **statements**

✗ ~~`x if x else default`~~ → ✓ `x or default`

FORMS

SYNTAX	MEANING
`x if c else y`	Pick one value
`a if c1 else b if c2 else d`	Three-way choice
`[a if c else b for v in xs]`	Map each item
`[v for v in xs if c]`	Filter (no else)
`x or default`	Falsy fallback

WHERE IT FITS

PLACE	EXAMPLE
Assignment	`y = a if c else b`
return	`return x if ok else None`
Argument	`print(a if c else b)`
lambda	`lambda v: v if v else 0`
f-string	`f"{'on' if on else 'off'}"`

TERNARY VS IF STATEMENT

TERNARY	IF STATEMENT
Expression	Statement
Produces one value	Runs any statements
`else` required	`else` optional
One line	Multiple lines

⚠ COMMON MISTAKES

- Leaving out `else` — `SyntaxError`
- Putting a ternary after `for` — that slot is for filter-`if` only
- Nesting three or more levels — switch to `if/elif`
- Side effects in branches — use a statement for actions

✓ TIPS & BEST PRACTICES

- Use `max()` / `min()` instead of comparing two values
- Use `x or default` only when every falsy value is invalid
- Wrap long ternaries in parentheses across lines
- If you need to explain it, use an `if` statement

QUICK SUMMARY

- `a if c else b` — expression form of if
- `else` — always required
- only one branch — runs
- comprehension — ternary before `for`

for Loops

ITERATING OVER ITERABLES

`for` walks through any **iterable** — lists, strings, dicts, files, ranges, generators — binding each item to the loop variable in turn. It is not a counter loop. Use `range()` for counts, `enumerate()` when you also need the index, and `zip()` to walk several sequences side by side. An optional `else` runs when the loop ends without `break`.

NOTES

- `range(stop)` starts at 0 and excludes `stop`
- The loop variable stays defined after the loop
- Iterating a dict yields keys
- `zip()` stops at the shortest input
- `for-else` is covered on Sheet 30

FOR PATTERNS

```
for fruit in ["apple", "fig"]:
    print(fruit)                    # apple / fig
for i in range(3): ...              # 0, 1, 2
for i in range(2, 10, 3): ...       # 2, 5, 8
for i in range(5, 0, -1): ...       # 5, 4, 3, 2, 1
for i, name in enumerate(["a", "b"], start=1):
    print(i, name)                  # 1 a / 2 b
for name, age in zip(["Ana", "Bo"], [30, 25]):
    print(name, age)                # Ana 30 / Bo 25
for key, val in {"x": 1}.items(): ...
for ch in "hi": ...                 # 'h', 'i'
for row in [[1, 2], [3, 4]]:
    for cell in row:
        print(cell, end=" ")        # 1 2 3 4
for x in reversed([1, 2, 3]): ...   # 3, 2, 1
```

CODE EXPLAINED

- → `range(a, b, step)` — negative step counts down
- → `enumerate(xs, start=1)` — index and item together
- → `zip(a, b)` — pairs items by position
- → `.items()` — unpack key and value
- → nested loops — inner runs fully for each outer item

PYTHON VERSIONS

- **3.0** — `range()` is lazy (Python 2's `xrange`)
- **3.10** — `zip(..., strict=True)`

THE PYTHONIC WAY

✗ ~~`for i in range(len(xs)):`~~ → ✓ `for i, x in enumerate(xs):`

✗ ~~`for i in range(len(a)): a[i], b[i]`~~ → ✓ `for x, y in zip(a, b):`

✗ ~~`for x in xs[::-1]:`~~ → ✓ `for x in reversed(xs):`

ITERATION HELPERS

CALL	YIELDS
`range(n)`	0 … n-1
`range(a, b, s)`	a, a+s, … < b
`enumerate(xs)`	(index, item)
`zip(a, b)`	(a_i, b_i)
`zip(a, b, strict=True)`	Error on unequal length (3.10+)
`reversed(seq)`	Items backward
`sorted(xs)`	Items in order

ITERATING DICTS

LOOP	GETS
`for k in d`	Keys
`for v in d.values()`	Values
`for k, v in d.items()`	Pairs
`for k in sorted(d)`	Keys, sorted

OTHER ITERABLES

SOURCE	ITEM
`str`	Character
`open(f)`	Line
`set`	Element (no order)
`generator`	Next yielded value

⚠ COMMON MISTAKES

- Removing items from a list while looping over it — items get skipped
- `for i in range(len(xs)):` just to read `xs[i]` — use `enumerate()`
- Unequal lists in `zip()` — extras dropped silently
- `range(1, 10)` expecting 10 — the stop is excluded

✓ TIPS & BEST PRACTICES

- Loop over a copy when removing: `for x in xs[:]:`
- Use `zip(..., strict=True)` when lengths must match
- Prefer `for item in items` over index math
- See `itertools` (Sheet 55) for chaining and grouping

QUICK SUMMARY

- `for x in iterable` — visit each item
- `range()` — counting
- `enumerate()` — index + item
- `zip()` — parallel iteration

while Loops

CONDITION-CONTROLLED LOOPS

`while` repeats its block as long as the condition is truthy, checking it before each pass. Use it when you don't know the number of iterations in advance: polling, reading until a sentinel, or retrying. Something in the body must eventually make the condition false or `break`, or the loop never ends. `else` runs when the condition turns false, not after `break`.

NOTES

- No `do-while`; use `while True` + `break`
- Condition uses truthiness: `while queue:`
- Ctrl+C stops a runaway loop (`KeyboardInterrupt`)
- Prefer `for` when looping over a collection
- Don't loop on float equality

WHILE PATTERNS

```
n = 3
while n > 0:
    print(n)                    # 3 2 1
    n -= 1

while True:                     # loop-and-a-half
    cmd = input("> ")
    if cmd == "quit":
        break
    print(f"ran {cmd}")

attempts = 0
while attempts < 3:
    attempts += 1
    if connect():
        break
else:
    print("gave up")            # only if never broke

while (line := f.readline()):  # until ''
    handle(line)
```

CODE EXPLAINED

→ counter — update the variable every pass

→ `while True` — emulates do-while; exit with `break`

→ `while ... else` — runs when retries run out

→ walrus — read and test in one step

THE PYTHONIC WAY

✗ ~~`while i < len(xs):` with `i += 1`~~ → ✓ `for x in xs:`

✗ ~~`cmd = input()` before and inside the loop~~ → ✓ `while (cmd := input()) != "q":`

✗ ~~`while len(queue) > 0:`~~ → ✓ `while queue:`

PATTERNS

CODE	USE
`while n > 0:`	Countdown
`while True: … break`	Run at least once
`while queue:`	Process until empty
`while x != SENTINEL:`	Stop on a marker
`while tries < MAX:`	Bounded retries

FOR VS WHILE

SITUATION	USE
Each item in a collection	for
Fixed count	for … range()
Until a condition changes	while
Event or input loop	while True

AVOIDING INFINITE LOOPS

GUARD	HOW
Update state	Change the tested variable
Cap iterations	Counter with a max
Back off	`time.sleep(delay)` when polling
Escape hatch	`break` on error

⚠ COMMON MISTAKES

- Forgetting to update the loop variable — infinite loop
- Index-based `while i < len(xs)` — use `for` instead
- `while x != 1.0:` with floats — may never be exactly equal
- Thinking `else` runs after `break` — it runs only when no break happened

✓ TIPS & BEST PRACTICES

- Make `while True` exits obvious and near the top
- Always cap retries and add a delay between attempts
- Use `while stack:` / `while queue:` for work lists
- Use `time.monotonic()` for timeout loops

QUICK SUMMARY

- `while cond:` — repeat while truthy
- `while True` + `break` — do-while style
- `else` — runs if loop wasn't broken
- update state — or it never ends

Loop Controls

BREAK, CONTINUE, PASS

`break` exits the innermost loop immediately. `continue` skips the rest of the current iteration and moves to the next. `pass` does nothing; it's a placeholder where a statement is required. A loop's `else` clause runs only when the loop finishes **without** `break`, which makes searches clean. Python has no labeled break, so leave nested loops with `return` or a flag.

NOTES

- `break` and `continue` affect only the innermost loop
- `else` works on both `for` and `while`
- `...` (Ellipsis) also works as an empty body
- `return` inside a loop exits the loop and the function

LOOP CONTROL

```
for n in [3, 8, 11, 4]:
    if n > 10:
        print("found", n); break    # found 11
for n in range(6):
    if n % 2:
        continue                    # skip odd
    print(n, end=" ")               # 0 2 4
for n in [4, 6, 8]:
    if n % 2:
        print("odd found"); break
else:
    print("all even")               # all even
def find(grid, target):             # leave nested loops
    for r, row in enumerate(grid):
        for c, val in enumerate(row):
            if val == target:
                return r, c
    return None
```

CODE EXPLAINED

→ `break` — stop searching once found

→ `continue` — skip the current item

→ `for … else` — runs only if no `break`

→ `return` — the cleanest exit from nested loops

THE PYTHONIC WAY

✗ ~~`found = False` + `break` + `if not found:`~~ → ✓ `for … else:`

✗ ~~`continue` to filter while building a list~~ → ✓ **a comprehension with `if`**

✗ ~~Nested `break` flags~~ → ✓ **move the loops into a function and `return`**

STATEMENTS

KEYWORD	EFFECT
`break`	Exit the loop now
`continue`	Jump to the next iteration
`pass`	Do nothing
`return`	Exit loop and function
`else`	Runs if no break

EXITING NESTED LOOPS

TECHNIQUE	HOW
`Function + return`	Wrap loops in a function
`Flag variable`	`done = True`; check outside
`itertools.product`	Flatten into one loop
`Exception`	Raise and catch outside

PLACEHOLDERS

BODY	NOTES
`pass`	Standard no-op
`...`	Ellipsis; common in stubs
`"""Docstring"""`	Counts as a body
`raise NotImplementedError`	For unfinished functions

⚠ COMMON MISTAKES

- Expecting `break` to exit all loops — only the innermost one
- `continue` before `i += 1` in a `while` — infinite loop
- Misaligned `else` — it belongs to the loop, not the inner `if`
- Leaving `pass` in finished code where logic was intended

✓ TIPS & BEST PRACTICES

- Move nested search loops into a function and `return`
- Use `for … else` instead of a `found` flag
- Filter with a comprehension instead of `continue` when building lists
- Loop over `itertools.product(a, b)` to avoid nesting

QUICK SUMMARY

- `break` — exit loop
- `continue` — next iteration
- `pass` — placeholder
- loop `else` — no-break case

List Comprehensions

[x for x in it] · if filter · if-else · nested comprehensions

BUILDING LISTS IN ONE EXPRESSION

A list comprehension `[expr for item in iterable if condition]` builds a new list in one expression. It replaces a loop with `append()`, is usually faster, and reads left to right. A filtering `if` goes at the **end**; a value choice `a if c else b` goes at the **front**. Multiple `for` clauses nest in the same order as written loops.

NOTES

- Always produces a new list
- The loop variable doesn't leak out (Python 3)
- Use parentheses instead for a lazy generator (Sheet 33)
- Dict and set versions: Sheet 32
- Keep them short enough to read in one glance

UNDER THE HOOD

- 3.12+ runs comprehensions inline; no hidden function
- Items are added with the `LIST_APPEND` instruction
- Faster than `.append()` — no method lookup per item

COMPREHENSION EXAMPLES

```
nums = [1, 2, 3, 4, 5]
[n * n for n in nums]                    # [1, 4, 9, 16, 25]
[n for n in nums if n % 2 == 0]          # [2, 4]
["even" if n % 2 == 0 else "odd" for n in nums[:3]]
# ['odd', 'even', 'odd']
[c.upper() for c in "abc"]               # ['A', 'B', 'C']
[w for w in "a bb ccc".split() if len(w) > 1]  # ['bb', 'ccc']
[(x, y) for x in [1, 2] for y in "ab"]
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
matrix = [[1, 2], [3, 4]]
[v for row in matrix for v in row]       # [1, 2, 3, 4]
[[r[i] for r in matrix] for i in range(2)]  # [[1, 3], [2, 4]]
[int(s) for s in "1,2,3".split(",")]     # [1, 2, 3]
```

CODE EXPLAINED

- → map — transform each item
- → filter — `if` at the end keeps matching items
- → choose — `if/else` at the front picks a value
- → two `for`s — outer loop first, like nested loops
- → nested comprehension — builds a list of lists

PYTHON VERSIONS

- **3.0** — the loop variable no longer leaks out
- **3.12** — comprehensions inlined — faster (PEP 709)

THE PYTHONIC WAY

✗ ~~`out = []` → `for` → `append`~~ → ✓ `[f(x) for x in xs]`

✗ ~~`list(map(lambda x: x * 2, xs))`~~ → ✓ `[x * 2 for x in xs]`

✗ ~~`[print(x) for x in xs]`~~ → ✓ **a plain `for` loop**

FORMS

SYNTAX	DOES
`[f(x) for x in xs]`	Map
`[x for x in xs if c]`	Filter
`[a if c else b for x in xs]`	Choose
`[x for r in m for x in r]`	Flatten
`[[0] * 3 for _ in range(2)]`	Grid

LOOP EQUIVALENT

COMPREHENSION PART	LOOP LINE
`[`	result = []
`for x in xs`	for x in xs:
`if c`	if c:
`f(x) ]`	result.append(f(x))

USE A LOOP INSTEAD WHEN

SITUATION	WHY
`Side effects (print, save)`	Builds a junk list
`3+ `for` clauses`	Hard to read
`Need `break``	Not supported
`Huge data`	Use a generator

⚠ COMMON MISTAKES

- `[print(x) for x in xs]` — use a plain `for` loop for side effects
- `[x for x in xs if c else y]` — `SyntaxError`; ternary goes before `for`
- Reversing nested `for` order — outer loop comes first
- `[[0] * 3] * 2` for a grid — rows are shared; use a comprehension

✓ TIPS & BEST PRACTICES

- Feed `sum()` / `any()` a generator instead of a list
- Name the loop variable for the item: `user`, not `x`
- Split long comprehensions over lines inside the brackets
- Use `:=` to avoid calling a function twice

QUICK SUMMARY

- `[f(x) for x in xs]` — map
- `if` at end — filter
- `a if c else b` at front — choose
- multiple `for` — nested order

Dict & Set Comprehensions

DICT AND SET COMPREHENSIONS

`{key: value for item in iterable}` builds a **dict**; `{expr for item in iterable}` builds a **set**, removing duplicates automatically. Both follow the same filtering and nesting rules as list comprehensions. Remember that empty braces `{}` create an empty dict, not a set.

NOTES

- Duplicate keys: the last value wins
- Sets are unordered; printed order may differ
- Dicts keep insertion order (3.7+)
- Inverting a dict needs unique values
- `set()` is the only way to write an empty set

DICT & SET COMPREHENSIONS

```
words = ["apple", "fig", "kiwi"]
{w: len(w) for w in words}          # {'apple': 5, 'fig': 3, 'kiwi': 4}
{w: len(w) for w in words if len(w) > 3}  # {'apple': 5, 'kiwi': 4}
prices = {"tea": 3, "cake": 5}
{k: v * 2 for k, v in prices.items()}     # {'tea': 6, 'cake': 10}
{v: k for k, v in prices.items()}  # {3: 'tea', 5: 'cake'}  inverted
{k.upper(): v for k, v in prices.items()} # {'TEA': 3, 'CAKE': 5}
{x: x ** 2 for x in range(4)}       # {0: 0, 1: 1, 2: 4, 3: 9}
dict(zip(["a", "b"], [1, 2]))       # {'a': 1, 'b': 2}
{n % 3 for n in range(10)}          # {0, 1, 2}
{w[0] for w in words}               # {'a', 'f', 'k'}
{c.lower() for c in "AaBb"}         # {'a', 'b'}
```

CODE EXPLAINED

- → `{k: v for ...}` — key and value expressions
- → `.items()` — loop over pairs to transform both
- → `{v: k ...}` — swap keys and values
- → `{expr for ...}` — set; duplicates collapse
- → `dict(zip(keys, values))` — build without a comprehension

PYTHON VERSIONS

- **2.7** — dict and set comprehensions
- **3.7** — dict insertion order guaranteed

THE PYTHONIC WAY

- ✗ ~~`d = {}` + loop assigning keys~~ → ✓ `{k: v for k, v in pairs}`
- ✗ ~~`dict(zip(...))` with a lambda transform~~ → ✓ `{k: f(v) for k, v in d.items()}`
- ✗ ~~`set([x for x in xs])`~~ → ✓ `{x for x in xs}`

FORMS

SYNTAX	BUILDS
`{k: f(k) for k in ks}`	Dict from keys
`{k: v for k, v in d.items() if c}`	Filtered dict
`{v: k for k, v in d.items()}`	Inverted dict
`{f(x) for x in xs}`	Set
`{}`	Empty dict (not set)

DICT HELPERS

CALL	RESULT
`dict.fromkeys("ab", 0)`	{'a': 0, 'b': 0}
`dict(zip(ks, vs))`	Pairs → dict
`d | {'c': 3}`	Merged copy (3.9+)
`list(dict.fromkeys(xs))`	Dedupe, keep order

SET USES

CODE	GETS
`{x.lower() for x in xs}`	Unique
`len({...})`	Count of distinct
`{...} & other`	Common items
`sorted({...})`	Unique, ordered list

⚠ COMMON MISTAKES

- Inverting with repeated values — earlier keys are overwritten
- `s = {}` expecting a set — it's a dict; use `set()`
- Relying on set order — it's arbitrary
- Counting with a comprehension — use `collections.Counter`

✓ TIPS & BEST PRACTICES

- `list(dict.fromkeys(xs))` removes duplicates and keeps order
- Use `dict(zip())` when you already have two parallel lists
- Filter dicts by value: `{k: v for k, v in d.items() if v}`
- Build a set once for fast repeated `in` checks

QUICK SUMMARY

- `{k: v for ...}` — dict comprehension
- `{x for ...}` — set comprehension
- `{}` — empty dict
- duplicates — last key wins / set dedupes

Generator Expressions

(x for x in it) · lazy evaluation · sum() · any() · all() · next()

LAZY COMPREHENSIONS

A generator expression `(expr for item in iterable)` looks like a list comprehension with parentheses, but it produces items **lazily**, one at a time, without building a list in memory. It's ideal for feeding `sum()`, `any()`, `all()`, `max()`, and `join()`, or for streaming large files. A generator can be consumed **only once**.

NOTES

- Parentheses can be dropped when it's the only argument
- `len()` and indexing don't work on generators
- `next(gen, default)` avoids `StopIteration`
- Generator functions with `yield` : Sheet 45
- Items are computed when requested, not upfront

GENERATOR EXPRESSIONS

```
nums = range(1_000_000)
sum(n * n for n in nums)           # no list is built
gen = (n * 2 for n in [1, 2, 3])
gen                                # <generator object <genexpr> ...>
next(gen), next(gen)               # (2, 4)
list(gen)                          # [6]  (what's left)
list(gen)                          # []   (exhausted)
any(w.startswith("x") for w in ["ax", "xy"])  # True
max(len(w) for w in ["a", "abc"])  # 3
",".join(str(n) for n in range(3)) # '0,1,2'
import sys
sys.getsizeof([n for n in range(10**6)])  # ~8 MB
sys.getsizeof((n for n in range(10**6)))  # ~200 bytes
lines = (l.strip() for l in open("app.log"))
errors = (l for l in lines if "ERROR" in l)  # pipeline
```

CODE EXPLAINED

- → `sum(... for ...)` — single argument, so no extra parens
- → `next(gen)` — pulls one item
- → exhausted — a second pass yields nothing
- → `any()` — stops at the first `True`
- → pipeline — each stage pulls from the previous one

THE PYTHONIC WAY

- ✗ ~~`sum([x * x for x in xs])`~~ → ✓ `sum(x * x for x in xs)`
- ✗ ~~`len([x for x in xs if p(x)])`~~ → ✓ `sum(1 for x in xs if p(x))`
- ✗ ~~`[x for x in xs if p(x)][0]`~~ → ✓ `next((x for x in xs if p(x)), None)`

LIST COMP VS GENERATOR

LIST [ ]	GENERATOR ( )
Eager	Lazy
Reusable	One pass
`len()`, indexing	Neither
Memory O(n)	Memory O(1)
Good for reuse	Good for one-shot totals

COMMON CONSUMERS

CALL	DOES
`sum(g)`	Total
`any(g)`	Any truthy? (short-circuits)
`max(g) / min(g)`	Extremes
`"".join(g)`	Concatenate strings
`list(g) / set(g)`	Materialize
`next(g, None)`	First item or None

SYNTAX RULES

CODE	RESULT
`sum(x for x in xs)`	✔ sole argument
`sorted(x for x in xs, reverse=True)`	✖ SyntaxError
`sorted((x for x in xs), reverse=True)`	✔ parenthesized
`(x for x in xs)[0]`	✖ TypeError

⚠ COMMON MISTAKES

- Looping over the same generator twice — the second loop is empty
- `len(gen)` — `TypeError`; count with `sum(1 for _ in gen)`
- Printing a generator to see values — wrap in `list()` first
- Closing a file before the generator reading it is consumed

✓ TIPS & BEST PRACTICES

- Use generators with `sum`, `any`, `all`, `max`, `join`
- Chain generator stages to process big files line by line
- Take the first n items with `itertools.islice(gen, n)`
- Use a list when you need the results more than once

QUICK SUMMARY

- `(x for x in xs)` — lazy sequence
- one pass — then exhausted
- `next(gen, default)` — safe single item
- low memory — ideal for streaming

try/except/finally Blocks

HANDLING EXCEPTIONS

`try` runs code that might fail. `except` catches matching exceptions, `else` runs only when nothing was raised, and `finally` **always** runs, even after `return` or an unhandled error, which makes it the place for cleanup. Python favors **EAFP** (easier to ask forgiveness than permission): try the operation and handle failure instead of pre-checking everything.

NOTES

- `except` clauses are checked top to bottom
- A parent class catches all its subclasses
- `e` from `except ... as e` is deleted after the block
- Unhandled exceptions print a traceback and exit
- For files and locks, `with` replaces most `finally` use

UNDER THE HOOD

- 3.11+: `try` costs nothing until something raises
- An exception table maps bytecode ranges to handlers
- Raising is the expensive part: it builds a traceback

TRY / EXCEPT / ELSE / FINALLY

```
text = "abc"
try:
    value = int(text)
except ValueError as e:
    print(f"bad data: {e}")    # bad data: invalid literal for int() ...
    value = 0
except TypeError:
    value = 0                  # text was None
else:
    print("parsed OK")         # only if no exception
finally:
    print("done")              # always runs

try:
    1 / 0
except (ZeroDivisionError, OverflowError) as e:
    print(type(e).__name__)    # ZeroDivisionError
```

CODE EXPLAINED

- → `except E as e` — binds the exception object
- → multiple `except` — first match handles it
- → `else` — success path kept out of the `try`
- → `finally` — cleanup that must happen
- → tuple — one handler for several types

PYTHON VERSIONS

- **3.11** — `except*` and `ExceptionGroup`
- **3.11** — zero-cost `try` on the happy path
- **3.14** — `except A, B:` without parentheses (PEP 758)

THE PYTHONIC WAY

✗ ~~`if os.path.exists(p): open(p)`~~ → ✓ `try: open(p)` + `except FileNotFoundError`

✗ ~~`except:`~~ → ✓ `except ValueError as e:`

✗ ~~`f = open(...)` … `finally: f.close()`~~ → ✓ `with open(...) as f:`

CLAUSES

CLAUSE	RUNS WHEN
`try:`	Always (first)
`except E:`	E or a subclass raised
`except (A, B):`	Either raised
`except E as e:`	Same, with the object
`else:`	No exception
`finally:`	Always (last)

COMMON EXCEPTIONS

EXCEPTION	RAISED BY
`ValueError`	Right type, bad value: `int("x")`
`TypeError`	Wrong type: `"a" + 1`
`KeyError`	Missing dict key
`IndexError`	Index out of range
`AttributeError`	Missing attribute
`FileNotFoundError`	`open()` on missing file
`ZeroDivisionError`	Divide by zero
`ModuleNotFoundError`	Failed import

HIERARCHY (EXCERPT)

PARENT	CHILDREN
`BaseException`	SystemExit, KeyboardInterrupt, Exception
`Exception`	Almost everything you catch
`ArithmeticError`	ZeroDivisionError, OverflowError
`LookupError`	KeyError, IndexError
`OSError`	FileNotFoundError, PermissionError

⚠ COMMON MISTAKES

- Bare `except:` — also catches Ctrl+C and `sys.exit()`
- `except Exception: pass` — hides real bugs silently
- Wrapping a whole function in `try` — you can't tell what failed
- `return` inside `finally` — swallows the active exception

✓ TIPS & BEST PRACTICES

- Catch the most specific exception that you can handle
- Keep the `try` block to the line(s) that can fail
- Log with `logger.exception("msg")` to keep the traceback
- Use `with` for files and locks instead of manual `finally`

QUICK SUMMARY

- `try` / `except` — catch errors
- `else` — success-only code
- `finally` — guaranteed cleanup
- specific exceptions — catch narrowly

Raising & Re-raising Exceptions

raise · raise from · re-raise · ValueError · TypeError · ExceptionGroup

SIGNALING ERRORS

`raise` signals an error by throwing an exception instance (or a class, which gets instantiated). Inside an `except` block, a bare `raise` **re-raises** the current exception with its original traceback. `raise New(...) from err` **chains** exceptions so the traceback shows the cause; `from None` hides the original. Python 3.11 added `ExceptionGroup` and `except*` for several errors at once.

NOTES

- Only `BaseException` subclasses can be raised
- Messages should state what was wrong and the bad value
- `e.add_note("...")` adds context (3.11+)
- `assert` is for debugging; `python -O` removes it
- Exceptions bubble up until something catches them

RAISE, RE-RAISE, CHAIN

```python
def set_age(age):
    if not isinstance(age, int):
        raise TypeError(f"age must be int, got {type(age).__name__}")
    if age < 0:
        raise ValueError(f"age must be >= 0, got {age}")
    return age

try:
    set_age(-1)
except ValueError as e:
    print(e)                        # age must be >= 0, got -1

try:
    config = json.loads(text)
except json.JSONDecodeError as e:
    raise ConfigError("bad config") from e   # chained
except OSError:
    raise                           # re-raise as-is
```

CODE EXPLAINED

→ validate early — raise before doing work

→ `TypeError` vs `ValueError` — wrong type vs bad value

→ `from e` — sets `__cause__`; traceback shows both

→ bare `raise` — let the same error propagate

PYTHON VERSIONS

- **3.0** — `raise ... from` exception chaining
- **3.11** — `e.add_note()` and `ExceptionGroup`

THE PYTHONIC WAY

✗ ~~`raise Exception("bad")`~~ → ✓ `raise ValueError(f"bad port: {port!r}")`

✗ ~~`raise e`~~ → ✓ **bare** `raise`

✗ ~~`raise AppError(str(e))`~~ → ✓ `raise AppError(...) from e`

RAISE FORMS

FORM	EFFECT
`raise E("msg")`	Raise with a message
`raise E`	Class is instantiated
`raise`	Re-raise current
`raise X from e`	Explicit cause
`raise X from None`	Hide original
`e.add_note("…")`	Extra context (3.11+)

WHICH EXCEPTION?

SITUATION	RAISE
`Bad value`	ValueError
`Wrong type`	TypeError
`Missing key`	KeyError
`Unfinished method`	NotImplementedError
`Invalid state`	RuntimeError
`Bad index`	IndexError

TRACEBACK WORDING

CAUSE	MESSAGE BETWEEN TRACEBACKS
`from e`	"…was the direct cause of…"
`implicit`	"During handling of…"
`from None`	Only the new exception

⚠ COMMON MISTAKES

- `raise "error"` — `TypeError`; raise an exception object
- Raising bare `Exception` — callers can't catch it precisely
- Using `assert` for input validation — disabled with `-O`
- `raise NewError()` inside `except` without `from` — muddled traceback

✓ TIPS & BEST PRACTICES

- Include the offending value in the message
- Use `from e` whenever you translate one error into another
- Prefer bare `raise` over `raise e` when re-raising
- Raise `NotImplementedError` in abstract-style base methods

QUICK SUMMARY

- `raise E("msg")` — signal an error
- `raise` — re-raise in `except`
- `from e` — chain the cause
- pick the most specific built-in

Custom Exceptions

DEFINING YOUR OWN ERRORS

Create custom exceptions by subclassing `Exception` (never `BaseException`). A small hierarchy with one **base error** for your app or package lets callers catch all of your errors at once, or a specific one. Add attributes to carry structured details, and call `super().__init__()` with a readable message.

NOTES

- Name them with an `Error` suffix
- An empty body (`pass` or a docstring) is often enough
- Subclass a built-in when the meaning matches
- Order `except` from specific to general

A SMALL EXCEPTION HIERARCHY

```
class AppError(Exception):
    """Base class for this app's errors."""

class ValidationError(AppError, ValueError):
    def __init__(self, field, message):
        super().__init__(f"{field}: {message}")
        self.field = field

def save(user):
    if "@" not in user["email"]:
        raise ValidationError("email", "missing @")

try:
    save({"email": "ana.example.com"})
except ValidationError as e:
    print(e, "|", e.field)      # email: missing @ | email
except AppError:
    print("other app error")
```

CODE EXPLAINED

→ `AppError` — one base to catch everything from your code

→ `(AppError, ValueError)` — also caught by `except ValueError`

→ `super().__init__(msg)` — sets `str(e)` and `e.args`

→ `self.field` — structured data for the handler

THE PYTHONIC WAY

✗ ~~`class MyException(Exception)`~~ → ✓ `class ConfigError(AppError)`

✗ ~~Error codes in strings: `"E42: ..."`~~ → ✓ **attributes:** `err.code`

✗ ~~A new class per failure~~ → ✓ **a few classes + clear messages**

DESIGN RULES

EXAMPLE	RULE
`class AppError(Exception)`	One base per package
`NotFoundError(AppError)`	Specific subclasses
`(AppError, ValueError)`	Reuse built-in meaning
`PaymentDeclinedError`	Suffix with Error

EXCEPTION ATTRIBUTES

ATTRIBUTE	HOLDS
`e.args`	Args passed to the constructor
`str(e)`	The message
`e.__cause__`	Set by `raise … from`
`e.__context__`	Error being handled at raise time
`e.__traceback__`	Traceback object
`e.__notes__`	Notes from `add_note()`

CUSTOM VS BUILT-IN

SITUATION	CHOOSE
`Plain bad argument`	ValueError
`Domain rule broken`	Custom error
`Callers must react differently`	Separate subclasses
`Library boundary`	Package base error

⚠ COMMON MISTAKES

- Subclassing `BaseException` — escapes `except Exception`
- Skipping `super().__init__()` — `str(e)` and `e.args` come out empty or odd
- Catching `AppError` before `ValidationError` — the specific handler never runs
- Creating a class for every possible failure — keep the hierarchy small

✓ TIPS & BEST PRACTICES

- Expose your errors at the package top level for easy import
- Document which exceptions each public function raises
- Wrap third-party errors in your own: `raise AppError(...) from e`
- Keep messages human-readable; put data in attributes

QUICK SUMMARY

- `class XError(Exception)` — custom error
- base class — catch all app errors
- attributes — structured details
- specific → general — except order

Defining & Calling Functions

FUNCTIONS

`def` creates a **function object** and binds it to a name. The indented body runs each time the function is called. `return` sends a value back and ends the call immediately; a function with no `return` gives `None`. Return several values as a tuple and unpack them. Functions are ordinary objects, so you can assign, pass, and store them.

NOTES

- Define before calling (at runtime, top to bottom)
- `greet` is the function; `greet()` calls it
- Code after `return` never runs
- Arguments are passed by object reference
- Type hints: `def f(x: int) -> str:`

DEFINE, CALL, RETURN

```
def greet(name: str) -> str:
    """Return a greeting for name."""
    return f"Hello, {name}!"

greet("Ana")                        # 'Hello, Ana!'

def log(msg):
    print(msg)                      # no return statement

result = log("hi")                  # prints: hi
result is None                      # True

def min_max(nums):
    return min(nums), max(nums)     # returns a tuple

lo, hi = min_max([4, 1, 9])         # lo=1, hi=9
say = greet                         # functions are objects
say("Bo")                           # 'Hello, Bo!'
```

CODE EXPLAINED

→ `-> str` — return type hint (not enforced)

→ docstring — first line of the body

→ no `return` — the call evaluates to `None`

→ `return a, b` — tuple; unpack at the call

→ `say = greet` — no parentheses, so no call

THE PYTHONIC WAY

✗ ~~`print(result)` inside the function~~ → ✓ `return result`

✗ ~~`return (a, b)`~~ → ✓ `return a, b`

✗ ~~`def calc(x):` with no hints~~ → ✓ `def calc(x: float) -> float:`

ANATOMY

PART	ROLE
`def`	Keyword
`name`	snake_case verb
`(params)`	Inputs
`-> type`	Return hint
`:`	Starts the body
`"""doc"""`	Documentation
`return`	Output and exit

RETURN FORMS

CODE	CALLER GETS
`return x`	x
`return a, b`	(a, b)
`return`	None
`(no return)`	None
`return early`	Exits immediately

FUNCTION ATTRIBUTES

ATTRIBUTE	VALUE
`f.__name__`	'greet'
`f.__doc__`	Docstring
`f.__annotations__`	Type hints
`f.__defaults__`	Default values
`callable(f)`	True

⚠ COMMON MISTAKES

- Calling before the `def` line runs — `NameError`
- Writing `greet` instead of `greet()` — you get the function, not a result
- Using `print()` where you meant `return` — caller gets `None`
- Putting code after `return` — it's unreachable

✓ TIPS & BEST PRACTICES

- Give each function one job and a verb name
- Return values; let the caller decide whether to print
- Return early to avoid deep nesting
- Write a one-line docstring for every public function

QUICK SUMMARY

- `def name(params):` — define
- `name(args)` — call
- `return` — send back and exit
- no return — `None`

Parameters & Arguments

PASSING VALUES IN

Parameters are the names in the `def`; **arguments** are the values passed in a call. Arguments match by **position** or by **keyword**. `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dict. At the call site, `*` and `**` do the reverse: they unpack a sequence or dict into arguments.

NOTES

- Positional arguments must come before keyword ones
- Keyword arguments can appear in any order
- `args` / `kwargs` are conventional names only
- Order in `def`: normal, `*args`, keyword-only, `**kwargs`
- Defaults and keyword-only: Sheet 39

POSITIONAL, KEYWORD, *ARGS, **KWARGS

```
def describe(name, age, city="Unknown"):
    return f"{name}, {age}, {city}"

describe("Ana", 30)                    # 'Ana, 30, Unknown'
describe(age=30, name="Ana")           # keywords, any order
describe("Ana", city="Rome", age=30)   # mixed
def total(*args):
    return sum(args)                   # args is a tuple
total(1, 2, 3)                         # 6
def tag(**kwargs):
    return kwargs                      # kwargs is a dict
tag(id=1, cls="box")                   # {'id': 1, 'cls': 'box'}
def show(a, *args, **kwargs):
    print(a, args, kwargs)
show(1, 2, 3, x=4)                     # 1 (2, 3) {'x': 4}
nums = [30]; opts = {"city": "Oslo"}
describe("Bo", *nums, **opts)          # 'Bo, 30, Oslo'
```

CODE EXPLAINED

- → positional — matched left to right
- → keyword — matched by name
- → `*args` — extra positionals → tuple
- → `**kwargs` — extra keywords → dict
- → `*seq` / `**dict` in a call — unpack into arguments

PYTHON VERSIONS

- **3.5** — several `*` / `**` unpacks in one call (PEP 448)
- **3.8** — positional-only `/` parameters

THE PYTHONIC WAY

- ✗ ~~`f(data[0], data[1], data[2])`~~ → ✓ `f(*data)`
- ✗ ~~`f("a.txt", True, False)`~~ → ✓ `f("a.txt", overwrite=True, backup=False)`
- ✗ ~~`def f(options):`~~ ~~`options["x"]`~~ → ✓ `def f(*, x, y):`

PARAMETER KINDS

IN DEF	ACCEPTS
`a`	Positional or keyword
`a=1`	Optional, with default
`*args`	Any extra positionals
`*, b`	Keyword-only b
`**kwargs`	Any extra keywords

CALL SYNTAX

CALL	PASSES
`f(1, 2)`	Positional
`f(a=1)`	Keyword
`f(*[1, 2])`	Unpacked list
`f(**{'a': 1})`	Unpacked dict
`f(*xs, **opts)`	Both

ARGUMENT ERRORS

MESSAGE	CAUSE
`missing 1 required positional argument`	Too few args
`got multiple values for argument`	Same arg twice
`got an unexpected keyword argument`	Unknown name
`positional argument follows keyword`	SyntaxError

⚠ COMMON MISTAKES

- `f(a=1, 2)` — positional after keyword is a `SyntaxError`
- Passing an argument both by position and by name
- Using `*args` when a single list parameter is clearer
- Forgetting `**` when forwarding: `f(kwargs)` passes one dict

✓ TIPS & BEST PRACTICES

- Pass booleans by keyword: `save(path, overwrite=True)`
- Forward everything with `wrapped(*args, **kwargs)`
- Unpack config dicts into calls with `**settings`
- Type `*args: int` and `**kwargs: str` for clarity

QUICK SUMMARY

- positional — by order
- keyword — by name
- `*args` — tuple of extras
- `**kwargs` — dict of extras

Default & Keyword Arguments

default values · mutable default pitfall · keyword-only · positional-only · / · *

DEFAULTS & ARGUMENT RULES

Default values make parameters optional. Defaults are evaluated **once**, when the `def` runs, so a mutable default such as `[]` or `{}` is shared across every call; use `None` and create the object inside. A bare `*` makes the parameters after it **keyword-only**, and `/` (3.8+) makes the parameters before it **positional-only**.

NOTES

- Parameters with defaults must follow those without
- Immutable defaults (numbers, strings, `None`) are safe
- `f.__defaults__` shows stored defaults
- Positional-only lets you rename parameters freely
- Use an `object()` sentinel when `None` is a valid value

DEFAULTS, *, /

```
def connect(host, port=5432, timeout=10): ...
connect("db")                          # port=5432, timeout=10
connect("db", timeout=3)               # skip port by name

def add_item(item, bucket=[]):         # BUG: one shared list
    bucket.append(item); return bucket
add_item(1); add_item(2)               # [1, 2]  surprise!

def add_item(item, bucket=None):       # fix
    if bucket is None:
        bucket = []
    bucket.append(item); return bucket

def resize(img, *, width, height): ...
resize(img, width=800, height=600)     # names required
resize(img, 800, 600)                  # TypeError

def dist(x, y, /, *, unit="m"): ...
dist(3, 4, unit="km")                  # x, y positional-only
```

CODE EXPLAINED

- → defaults — fill in what the caller omits
- → `bucket=[]` — created once, reused forever
- → `None` sentinel — fresh list per call
- → `*` — everything after must be named
- → `/` — everything before must be positional

PYTHON VERSIONS

- **3.0** — keyword-only parameters after `*`
- **3.8** — positional-only parameters before `/`

THE PYTHONIC WAY

- ✗ ~~`def f(items=[]):`~~ → ✓ `items=None`, then `if items is None: items = []`
- ✗ ~~`def save(path, overwrite=False)`~~ → ✓ `def save(path, *, overwrite=False)`
- ✗ ~~`def f(when=time.time()):`~~ → ✓ `when=None` and compute inside

SIGNATURE MARKERS

MARKER	MEANING
`p=val`	Default value
`/`	Params before: positional-only
`*`	Params after: keyword-only
`*args`	Also forces later params keyword-only

READING DEF F(A, /, B, *, C)

PARAM	PASS AS
a	Position only
b	Position or keyword
c	Keyword only
`f(1, 2, c=3)`	✔ valid
`f(a=1, b=2, c=3)`	✖ TypeError

DEFAULT TIMING

DEFAULT	BEHAVIOR
`0, "", None`	Safe (immutable)
`[] / {} / set()`	Shared across calls
`datetime.now()`	Frozen at def time
`None + create inside`	Fresh each call

⚠ COMMON MISTAKES

- Mutable default `[]` or `{}` — state leaks between calls
- `def f(a=1, b):` — `SyntaxError`; defaults go last
- `def f(when=datetime.now())` — timestamp frozen at import
- Bare positional flags `f(x, True, False)` — unreadable

✓ TIPS & BEST PRACTICES

- Make flags keyword-only: `def save(path, *, overwrite=False)`
- Use `_MISSING = object()` when `None` is meaningful
- Inspect signatures with `inspect.signature(f)`
- Prefer `/` in public APIs whose parameter names may change

QUICK SUMMARY

- `p=val` — optional parameter
- `None` default — avoid shared mutables
- `*` — keyword-only after
- `/` — positional-only before

Variable Scope

WHERE NAMES LIVE

Python looks up names with the **LEGB** rule: **L**ocal, **E**nclosing function, **G**lobal (module), then **B**uilt-in. Reading an outer name needs no declaration. But **assigning** to a name anywhere in a function makes it local for the whole function, unless you declare it `global` or `nonlocal`.

NOTES

- `if`, `for`, `while`, `with` do **not** create scopes
- Functions, classes, and comprehensions do
- Loop variables remain after the loop ends
- `global` targets module level; `nonlocal` the enclosing function
- Shadowing a built-in (`list = ...`) hides it module-wide

UNDER THE HOOD

- Local vs global is decided at compile time, per function
- Locals live in a fast array (`LOAD_FAST`), not a dict
- So assigning later makes an earlier read `UnboundLocalError`

LEGB IN ACTION

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)                # local
    inner()
    print(x)                    # enclosing
outer(); print(x)               # global

count = 0
def bump():
    global count                # rebind the module-level name
    count += 1
bump(); print(count)            # 1

def broken():
    print(count)                # UnboundLocalError
    count = 5                   # this makes count local everywhere
```

CODE EXPLAINED

- → each function has its own `x`
- → lookup stops at the first scope that has the name
- → `global count` — assignment changes the module variable
- → `broken()` — the later assignment makes `count` local, so the read fails

PYTHON VERSIONS

- **3.0** — `nonlocal` keyword

THE PYTHONIC WAY

- ✗ ~~`global total` to share state~~ → ✓ **pass it in and `return` it**
- ✗ ~~`list = [1, 2]`~~ → ✓ **`items = [1, 2]`**
- ✗ ~~Mutable module-level globals~~ → ✓ **constants in `UPPER_CASE` only**

LEGB

SCOPE	CONTAINS
`Local`	Names assigned in this function
`Enclosing`	Names in outer functions
`Global`	Module-level names
`Built-in`	len, print, range, …

SCOPE TOOLS

CODE	DOES
`global x`	Assign module-level x
`nonlocal x`	Assign enclosing x (Sheet 41)
`globals()`	Dict of module names
`locals()`	Dict of local names
`import builtins`	Access the built-in scope

BLOCK BEHAVIOR

CONSTRUCT	NEW SCOPE?
`def / lambda`	Yes
`class`	Yes
`comprehension`	Yes
`if / for / while`	No
`with / try`	No

⚠ COMMON MISTAKES

- `UnboundLocalError` — reading a name you assign later in the function
- Overusing `global` — hidden coupling; pass and return values
- Naming variables `list`, `sum`, `input` — shadows built-ins
- Expecting `for` or `if` blocks to hide variables

✓ TIPS & BEST PRACTICES

- Pass data in as arguments and return results
- Module-level constants are fine; mutable globals are not
- Use `nonlocal` for small closures, a class for bigger state
- Lint with `ruff` to catch shadowed built-ins

QUICK SUMMARY

- LEGB — lookup order
- assign inside → local
- `global` — rebind module name
- blocks — no new scope

Closures & Nested Functions

FUNCTIONS THAT REMEMBER

A nested function can read variables from the function that encloses it. When the inner function is **returned** and outlives the outer call, it keeps those variables alive; that pairing of function plus captured variables is a **closure**. Use `nonlocal` to rebind a captured variable. Closures give you lightweight state and function factories.

NOTES

- Closures capture **variables**, not values (late binding)
- Reading a captured name needs no keyword
- Rebinding it needs `nonlocal`
- Mutating a captured list works without `nonlocal`

UNDER THE HOOD

- Captured variables live in shared cell objects
- `f.__closure__` holds the cells
- `f.__code__.co_freevars` lists their names

FACTORIES, STATE, LATE BINDING

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor           # factor is captured
    return multiply
double = make_multiplier(2)
triple = make_multiplier(3)
double(5), triple(5)                # (10, 15)

def counter():
    n = 0
    def inc():
        nonlocal n                  # rebind enclosing n
        n += 1
        return n
    return inc
c = counter(); c(); c()             # 2

fns = [lambda: i for i in range(3)]
[f() for f in fns]                  # [2, 2, 2]  late binding!
```

CODE EXPLAINED

- → `make_multiplier` — a factory that returns configured functions
- → each call to the factory creates a new closure
- → `nonlocal n` — without it, `n += 1` raises `UnboundLocalError`
- → late binding — every lambda sees the final `i`

PYTHON VERSIONS

- **3.0** — `nonlocal` lets a closure rebind captured names

THE PYTHONIC WAY

- ✗ ~~`lambda: i` in a loop~~ → ✓ `lambda i=i: i`
- ✗ ~~Closure-based counter with many methods~~ → ✓ **a small class**
- ✗ ~~Closure just to pre-fill an argument~~ → ✓ `functools.partial(f, x)`

TERMS

TERM	MEANING
free variable	Used but not defined locally
cell	Storage for a captured variable
__closure__	Tuple of cells
nonlocal	Rebind an enclosing name
factory	Function that builds functions

COMMON USES

USE	EXAMPLE
Configured function	make_multiplier(2)
Private counter	counter()
Callback with data	on_click(item) handler
Decorators	Sheet 44
Fix late binding	lambda i=i: i

CLOSURE VS CLASS

NEED	PICK
One operation, small state	Closure
Several methods	Class
Inspect/modify state	Class
Pre-filled arguments	functools.partial

⚠ COMMON MISTAKES

- Building lambdas in a loop — all share the last value; bind with `i=i`
- Rebinding without `nonlocal` — `UnboundLocalError`
- Assuming each closure copies values — it shares the variable
- Hiding lots of state in nested functions — use a class

✓ TIPS & BEST PRACTICES

- Use `functools.partial(f, x)` for simple pre-filled arguments
- Inspect captured values: `f.__closure__[0].cell_contents`
- Prefer a class once you need more than one operation
- Closures are the building block for decorators

QUICK SUMMARY

- closure — function + captured variables
- factory — returns configured functions
- `nonlocal` — rebind captured name
- late binding — captures variables, not values

Lambda Functions

ANONYMOUS FUNCTIONS

`lambda params: expression` creates a small anonymous function. The body is a **single expression** whose value is returned automatically: no statements, no `return`, no annotations. Lambdas shine as short `key=` and callback arguments. If you'd give it a name or reuse it, write a `def` instead.

NOTES

- `lambda` creates the same function type as `def`
- Its `__name__` is `'<lambda>'` (worse tracebacks)
- Defaults, `*args`, `**kwargs` all work
- Ternary expressions are allowed inside
- PEP 8 discourages `f = lambda ...`

LAMBDAS IN CONTEXT

```
square = lambda x: x * x            # works, but prefer def
square(4)                           # 16
(lambda a, b=2: a + b)(3)           # 5
names = ["bo", "Ana", "cy"]
sorted(names, key=str.lower)        # ['Ana', 'bo', 'cy']
sorted(names, key=lambda s: len(s))  # ['bo', 'cy', 'Ana']
pairs = [("a", 3), ("b", 1)]
sorted(pairs, key=lambda p: p[1])  # [('b', 1), ('a', 3)]
max(pairs, key=lambda p: p[1])     # ('a', 3)
list(map(lambda x: x * 2, [1, 2])) # [2, 4]
list(filter(lambda x: x > 1, [1, 2, 3]))  # [2, 3]
users = [{"n": "Bo", "age": 30}, {"n": "Ana", "age": 30}]
sorted(users, key=lambda u: (u["age"], u["n"]))  # Ana first
button.config(command=lambda: save(doc))  # Tkinter callback
```

CODE EXPLAINED

→ `key=str.lower` — pass an existing function when one fits

→ `key=lambda p: p[1]` — sort by the second field

→ tuple key — sorts by age, then name

→ `command=lambda: ...` — delays the call until the click

THE PYTHONIC WAY

✗ ~~`square = lambda x: x * x`~~ → ✓ `def square(x): return x * x`

✗ ~~`key=lambda p: p["age"]`~~ → ✓ `key=itemgetter("age")`

✗ ~~`key=lambda s: len(s)`~~ → ✓ `key=len`

LAMBDA VS DEF

LAMBDA	DEF
Expression	Statement
One expression	Any statements
Anonymous	Named
No annotations	Type hints
Inline, throwaway	Reusable

FORMS

LAMBDA	TAKES
`lambda: 0`	No args
`lambda x: x`	One arg
`lambda x, y: x + y`	Two args
`lambda *a: sum(a)`	Any number
`lambda x=1: x`	Default

OFTEN BETTER

INSTEAD OF	USE
`lambda p: p[1]`	itemgetter(1)
`lambda u: u.age`	attrgetter("age")
`lambda s: len(s)`	len
`map(lambda …)`	A comprehension

⚠ COMMON MISTAKES

- `f = lambda x: ...` — use `def f(x):` for named functions
- Trying statements in a lambda (`=`, `if:` blocks, `return`) — `SyntaxError`
- Lambdas created in a loop capture the last loop value
- Long, clever lambdas — turn them into a named `def`

✓ TIPS & BEST PRACTICES

- Sort by several fields with a tuple key
- `operator.itemgetter` and `attrgetter` are faster and clearer
- Pass `key=len`, `key=str.lower` directly — no lambda needed
- In GUIs, use `lambda` to pass arguments to callbacks

QUICK SUMMARY

- `lambda x: expr` — inline function
- single expression — auto-returned
- best for `key=` and callbacks
- reuse it → `def`

Higher-Order Functions

FUNCTIONS AS VALUES

A **higher-order function** takes a function as an argument, returns a function, or both. Python functions are first-class values: you can store them in variables, lists, and dicts, and pass them around. Built-ins like `map()`, `filter()`, `sorted()`, `min()`, and `max()` accept functions, and `functools.reduce()` folds a sequence into one value.

NOTES

- `map()` and `filter()` return lazy iterators
- `filter(None, xs)` keeps the truthy items
- `map()` accepts several iterables in parallel
- Comprehensions are often clearer than `map` / `filter`
- Decorators (Sheet 44) return functions

PASSING AND RETURNING FUNCTIONS

```
def apply(func, value):
    return func(value)
apply(str.upper, "hi")              # 'HI'
apply(len, [1, 2, 3])               # 3
list(map(int, ["1", "2"]))          # [1, 2]
list(map(pow, [2, 3], [3, 2]))      # [8, 9]  two iterables
list(filter(str.isdigit, ["1", "a", "2"]))  # ['1', '2']
list(filter(None, [0, 1, "", "x"]))         # [1, 'x']
from functools import reduce
reduce(lambda a, b: a * b, [1, 2, 3, 4])    # 24
ops = {"+": lambda a, b: a + b, "-": lambda a, b: a - b}
ops["-"](7, 2)                      # 5  dispatch table
def compose(f, g):
    return lambda x: f(g(x))
shout = compose(str.upper, str.strip)
shout("  hey ")                     # 'HEY'
```

CODE EXPLAINED

- → `apply(str.upper, ...)` — pass the function, don't call it
- → `map(f, xs)` — f applied to each item
- → `filter(pred, xs)` — keep items where pred is truthy
- → `reduce(f, xs)` — combine items pairwise into one
- → `compose` — returns a brand-new function

THE PYTHONIC WAY

- ✗ ~~`list(filter(lambda x: x > 0, xs))`~~ → ✓ `[x for x in xs if x > 0]`
- ✗ ~~`reduce(lambda a, b: a + b, xs)`~~ → ✓ `sum(xs)`
- ✗ ~~`if op == "add": ... elif ...`~~ → ✓ `OPS = {"add": add, ...}`

FUNCTION-TAKING BUILT-INS

CALL	RETURNS
`map(f, xs)`	Iterator of f(x)
`filter(p, xs)`	Iterator of x where p(x)
`sorted(xs, key=f)`	Sorted list
`max(xs, key=f)`	Largest by f
`reduce(f, xs, init)`	Single value

COMPREHENSION EQUIVALENTS

FUNCTIONAL	COMPREHENSION
`map(f, xs)`	(f(x) for x in xs)
`filter(p, xs)`	(x for x in xs if p(x))
`reduce(add, xs)`	sum(xs)
`map(f, a, b)`	(f(x, y) for x, y in zip(a, b))

OPERATOR MODULE

FUNCTION	SAME AS
`add(a, b)`	a + b
`mul(a, b)`	a * b
`itemgetter(1)`	lambda x: x[1]
`attrgetter('n')`	lambda x: x.n
`methodcaller('strip')`	lambda x: x.strip()

⚠ COMMON MISTAKES

- Printing `map(...)` — shows an iterator; wrap with `list()`
- Reusing a `map` or `filter` result — it's exhausted after one pass
- Passing `f()` instead of `f` — you pass the result, not the function
- `reduce()` on an empty list without `initial` — `TypeError`

✓ TIPS & BEST PRACTICES

- Use `sum`, `max`, `any` before reaching for `reduce`
- Replace long `if/elif` chains with a dict of functions
- Prefer comprehensions for simple map/filter work
- Use `operator` functions instead of tiny lambdas

QUICK SUMMARY

- first-class — functions are values
- `map` / `filter` — lazy transforms
- `reduce` — fold to one value
- return functions — compose behavior

Decorators

WRAPPING FUNCTIONS

A **decorator** is a function that takes a function and returns a replacement, usually a wrapper that adds behavior before or after the original runs. Writing `@decorator` above a `def` is shorthand for `func = decorator(func)`. Use `functools.wraps` so the wrapper keeps the original name and docstring. A decorator that takes arguments needs one extra layer: a factory that returns the decorator.

NOTES

- Decorators run once, at definition time
- The wrapper runs on every call
- Forward arguments with `*args, **kwargs`
- Stacked decorators apply bottom-up
- Classes can be decorated too (`@dataclass`)

UNDER THE HOOD

- `@d` above `def f` is just `f = d(f)`
- It runs once, when the `def` executes
- `@wraps` copies `__name__` / `__doc__`, sets `__wrapped__`

A TIMER AND A DECORATOR FACTORY

```
import functools, time
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.perf_counter() - start:.3f}s")
        return result
    return wrapper
@timer                              # same as: nap = timer(nap)
def nap(secs):
    time.sleep(secs)
nap(0.1)                            # nap: 0.100s
def repeat(times):                  # factory: returns a decorator
    def deco(func):
        return lambda *a, **kw: [func(*a, **kw) for _ in range(times)]
    return deco
@repeat(3)
def hi(): return "hi"
hi()                                # ['hi', 'hi', 'hi']
```

CODE EXPLAINED

- → `wrapper(*args, **kwargs)` — accepts any signature
- → `return result` — pass the original return value back
- → `@functools.wraps(func)` — copies `__name__`, `__doc__`
- → `@repeat(3)` — call the factory, then decorate

PYTHON VERSIONS

- **3.2** — `@wraps` sets `__wrapped__`
- **3.9** — any expression as a decorator (PEP 614)
- **3.9** — `@functools.cache`

THE PYTHONIC WAY

- ✗ ~~Wrapper without `@wraps(func)`~~ → ✓ `@functools.wraps(func)`
- ✗ ~~Hand-rolled memo dict~~ → ✓ `@functools.cache`
- ✗ ~~`f = timer(f)` after the def~~ → ✓ `@timer` **above the def**

BUILT-IN DECORATORS

DECORATOR	DOES
`@property`	Method as attribute
`@staticmethod`	No self/cls
`@classmethod`	Receives cls
`@functools.cache`	Memoize results
`@functools.lru_cache(128)`	Bounded memo
`@dataclass`	Generate __init__ etc.
`@contextmanager`	Generator → with

STACKING

CODE	MEANS
`@a @b def f`	f = a(b(f))
`Apply order`	b first, then a
`Call order`	a's wrapper runs first
`f.__wrapped__`	Original (with wraps)

TYPICAL USES

USE	EXAMPLE
`Timing`	@timer
`Caching`	@cache
`Retry`	@retry(times=3)
`Access control`	@login_required
`Registration`	@app.route("/")

⚠ COMMON MISTAKES

- Forgetting `return result` — decorated function returns `None`
- Skipping `@wraps` — every function looks like `wrapper` in logs
- `@repeat` without `(3)` — the factory receives the function instead
- `@timer()` with parentheses on a plain decorator — `TypeError`

✓ TIPS & BEST PRACTICES

- Keep wrappers thin; put real logic in helpers
- Test the undecorated function via `f.__wrapped__`
- Use `@functools.cache` before writing your own memoizer
- Name the factory parameter clearly: `@retry(times=3)`

QUICK SUMMARY

- `@deco` — `f = deco(f)`
- wrapper — adds behavior around calls
- `@wraps` — keep metadata
- factory — decorator with arguments

Generators & yield

LAZY FUNCTIONS

A function that contains `yield` is a **generator function**. Calling it returns a generator object and runs **none** of the body. Each `next()` resumes the body until the next `yield`, hands back a value, and pauses with all local state preserved. When the function returns, iteration stops with `StopIteration`. Generators are ideal for lazy sequences, infinite streams, and pipelines.

NOTES

- `for` loops call `next()` and catch `StopIteration` for you
- A generator runs once; create a new one to restart
- `yield from iterable` delegates to another iterable
- `return x` inside sets `StopIteration.value`
- Generator expressions are the one-line version (Sheet 33)

UNDER THE HOOD

- A generator keeps its frame (locals + position) between `next()` calls
- `inspect.getgeneratorstate(g)` shows where it is
- Memory stays flat however many items it yields

YIELD IN PRACTICE

```
def countdown(n):
    while n > 0:
        yield n                    # pause here
        n -= 1
gen = countdown(3)                 # body hasn't run yet
next(gen), next(gen)               # (3, 2)
list(gen)                          # [1]
for x in countdown(2): print(x)    # 2 / 1
def fib():                         # infinite stream
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
from itertools import islice
list(islice(fib(), 7))             # [0, 1, 1, 2, 3, 5, 8]
def flatten(xss):
    for xs in xss:
        yield from xs              # delegate
list(flatten([[1, 2], [3]]))       # [1, 2, 3]
```

CODE EXPLAINED

- → `yield n` — produce a value and pause
- → `next(gen)` — resume until the next `yield`
- → `while True` — fine; values are produced on demand
- → `islice(gen, 7)` — take the first 7 safely
- → `yield from` — yields every item of `xs`

PYTHON VERSIONS

- **3.3** — `yield from` and `return value` in generators
- **3.7** — `StopIteration` inside becomes `RuntimeError` (PEP 479)

THE PYTHONIC WAY

- ✗ ~~Build a list, then `return` it~~ → ✓ `yield` **each item**
- ✗ ~~`for x in sub(): yield x`~~ → ✓ `yield from sub()`
- ✗ ~~`list(gen)[:10]`~~ → ✓ `islice(gen, 10)`

GENERATOR API

CALL	DOES
`next(g)`	Next value or StopIteration
`next(g, default)`	Next value or default
`g.send(v)`	Resume; yield expr gets v
`g.throw(E)`	Raise E at the yield
`g.close()`	Stop the generator

GENERATOR VS LIST FUNCTION

YIELD	RETURN LIST
Lazy	Eager
O(1) memory	O(n) memory
Can be infinite	Must finish
One pass	Reusable

GOOD FITS

TASK	WHY
Reading huge files	One chunk at a time
Infinite sequences	Values on demand
Data pipelines	Stage feeds stage
Tree traversal	yield from children
@contextmanager	Setup / yield / cleanup

⚠ COMMON MISTAKES

- Expecting the body to run on call — nothing happens until `next()`
- Iterating an exhausted generator — silently empty
- `len()` or indexing a generator — `TypeError`
- `list()` on an infinite generator — hangs; use `islice`

✓ TIPS & BEST PRACTICES

- Turn "build a list and return it" functions into generators
- Use `yield from` instead of a `for` + `yield` loop
- Limit infinite generators with `islice` or `takewhile`
- `send()` is rarely needed; prefer plain iteration

QUICK SUMMARY

- `yield` — makes a generator function
- `next()` — resume to next yield
- `yield from` — delegate
- lazy — memory-friendly streams

Lists — Basics

list literals · indexing · slicing · mutability · len() · iteration

ORDERED, MUTABLE SEQUENCES

A **list** is an ordered, **mutable** sequence that can hold items of any type, duplicates included. Create one with brackets or `list()`. Indexing and slicing work exactly as they do on strings, but unlike strings you can change items, replace slices, and grow or shrink the list in place.

NOTES

- Indexes start at 0; `-1` is the last item
- Slicing returns a **new** list
- Assignment `b = a` shares the same list
- Lists can nest: `[[1, 2], [3]]`
- Methods are on Sheet 47; copying on Sheet 48

UNDER THE HOOD

- A list is an array of pointers to objects
- `append` over-allocates, so it's amortized O(1)
- `insert(0, x)` shifts every pointer: O(n)

LIST BASICS

```
nums = [10, 20, 30, 40]
mixed = [1, "two", 3.0, [4]]        # any types
empty = []; also_empty = list()
nums[0], nums[-1]                   # (10, 40)
nums[1:3]                           # [20, 30]  (new list)
nums[0] = 99                        # [99, 20, 30, 40]
nums[1:3] = [7, 8, 9]               # [99, 7, 8, 9, 40]
del nums[0]                         # [7, 8, 9, 40]
len(nums)                           # 4
30 in nums                          # False
[0] * 3                             # [0, 0, 0]
[1, 2] + [3]                        # [1, 2, 3]
list("abc"), list(range(3))         # (['a', 'b', 'c'], [0, 1, 2])
```

CODE EXPLAINED

- → `nums[0] = 99` — replace one item in place
- → `nums[1:3] = [...]` — slice assignment can change length
- → `del nums[0]` — remove by index
- → `[0] * 3` — repeat; safe for immutable items
- → `list(iterable)` — build from any iterable

THE PYTHONIC WAY

- ✗ ~~`if len(xs) == 0:`~~ → ✓ `if not xs:`
- ✗ ~~`xs[len(xs) - 1]`~~ → ✓ `xs[-1]`
- ✗ ~~`b = a to copy`~~ → ✓ `b = a.copy()`

CREATING

CODE	RESULT
`[]`	Empty list
`[1, 2, 3]`	Literal
`list("ab")`	['a', 'b']
`list(range(3))`	[0, 1, 2]
`[0] * 4`	[0, 0, 0, 0]
`[x * 2 for x in xs]`	Comprehension

ACCESS & CHANGE

CODE	DOES
`lst[i]`	Read item
`lst[i] = v`	Replace item
`lst[a:b] = [...]`	Replace slice
`del lst[i]`	Delete item
`del lst[a:b]`	Delete slice
`lst[:] = []`	Empty in place

SEQUENCE OPERATIONS

CODE	RETURNS
`len(lst)`	Item count
`x in lst`	bool
`a + b`	New joined list
`lst * 2`	New repeated list
`sum / min / max`	Aggregates
`a == b`	Item-by-item equality

⚠ COMMON MISTAKES

- `lst[len(lst)]` — `IndexError`; the last item is `lst[-1]`
- `[[0] * 3] * 3` — three references to one row
- `b = a` then changing `b` — `a` changes too
- Removing items while looping over the same list

✓ TIPS & BEST PRACTICES

- Copy with `lst[:]` or `lst.copy()` before modifying
- Use a tuple for a fixed record that shouldn't change
- `lst[-n:]` gets the last n items safely
- Check emptiness with `if not lst:`

QUICK SUMMARY

- `[...]` — ordered, mutable
- `lst[i] = v` — change in place
- slices — new lists
- `b = a` — alias, not copy

Lists — Methods

append() · insert() · extend() · remove() · pop() · sort() · reverse() · copy()

BUILT-IN LIST METHODS

List methods change the list **in place** and most return `None`. Add items with `append()`, `extend()`, and `insert()`; remove them with `remove()` (by value), `pop()` (by index), and `clear()`. `sort()` and `reverse()` reorder, while `index()` and `count()` answer questions without changing anything.

NOTES

- `append(x)` adds x as **one** item
- `extend(it)` adds each item of an iterable
- `pop()` is the only remover that returns the item
- `sort()` is stable; equal items keep their order
- For a new sorted list, use `sorted()`

METHODS IN ORDER

```
xs = [3, 1, 2]
xs.append(4)                    # [3, 1, 2, 4]
xs.extend([5, 6])               # [3, 1, 2, 4, 5, 6]
xs.insert(0, 0)                 # [0, 3, 1, 2, 4, 5, 6]
xs.remove(3)                    # [0, 1, 2, 4, 5, 6]  first match
xs.pop()                        # 6  → [0, 1, 2, 4, 5]
xs.pop(0)                       # 0  → [1, 2, 4, 5]
xs.index(4), xs.count(2)        # (2, 1)
xs.sort(reverse=True)           # [5, 4, 2, 1]
xs.reverse()                    # [1, 2, 4, 5]
ys = xs.copy()                  # shallow copy
xs.clear()                      # []
result = [3, 1].sort()          # None! sorts in place
```

CODE EXPLAINED

- → `insert(i, x)` — x lands at index i
- → `remove(x)` — first matching value; `ValueError` if absent
- → `pop(i)` — removes and returns; default is last
- → `sort(key=..., reverse=...)` — same options as `sorted()`

PYTHON VERSIONS

- **3.3** — `list.copy()` and `list.clear()`

THE PYTHONIC WAY

- ✗ ~~`xs = xs + [x]`~~ → ✓ `xs.append(x)`
- ✗ ~~`for y in ys: xs.append(y)`~~ → ✓ `xs.extend(ys)`
- ✗ ~~`xs.pop(0)`~~ ~~as a queue~~ → ✓ `deque.popleft()`

ADD & REMOVE

METHOD	DOES
`append(x)`	Add x at end
`extend(it)`	Add each item
`insert(i, x)`	Add at index
`remove(x)`	Delete first x
`pop(i=-1)`	Delete + return
`clear()`	Delete all

ORDER & QUERY

METHOD	RETURNS
`sort(key, reverse)`	None (in place)
`reverse()`	None (in place)
`index(x)`	Position or ValueError
`count(x)`	Occurrences
`copy()`	Shallow copy

SPEED

OPERATION	COST
`append / pop()`	O(1)
`pop(0) / insert(0, x)`	O(n)
`x in lst / index`	O(n)
`sort()`	O(n log n)
`lst[i]`	O(1)

⚠ COMMON MISTAKES

- `lst = lst.sort()` — `lst` becomes `None`
- `a.append([1, 2])` — nests a list; use `extend()`
- `remove(x)` on a missing value — `ValueError`
- `pop(0)` in a loop — slow; use `collections.deque`

✓ TIPS & BEST PRACTICES

- `sorted(xs)` keeps the original; `xs.sort()` changes it
- Use `deque` when you add/remove at the front
- `extend()` accepts generators: `xs.extend(n * 2 for n in ys)`
- Guard `remove`: `if x in xs: xs.remove(x)`

QUICK SUMMARY

- `append` / `extend` / `insert` — add
- `remove` / `pop` / `clear` — delete
- `sort` / `reverse` — reorder in place
- most methods — return `None`

Lists — Advanced Operations

UNPACKING, NESTING, COPYING

Beyond basic methods, lists support **unpacking** with ∗, **nesting** into grids, and two kinds of copies. A **shallow copy** creates a new outer list that still shares inner objects; a **deep copy** duplicates everything. Knowing the difference prevents the most confusing list bugs, especially with nested data.

NOTES

- Only one starred target per unpacking
- `*` in a list literal merges iterables
- `a.copy()`, `a[:]`, `list(a)` are all shallow
- `copy.deepcopy()` handles any nesting depth
- `zip(*matrix)` transposes rows and columns

ADVANCED LIST PATTERNS

```
first, second, *rest = [1, 2, 3, 4]    # 1, 2, [3, 4]
head, *_, tail = "abcde"               # 'a', 'e'
[*range(3), *"ab"]                     # [0, 1, 2, 'a', 'b']
grid = [[0] * 3 for _ in range(2)]     # independent rows
grid[0][1] = 5                         # [[0, 5, 0], [0, 0, 0]]
bad = [[0] * 3] * 2
bad[0][1] = 5                          # [[0, 5, 0], [0, 5, 0]]  shared!
a = [[1, 2], [3]]
b = a.copy()                           # shallow: inner lists shared
b[0].append(9); a                      # [[1, 2, 9], [3]]
import copy
c = copy.deepcopy(a)                   # fully independent
c[0].append(7); a                      # [[1, 2, 9], [3]]  unchanged
list(zip(*[[1, 2], [3, 4]]))           # [(1, 3), (2, 4)]
```

CODE EXPLAINED

- → `*rest` — collects the remaining items
- → `*_` — ignore the middle
- → comprehension grid — each row is a new list
- → `[[0] * 3] * 2` — the same row twice
- → shallow vs deep — outer only vs everything

PYTHON VERSIONS

- **3.0** — starred targets: `first, *rest = xs`
- **3.5** — `*` in literals: `[*a, *b]`

THE PYTHONIC WAY

✗ ~~`first = xs[0]; rest = xs[1:]`~~ → ✓ `first, *rest = xs`

✗ ~~`a + b + c`~~ ~~for lists~~ → ✓ `[*a, *b, *c]`

✗ ~~`[[0] * w] * h`~~ → ✓ `[[0] * w for _ in range(h)]`

COPYING

CODE	KIND
`b = a`	Alias (no copy)
`a.copy()`	Shallow
`a[:]`	Shallow
`list(a)`	Shallow
`copy.deepcopy(a)`	Deep

UNPACKING

CODE	GETS
`a, b = xs`	Exactly 2 items
`a, *b = xs`	First + rest
`*a, b = xs`	Rest + last
`a, *_, b = xs`	Ends only
`[*a, *b]`	Merged list
`f(*xs)`	Items as arguments

LIST OR SOMETHING ELSE?

TYPE	BEST FOR
`list`	General ordered data
`tuple`	Fixed records
`deque`	Fast adds at both ends
`array.array`	Compact numbers
`numpy.ndarray`	Math on arrays

⚠ COMMON MISTAKES

- `[[0] * n] * m` for grids — every row is the same object
- Expecting `.copy()` to copy nested lists — it's shallow
- `a, b = [1, 2, 3]` — `ValueError: too many values`
- Two starred targets — `SyntaxError`

✓ TIPS & BEST PRACTICES

- Use `copy.deepcopy()` for nested config or game state
- Build grids with a comprehension
- Transpose with `[list(r) for r in zip(*m)]`
- Ignore unpacked values with `_`

QUICK SUMMARY

- `*rest` — star unpacking
- `[*a, *b]` — merge
- shallow copy — shares inner objects
- `deepcopy` — fully independent

Tuples

IMMUTABLE SEQUENCES

A **tuple** is an ordered, **immutable** sequence. Commas create tuples; parentheses are only needed for the empty tuple or to avoid ambiguity. Because tuples can't change, those holding only hashable items are **hashable**, so they work as dict keys and set members. Use them for fixed records, multiple return values, and unpacking. `namedtuple` adds field names.

NOTES

- A one-item tuple needs a trailing comma: `(5,)`
- Only two methods: `count()` and `index()`
- Immutable container, but inner lists can still change
- Slightly smaller and faster than lists
- `typing.NamedTuple` gives a class-style definition

TUPLE BASICS

```
point = (3, 4)
also = 3, 4                          # parentheses optional
single = (5,)                        # trailing comma required
not_tuple = (5)                      # just the int 5
x, y = point                         # unpacking
point[0]                             # 3
point[0] = 9                         # TypeError
(1, 2) + (3,)                        # (1, 2, 3)
locs = {(0, 0): "origin"}            # tuple as dict key
t = ([1], 2); t[0].append(5)         # ([1, 5], 2)
from collections import namedtuple
P = namedtuple("P", "x y")
p = P(3, 4)
p.x, p._asdict()                     # (3, {'x': 3, 'y': 4})
```

CODE EXPLAINED

- → `3, 4` — the comma makes the tuple
- → `(5)` — parentheses alone just group
- → `point[0] = 9` — tuples can't be changed
- → `t[0].append(5)` — the list inside is still mutable
- → `namedtuple` — access by name or index

PYTHON VERSIONS

- **3.6** — class-style `typing.NamedTuple`
- **3.7** — `namedtuple(..., defaults=)`

THE PYTHONIC WAY

✗ ~~`p[0], p[1]` on a record~~ → ✓ `NamedTuple` **fields:** `p.x, p.y`

✗ ~~Returning a list of fixed fields~~ → ✓ `return x, y`

✗ ~~`(5)` for one item~~ → ✓ `(5,)`

OPERATIONS

CODE	RESULT
`t.count(x)`	Occurrences
`t.index(x)`	First position
`len(t)`	Length
`x in t`	Membership
`sorted(t)`	New sorted list
`tuple(lst)`	Convert from list

TUPLE VS LIST

TUPLE	LIST
`Immutable`	Mutable
`Hashable*`	Unhashable
`Fixed record`	Growing collection
`2 methods`	11 methods

WHERE TUPLES APPEAR

CODE	USE
`return a, b`	Multiple returns
`for k, v in d.items()`	Pairs
`*args`	Extra arguments
`a, b = b, a`	Swap
`{(r, c): v}`	Grid keys

⚠ COMMON MISTAKES

- `(5)` is an int — write `(5,)`
- Trying `t.append()` — tuples have no mutating methods
- Using a tuple that contains a list as a dict key — `TypeError`
- Long positional tuples — use `namedtuple` or a dataclass

✓ TIPS & BEST PRACTICES

- Use `typing.NamedTuple` for typed, named fields
- Return tuples and unpack them at the call
- Use tuples for constants: `RGB = (255, 128, 0)`
- `_replace()` makes a modified copy of a namedtuple

QUICK SUMMARY

- `(a, b)` / `a, b` — tuple
- `(x,)` — single item
- immutable — can be dict keys
- `namedtuple` — named fields

Dictionaries — Basics

KEY–VALUE MAPPINGS

A **dict** maps unique, **hashable keys** to values with fast average O(1) lookups and keeps insertion order (3.7+). Create one with `{}` or `dict()`. Reading `d[key]` raises `KeyError` when the key is missing, so use `get()` when absence is normal. Assigning to a key adds it or updates its value.

NOTES

- Keys must be hashable: str, int, tuple, frozenset
- `1`, `1.0`, and `True` count as the same key
- `in` checks keys, not values
- Values can be anything, including other dicts
- Methods: Sheet 51; patterns: Sheet 52

UNDER THE HOOD

- A compact hash table: dense entries + sparse index
- Order is kept because entries append to the dense array
- Lookup: `hash(key)`, probe the slot, then compare with `==`

DICT BASICS

```
user = {"name": "Ana", "age": 30}
also = dict(name="Ana", age=30)
user["name"]                    # 'Ana'
user["email"]                   # KeyError
user.get("email")               # None
user.get("email", "n/a")        # 'n/a'
user["age"] = 31                # update
user["city"] = "Lima"           # add
del user["city"]                # remove
"age" in user                   # True  (keys)
len(user)                       # 2
for key, val in user.items():
    print(f"{key}={val}")       # name=Ana / age=31
```

CODE EXPLAINED

- → `d[key]` — strict lookup
- → `d.get(key, default)` — forgiving lookup
- → `d[key] = v` — add or replace
- → `del d[key]` — remove; `KeyError` if missing
- → `.items()` — loop over key–value pairs

PYTHON VERSIONS

- **3.7** — insertion order guaranteed
- **3.9** — `|` and `|=` merge operators

THE PYTHONIC WAY

✗ ~~`if k in d: v = d[k]` / `else: v = 0`~~ → ✓ `v = d.get(k, 0)`

✗ ~~`for k in d: v = d[k]`~~ → ✓ `for k, v in d.items():`

✗ ~~`d = dict()`~~ → ✓ `d = {}`

VALID KEYS

TYPE	KEY?
`str`	✔
`int / float`	✔
`tuple of immutables`	✔
`frozenset`	✔
`list`	✖ unhashable
`dict / set`	✖ unhashable

BASIC OPERATIONS

CODE	DOES
`d[k]`	Read (KeyError)
`d.get(k, d)`	Read with default
`d[k] = v`	Add / update
`del d[k]`	Remove
`k in d`	Key exists?
`len(d)`	Pair count

CREATING

CODE	RESULT
`{}`	Empty dict
`{"a": 1}`	Literal
`dict(a=1)`	Keyword form
`dict([("a", 1)])`	From pairs
`dict(zip(ks, vs))`	From two lists
`dict.fromkeys(ks, 0)`	Same default

⚠ COMMON MISTAKES

- `d[k]` on a missing key — `KeyError`; use `get()` or `in`
- `s = {}` expecting a set — it's an empty dict
- Using a list as a key — `TypeError: unhashable type`
- `d[1]` and `d[True]` — the same slot

✓ TIPS & BEST PRACTICES

- Use `get()` with a default for optional fields
- Loop with `.items()` instead of `d[k]` lookups
- Nested dicts mirror JSON naturally
- Use `collections.Counter` for counting

QUICK SUMMARY

- `{k: v}` — mapping
- `d[k]` — strict; `get()` — safe
- `k in d` — key check
- keys — hashable only

Dictionaries — Methods

BUILT-IN DICT METHODS

Dict methods read, insert, merge, and remove entries. `keys()`, `values()`, and `items()` return **live views** that reflect later changes. `setdefault()` inserts only when a key is missing, `update()` merges another mapping, and `pop()` removes a key and returns its value, with an optional default to avoid `KeyError`.

NOTES

- Views update when the dict changes
- `keys()` views support set operators
- `popitem()` removes the last-inserted pair
- Changing the dict's size while looping raises `RuntimeError`
- `copy()` is shallow

DICT METHODS

```
d = {"a": 1, "b": 2}
d.keys()                        # dict_keys(['a', 'b'])
d.values()                      # dict_values([1, 2])
d.items()                       # dict_items([('a', 1), ('b', 2)])
d.get("z", 0)                   # 0
d.setdefault("c", 3)            # 3  (inserted)
d.setdefault("a", 99)           # 1  (kept existing)
d.update({"b": 20, "d": 4})     # {'a': 1, 'b': 20, 'c': 3, 'd': 4}
d.pop("d")                      # 4
d.pop("zz", None)               # None  (no KeyError)
d.popitem()                     # ('c', 3)  last inserted
d.copy()                        # {'a': 1, 'b': 20}
d.clear()                       # {}
```

CODE EXPLAINED

→ views — iterate, `len()`, `in`, but no indexing

→ `setdefault(k, v)` — get, inserting v if missing

→ `update()` — merges; incoming values win

→ `pop(k, default)` — remove safely

PYTHON VERSIONS

- **3.8** — `reversed()` works on dicts and views
- **3.10** — views expose `.mapping`

THE PYTHONIC WAY

✗ ~~`if k not in d: d[k] = []`~~ → ✓ `d.setdefault(k, []).append(x)`

✗ ~~`for k in d.keys():`~~ → ✓ `for k in d:`

✗ ~~`d2 = {}; d2.update(d1)`~~ → ✓ `d2 = d1.copy()` **or** `{**d1}`

READING

METHOD	RETURNS
`keys()`	View of keys
`values()`	View of values
`items()`	View of (k, v)
`get(k, d=None)`	Value or d
`copy()`	Shallow copy

WRITING

METHOD	DOES
`setdefault(k, v)`	Insert if missing
`update(m, **kw)`	Merge in
`pop(k, d)`	Remove + return
`popitem()`	Remove last pair
`clear()`	Remove all

VIEW TRICKS

CODE	RESULT
`d1.keys() & d2.keys()`	Shared keys
`d1.keys() - d2.keys()`	Keys only in d1
`list(d)`	Key snapshot
`sorted(d.items())`	Pairs sorted by key

⚠ COMMON MISTAKES

- Deleting keys while iterating — loop over `list(d)` instead
- `d.pop(k)` with no default on a missing key — `KeyError`
- `setdefault(k, expensive())` — the default is built every call
- Holding a view and expecting a snapshot — views are live

✓ TIPS & BEST PRACTICES

- Use set operators on `keys()` to compare dicts
- `d.update(port=8080)` accepts keyword arguments
- Prefer `defaultdict` over repeated `setdefault`
- Snapshot with `list(d.items())` before mutating

QUICK SUMMARY

- `keys` / `values` / `items` — live views
- `get` / `setdefault` — safe access
- `update` — merge
- `pop` / `popitem` / `clear` — remove

Dictionaries — Advanced

MERGING, GROUPING, NESTING

Beyond basic methods, dicts support **merging** with | (3.9+), **auto-initializing** values with `defaultdict`, and **nesting** for JSON-like data. These patterns cover most real-world dict work: combining configs, grouping items, counting, and safely reading deep structures.

NOTES

- In `a | b`, b's values win on conflicts
- `defaultdict` creates a missing key on first access
- Sorting a dict gives a list; rebuild with `dict()`
- `OrderedDict` adds `move_to_end()` for LRU logic
- Nested dicts copy shallowly unless you `deepcopy`

ADVANCED DICT PATTERNS

```
from collections import defaultdict
base = {"host": "localhost", "port": 80}
extra = {"port": 8080}
base | extra                        # {'host': 'localhost', 'port': 8080}
{**base, **extra}                   # same result (any 3.x)
base |= extra                       # merge in place (3.9+)
groups = defaultdict(list)
for word in ["apple", "avocado", "kiwi"]:
    groups[word[0]].append(word)    # no KeyError
dict(groups)                        # {'a': ['apple', 'avocado'], 'k': ['kiwi']}
cfg = {"db": {"host": "x", "port": 5432}}
cfg["db"]["port"]                   # 5432
cfg.get("cache", {}).get("ttl", 60)  # 60  safe nested read
scores = {"ana": 9, "bo": 7}
sorted(scores, key=scores.get)      # ['bo', 'ana']
max(scores, key=scores.get)         # 'ana'
```

CODE EXPLAINED

- → `|` — new merged dict; `|=` — in place
- → `defaultdict(list)` — missing keys start as `[]`
- → chained `.get(..., {})` — no `KeyError` on missing levels
- → `key=scores.get` — sort or pick keys by their values

PYTHON VERSIONS

- **3.7** — plain `dict` keeps order; `OrderedDict` rarely needed
- **3.9** — `d1 | d2` and `d1 |= d2`

THE PYTHONIC WAY

- ✗ ~~`{**a, **b}`~~ → ✓ `a | b` **(3.9+)**
- ✗ ~~`if k not in d: d[k] = []` everywhere~~ → ✓ `defaultdict(list)`
- ✗ ~~`OrderedDict` for plain ordering~~ → ✓ **a regular** `dict`

MERGING

CODE	RESULT
`a | b`	New dict
`a |= b`	Update a
`{**a, **b}`	New dict
`a.update(b)`	Update a
`ChainMap(b, a)`	Layered view, no copy

DEFAULTDICT FACTORIES

FACTORY	MISSING KEY STARTS AS
`list`	[] (grouping)
`set`	set() (unique groups)
`int`	0 (counting)
`dict`	{} (nesting)
`lambda: "n/a"`	Custom default

PATTERNS

CODE	DOES
`{v: k for k, v in d.items()}`	Invert
`dict(sorted(d.items()))`	Sort by key
`Counter(words)`	Count
`od.move_to_end(k)`	Mark recently used

⚠ COMMON MISTAKES

- Reading a `defaultdict` key to test it — the read creates the key
- Assuming `|` keeps the left value on conflict — the right side wins
- `sorted(d)` expecting a dict — you get a list of keys
- Copying nested configs with `.copy()` — inner dicts are shared

✓ TIPS & BEST PRACTICES

- Test with `k in dd` on defaultdicts to avoid insertion
- Layer settings: `defaults | file_cfg | cli_args`
- Use `TypedDict` to document dict shapes for type checkers
- Convert to a plain dict with `dict(dd)` before printing

QUICK SUMMARY

- `a | b` — merge (right wins)
- `defaultdict` — automatic defaults
- nested `.get()` — safe deep reads
- `key=d.get` — rank keys by value

Sets

UNIQUE, UNORDERED COLLECTIONS

A **set** is an unordered collection of **unique, hashable** items with O(1) average membership tests. Sets remove duplicates automatically and support math-style operations: union, intersection, difference, and symmetric difference. `frozenset` is the immutable, hashable version. Note that `{}` creates a dict; use `set()` for an empty set.

NOTES

- No indexing or slicing; order is arbitrary
- Items must be hashable (no lists or dicts)
- `discard()` ignores missing items; `remove()` raises
- `pop()` removes an arbitrary item
- Operators need sets; methods accept any iterable

UNDER THE HOOD

- A hash table of keys with no values
- `in` is O(1) on average: hash, then probe
- `frozenset` is hashable because it can't change

SET OPERATIONS

```
a = {1, 2, 3}; b = {3, 4}
empty = set()                    # {} would be a dict
set([1, 1, 2])                   # {1, 2}
a.add(4)                         # {1, 2, 3, 4}
a.discard(9)                     # no error if missing
a.remove(9)                      # KeyError
a | b                            # {1, 2, 3, 4}  union
a & b                            # {3, 4}  intersection
a - b                            # {1, 2}  difference
a ^ b                            # {1, 2}  in one, not both
{1, 2} <= a                      # True  subset
a.isdisjoint({7, 8})             # True  nothing shared
fs = frozenset([1, 2])           # immutable, hashable
```

CODE EXPLAINED

→ `set(iterable)` — dedupe anything iterable

→ `|` `&` `-` `^` — set algebra

→ `<=` / `>=` — subset / superset

→ `frozenset` — usable as a dict key or set member

THE PYTHONIC WAY

✗ ~~`list(set(xs))` when order matters~~ → ✓ `list(dict.fromkeys(xs))`

✗ ~~`[x for x in a if x in b]` (lists)~~ → ✓ `set(a) & set(b)`

✗ ~~`s = {}` for an empty set~~ → ✓ `s = set()`

OPERATORS & METHODS

OP	METHOD
`a | b`	a.union(b)
`a & b`	a.intersection(b)
`a - b`	a.difference(b)
`a ^ b`	a.symmetric_difference(b)
`a <= b`	a.issubset(b)
`a >= b`	a.issuperset(b)

CHANGING A SET

METHOD	DOES
`add(x)`	Add one
`update(it)`	Add many (|=)
`discard(x)`	Remove if present
`remove(x)`	Remove or KeyError
`pop()`	Remove arbitrary
`clear()`	Remove all

COMMON USES

CODE	GETS
`set(xs)`	Unique items
`len(set(xs))`	Distinct count
`x in s`	Fast lookup
`set(a) - set(b)`	Items only in a
`set(a) & set(b)`	Shared items

⚠ COMMON MISTAKES

- `s = {}` for an empty set — it's a dict
- Indexing `s[0]` — sets aren't ordered
- Adding a list — `TypeError: unhashable type`
- Relying on `pop()` returning a particular item

✓ TIPS & BEST PRACTICES

- Convert lists to sets for repeated `in` checks
- Use `sorted(s)` when you need ordered output
- Methods accept lists: `a.union([5, 6])`
- Dedupe but keep order: `list(dict.fromkeys(xs))`

QUICK SUMMARY

- `{1, 2}` / `set()` — unique items
- `| & - ^` — set algebra
- O(1) `in` — fast membership
- `frozenset` — immutable set

Collections Module

SPECIALIZED CONTAINERS

The `collections` module adds containers for common jobs. `Counter` counts hashable items, `defaultdict` supplies missing values, `deque` gives fast appends and pops at both ends, `namedtuple` creates lightweight records, and `ChainMap` layers several dicts into one view without copying.

NOTES

- `Counter` returns 0 for missing keys, no `KeyError`
- `deque(maxlen=n)` drops old items automatically
- `deque` indexing in the middle is O(n)
- `OrderedDict` still offers `move_to_end()`
- All are in the standard library; no install needed

COLLECTIONS IN ACTION

```
from collections import Counter, defaultdict, deque, namedtuple, ChainMap
c = Counter("mississippi")
c.most_common(2)                    # [('i', 4), ('s', 4)]
c["s"], c["z"]                      # (4, 0)  missing → 0
Counter(a=3) + Counter(a=1, b=2)    # Counter({'a': 4, 'b': 2})
dd = defaultdict(int)
for w in "a b a".split(): dd[w] += 1   # {'a': 2, 'b': 1}
q = deque([1, 2, 3], maxlen=3)
q.appendleft(0)                     # deque([0, 1, 2])  3 dropped
q.rotate(1)                         # deque([2, 0, 1])
Point = namedtuple("Point", ["x", "y"])
Point(1, 2).x                       # 1
defaults = {"theme": "light"}; user = {"theme": "dark"}
ChainMap(user, defaults)["theme"]  # 'dark'
```

CODE EXPLAINED

- → `most_common(n)` — top n by count
- → Counters add, subtract, and compare
- → `maxlen` — a bounded buffer for recent items
- → `ChainMap` — first mapping with the key wins

PYTHON VERSIONS

- **3.3** — `ChainMap`
- **3.7** — `namedtuple(defaults=)`
- **3.10** — `Counter.total()`

THE PYTHONIC WAY

- ✗ ~~`d[w] = d.get(w, 0) + 1` loop~~ → ✓ `Counter(words)`
- ✗ ~~`list.insert(0, x)` / `pop(0)`~~ → ✓ `deque.appendleft()` / `popleft()`
- ✗ ~~`p = ("Ana", 30)` for records~~ → ✓ `Person = namedtuple("Person", "name age")`

CLASSES

CLASS	USE
`Counter`	Count items
`defaultdict`	Auto-default values
`deque`	Queue / stack / buffer
`namedtuple`	Named record
`ChainMap`	Layered lookups
`OrderedDict`	Reorderable dict

COUNTER METHODS

METHOD	DOES
`most_common(n)`	Top n pairs
`elements()`	Items repeated by count
`update(it)`	Add counts
`subtract(it)`	Remove counts
`total()`	Sum of counts (3.10+)

DEQUE METHODS

METHOD	DOES
`append / appendleft`	Add right / left
`pop / popleft`	Remove right / left
`extendleft(it)`	Add many on left
`rotate(n)`	Shift right by n
`maxlen`	Size cap

⚠ COMMON MISTAKES

- Using a list as a queue — `pop(0)` is O(n); use `deque.popleft()`
- Expecting `Counter` subtraction to keep negatives — `-` drops them
- Indexing deep into a `deque` — slow in the middle
- Mutating a `namedtuple` field — it's immutable; use `_replace()`

✓ TIPS & BEST PRACTICES

- Count words: `Counter(text.split()).most_common(10)`
- Keep the last N log lines: `deque(lines, maxlen=N)`
- Use `typing.NamedTuple` or a dataclass for richer records
- `ChainMap(cli, env, defaults)` for layered settings

QUICK SUMMARY

- `Counter` — counting
- `defaultdict` — missing-key defaults
- `deque` — fast double-ended queue
- `namedtuple` / `ChainMap` — records, layers

itertools Module

ITERATOR BUILDING BLOCKS

`itertools` provides fast, memory-efficient tools that build and combine iterators. There are three families: **infinite** iterators (`count`, `cycle`, `repeat`), **combinatoric** generators (`product`, `permutations`, `combinations`), and **terminating** tools that slice, chain, group, and accumulate. All return lazy iterators, so wrap them in `list()` to see the values.

NOTES

- Always limit infinite iterators with `islice` or `break`
- `groupby()` groups **consecutive** keys; sort first
- `pairwise` (3.10+) and `batched` (3.12+) are newer
- Results are lazy and single-pass
- `more-itertools` (PyPI) adds many more recipes

ITERTOOLS HIGHLIGHTS

```
from itertools import (count, cycle, repeat, chain, islice, combinations,
                       permutations, product, accumulate, groupby, pairwise)
list(islice(count(10, 5), 3))          # [10, 15, 20]
list(islice(cycle("AB"), 5))           # ['A', 'B', 'A', 'B', 'A']
list(repeat("x", 3))                   # ['x', 'x', 'x']
list(chain([1, 2], (3,), "ab"))        # [1, 2, 3, 'a', 'b']
list(combinations("ABC", 2))           # [('A','B'), ('A','C'), ('B','C')]
len(list(permutations("ABC", 2)))      # 6
list(product([0, 1], repeat=2))        # [(0,0), (0,1), (1,0), (1,1)]
list(accumulate([1, 2, 3, 4]))         # [1, 3, 6, 10]
list(pairwise([1, 2, 3]))              # [(1, 2), (2, 3)]
words = sorted(["apple", "bean", "avocado"])
for k, grp in groupby(words, key=lambda w: w[0]):
    print(k, list(grp))                # a ['apple', 'avocado'] / b ['bean']
```

CODE EXPLAINED

- → `islice` — take part of any iterator
- → `chain` — iterate several iterables as one
- → `combinations` — order doesn't matter; `permutations` — it does
- → `groupby` — needs input sorted by the same key

PYTHON VERSIONS

- **3.10** — `pairwise()`
- **3.12** — `batched()`
- **3.13** — `batched(..., strict=True)`

THE PYTHONIC WAY

✗ ~~Nested `for` loops over two lists~~ → ✓ `itertools.product(a, b)`

✗ ~~`for i in range(len(xs) - 1): xs[i], xs[i+1]`~~ → ✓ `itertools.pairwise(xs)`

✗ ~~`sum(lists, [])`~~ → ✓ `list(chain.from_iterable(lists))`

INFINITE

FUNCTION	YIELDS
`count(start, step)`	start, start+step, …
`cycle(it)`	it repeated forever
`repeat(x, n)`	x, n times (or forever)

COMBINATORIC

FUNCTION	EXAMPLE
`product(a, b)`	Cartesian pairs
`permutations(it, r)`	Ordered r-tuples
`combinations(it, r)`	Unordered r-tuples
`combinations_with_replacement`	Items may repeat

TERMINATING

FUNCTION	DOES
`chain(*its)`	Concatenate
`islice(it, n)`	First n
`accumulate(it)`	Running totals
`groupby(it, key)`	Consecutive groups
`takewhile(p, it)`	Until p fails
`batched(it, n)`	n-sized chunks (3.12+)
`zip_longest(a, b)`	Pad shorter input

⚠ COMMON MISTAKES

- `groupby()` on unsorted data — the same key appears in several groups
- `list(count())` — never finishes
- Reusing an itertools iterator — it's exhausted after one pass
- Consuming `grp` after moving to the next group — it's gone

✓ TIPS & BEST PRACTICES

- Replace nested loops with `product()`
- Flatten one level with `chain.from_iterable(lists)`
- Running totals: `accumulate(xs)`; running max: `accumulate(xs, max)`
- Chunk large data with `batched(rows, 500)`

QUICK SUMMARY

- infinite — `count` `cycle` `repeat`
- combinatoric — `product` `combinations`
- terminating — `chain` `islice` `groupby`
- lazy — wrap in `list()` to view

Sorting & Searching

ORDERING AND FINDING DATA

`sorted()` returns a new sorted list from any iterable; `list.sort()` sorts a list in place and returns `None`. Both accept `key=` (a function that extracts the sort value) and `reverse=True`. Python's sort is **stable**, so items with equal keys keep their order. For fast searching in sorted data, use the `bisect` module.

NOTES

- Sorting uses Timsort: O(n log n)
- Stability lets you sort by several keys in passes
- Mixed types like `[1, "a"]` can't be sorted
- `in` and `index()` are linear searches
- `heapq` finds the n smallest/largest efficiently

SORTING & SEARCHING

```
nums = [5, 2, 9, 1]
sorted(nums)                        # [1, 2, 5, 9]  new list
sorted(nums, reverse=True)          # [9, 5, 2, 1]
nums.sort()                         # in place; returns None
words = ["banana", "Apple", "cherry"]
sorted(words, key=str.lower)        # case-insensitive
sorted(words, key=len)              # ['Apple', 'banana', 'cherry']
people = [("Ana", 30), ("Bo", 25), ("Cy", 30)]
sorted(people, key=lambda p: (-p[1], p[0]))
# [('Ana', 30), ('Cy', 30), ('Bo', 25)]  age desc, name asc
import bisect
a = [1, 3, 5, 7]
bisect.bisect_left(a, 5)            # 2  (where 5 is)
bisect.insort(a, 4)                 # [1, 3, 4, 5, 7]
nums.index(9)                       # 3  (linear search)
```

CODE EXPLAINED

- → `key=str.lower` — compare by a derived value
- → `key=len` — ties (`banana` , `cherry`) keep original order
- → tuple key with `-` — mix descending and ascending
- → `bisect` — binary search on sorted lists

PYTHON VERSIONS

- **3.0** — `cmp=` removed; use `key=` or `cmp_to_key()`
- **3.10** — `bisect` functions accept `key=`

THE PYTHONIC WAY

- ✗ ~~`xs.sort(); return xs`~~ → ✓ `return sorted(xs)`
- ✗ ~~`key=lambda r: (r[1], r[0])`~~ → ✓ `key=itemgetter(1, 0)`
- ✗ ~~`sorted(xs)[:5]` on big data~~ → ✓ `heapq.nsmallest(5, xs)`

SORTED VS SORT

SORTED(X)	X.SORT()
`Any iterable`	Lists only
`New list`	In place
`Original kept`	Original changed
`Returns list`	Returns None

KEY FUNCTIONS

KEY	SORTS BY
`str.lower`	Case-insensitive
`len`	Length
`abs`	Magnitude
`itemgetter(1)`	Second field
`attrgetter("age")`	Attribute
`lambda x: (a, b)`	Several fields

SEARCHING

TOOL	COST
`x in lst`	O(n)
`lst.index(x)`	O(n)
`x in set_`	O(1) avg
`bisect_left(a, x)`	O(log n)
`heapq.nsmallest(k, xs)`	O(n log k)

⚠ COMMON MISTAKES

- `x = x.sort()` — `x` becomes `None`
- `reverse=True` with a tuple key when only one field should flip — negate that field instead
- Sorting mixed types — `TypeError`
- Using `bisect` on unsorted data — wrong answers, no error

✓ TIPS & BEST PRACTICES

- Use `operator.itemgetter` for record sorts
- Get top items with `heapq.nlargest(5, xs, key=...)`
- Keep a list sorted while inserting with `bisect.insort`
- Sort dict items by value: `sorted(d.items(), key=lambda kv: kv[1])`

QUICK SUMMARY

- `sorted()` — new list
- `.sort()` — in place, `None`
- `key=` / `reverse=` — control order
- `bisect` — binary search

Classes & Objects

BLUEPRINTS AND INSTANCES

A **class** is a blueprint that bundles data (**attributes**) with behavior (**methods**). Calling the class creates an **instance**. Every method receives the instance as its first parameter, named `self` by convention, which is how methods read and change that object's data. Attributes set on `self` belong to one instance; attributes in the class body are shared.

NOTES

- Class names use PascalCase
- Everything in Python is an object of some class
- `self` is passed automatically: `rex.sit()` → `Dog.sit(rex)`
- Add `__repr__` for readable debugging output
- Data-only classes: see `@dataclass` on Sheet 58

UNDER THE HOOD

- Instance attributes live in `obj.__dict__`
- `obj.method` creates a bound method holding `self`
- `rex.sit()` runs `Dog.sit(rex)`

A COMPLETE SMALL CLASS

```
class Dog:
    """A dog with a name and tricks."""
    species = "Canis familiaris"        # class attribute (shared)

    def __init__(self, name, age):
        self.name = name                # instance attributes
        self.age = age
        self.tricks = []

    def add_trick(self, trick):         # method
        self.tricks.append(trick)

    def __repr__(self):
        return f"Dog({self.name!r}, {self.age})"

rex = Dog("Rex", 3)                     # create an instance
rex.add_trick("sit")
rex.name, rex.tricks                    # ('Rex', ['sit'])
rex                                     # Dog('Rex', 3)
```

CODE EXPLAINED

- → `class Dog:` — defines the blueprint
- → `__init__` — sets up each new instance
- → `self.tricks = []` — a separate list per dog
- → `rex.add_trick("sit")` — calls the method on rex
- → `__repr__` — what the REPL shows for the object

THE PYTHONIC WAY

- ✗ ~~`get_name()` / `set_name()` methods~~ → ✓ **plain attribute `p.name`**
- ✗ ~~Class with only `__init__` storing fields~~ → ✓ `@dataclass`
- ✗ ~~No `__repr__`~~ → ✓ `__repr__` **that looks like the call**

TERMS

TERM	MEANING
`class`	Blueprint / type
`instance`	One object built from it
`attribute`	Data on an object
`method`	Function defined in a class
`self`	The current instance

INSPECTING OBJECTS

CALL	RETURNS
`type(rex)`	<class 'Dog'>
`isinstance(rex, Dog)`	True
`vars(rex)`	Instance attribute dict
`dir(rex)`	All attribute names
`getattr(rex, "age")`	3
`setattr(rex, "age", 4)`	Sets age

CLASS FORMS

CODE	MEANING
`class A:`	New class
`class A(B):`	Subclass of B
`class A: pass`	Empty class
`A()`	Create instance
`A`	The class itself

⚠ COMMON MISTAKES

- Leaving out `self` in a method — `TypeError` about arguments
- Writing `name = name` instead of `self.name = name` in `__init__`
- Mutable class attribute `tricks = []` — shared by every dog
- `rex = Dog` without `()` — you get the class, not an instance

✓ TIPS & BEST PRACTICES

- Always give classes a `__repr__`
- Keep each class focused on one responsibility
- Use `@dataclass` when a class mainly holds data
- Initialize every attribute in `__init__`, even to `None`

QUICK SUMMARY

- `class` — define a type
- `Dog(...)` — create an instance
- `self.attr` — per-instance data
- methods — functions that take `self`

Constructors & Initialization

INITIALIZING INSTANCES

When you call a class, Python creates the object with `__new__` and then runs `__init__` to initialize it. `__init__` receives the new object as `self` plus the call's arguments, sets attributes, and must return `None`. Python has no constructor overloading; use default arguments or `@classmethod` **alternative constructors** instead. `@dataclass` writes `__init__` for you.

NOTES

- `__init__` initializes; `__new__` creates (rarely overridden)
- Validate arguments in `__init__` and raise early
- Subclasses must call `super().__init__(...)`
- Name alternative constructors `from_x()`
- `__post_init__` runs after a dataclass's `__init__`

__INIT__, ALTERNATIVE CONSTRUCTOR, DATACLASS

```
class Account:
    def __init__(self, owner, balance=0.0):
        if balance < 0:
            raise ValueError("balance must be >= 0")
        self.owner = owner
        self.balance = balance
        self.history = []               # fresh list per instance
    @classmethod
    def from_csv(cls, line):            # alternative constructor
        owner, bal = line.split(",")
        return cls(owner, float(bal))
b = Account.from_csv("Bo,25.5")         # owner='Bo', balance=25.5
from dataclasses import dataclass, field
@dataclass
class Item:
    name: str
    price: float = 0.0
    tags: list = field(default_factory=list)
Item("pen", 1.5)                        # Item(name='pen', price=1.5, tags=[])
```

CODE EXPLAINED

- → default `balance=0.0` — optional argument
- → `raise ValueError` — refuse invalid objects
- → `cls(...)` — works for subclasses too
- → `field(default_factory=list)` — safe mutable default

PYTHON VERSIONS

- **3.7** — `@dataclass` (PEP 557)
- **3.10** — `@dataclass(slots=True, kw_only=True)`

THE PYTHONIC WAY

- ✗ ~~`__init__` that just copies args to self~~ → ✓ `@dataclass`
- ✗ ~~Overloaded `__init__(self, *args)`~~ → ✓ `@classmethod from_dict(cls, d)`
- ✗ ~~Validation scattered in methods~~ → ✓ **check in** `__init__` / `__post_init__`

WHAT A(...) DOES

STEP	METHOD
1. Create object	A.__new__(A)
2. Initialize	obj.__init__(...)
3. Return	The instance
__init__ returns	None (always)

CONSTRUCTOR PATTERNS

PATTERN	USE
Defaults	Optional args
Validation	Reject bad data
@classmethod from_x	Other input formats
super().__init__()	Parent setup
@dataclass	Generated __init__

DATACLASS OPTIONS

OPTION	EFFECT
frozen=True	Immutable, hashable
order=True	Adds < > <= >=
slots=True	Uses __slots__ (3.10+)
kw_only=True	Keyword-only fields (3.10+)
__post_init__	Extra setup / validation

⚠ COMMON MISTAKES

- Returning a value from `__init__` — `TypeError`
- Mutable default parameter `history=[]` — shared across instances
- Forgetting `super().__init__()` in a subclass — parent attributes missing
- Misspelling `__init__` (e.g. `_init_`) — never called, no error

✓ TIPS & BEST PRACTICES

- Use `@dataclass` for classes that mostly hold data
- Add `from_dict`, `from_json`, `from_csv` classmethods
- Keep `__init__` cheap; no network or file I/O
- Validate dataclass fields in `__post_init__`

QUICK SUMMARY

- `__init__(self, ...)` — initialize
- `@classmethod` — alternative constructors
- `@dataclass` — generated `__init__`
- `default_factory` — safe mutable defaults

Instance, Class & Static Members

THREE KINDS OF MEMBERS

Instance attributes and methods belong to one object and use `self`. **Class** attributes live on the class and are shared by every instance; `@classmethod` methods receive the class as `cls`. `@staticmethod` methods receive neither; they're plain functions grouped inside the class because they relate to it.

NOTES

- Lookup order: instance → class → base classes
- Assigning through an instance creates an instance attribute
- Classmethods are inherited and receive the subclass
- Static methods can't see `self` or `cls`

ALL THREE ON ONE CLASS

```
class Pizza:
    count = 0                          # class attribute (shared)
    def __init__(self, size):
        self.size = size               # instance attribute
        Pizza.count += 1
    def area(self):                    # instance method
        return 3.14159 * (self.size / 2) ** 2
    @classmethod
    def small(cls):                    # class method
        return cls(8)
    @staticmethod
    def is_valid(size):                # static method
        return 6 <= size <= 18
p = Pizza(12)
round(p.area(), 1)                     # 113.1
Pizza.small().size                     # 8
Pizza.is_valid(20)                     # False
Pizza.count                            # 2
p.count = 99                           # new instance attribute!
Pizza.count                            # still 2
```

CODE EXPLAINED

- → `Pizza.count += 1` — update shared data via the class
- → `self.size` — unique per pizza
- → `cls(8)` — builds an instance of whatever class called it
- → `p.count = 99` — shadows, doesn't change, the class attribute

THE PYTHONIC WAY

- ✗ ~~`@staticmethod` factory~~ → ✓ `@classmethod` **returning** `cls(...)`
- ✗ ~~Utility class full of static methods~~ → ✓ **module-level functions**
- ✗ ~~`self.count += 1` on a class counter~~ → ✓ `type(self).count += 1`

COMPARISON

KIND	FIRST PARAM	USE FOR
`Instance method`	self	Per-object behavior
`@classmethod`	cls	Factories, class data
`@staticmethod`	(none)	Related helpers
`Instance attr`	—	Per-object data
`Class attr`	—	Shared constants/counters

ATTRIBUTE LOOKUP

ACCESS	FINDS
`p.size`	Instance dict
`p.count`	Class (if not on instance)
`Pizza.count`	Class
`p.__dict__`	Instance attributes
`Pizza.__dict__`	Class attributes

CALLING

CALL	WORKS?
`p.area()`	✔
`Pizza.area(p)`	✔ (explicit self)
`Pizza.small()`	✔
`p.is_valid(10)`	✔
`Pizza.area()`	✖ missing self

⚠ COMMON MISTAKES

- Mutable class attribute (`items = []`) — shared by all instances
- `self.count += 1` — creates an instance copy instead of updating the class
- Static method that needs class data — make it a classmethod
- Forgetting `@classmethod` — `cls` receives an instance or nothing

✓ TIPS & BEST PRACTICES

- Update shared counters through the class name or `cls`
- Use classmethods for factories so subclasses work
- Use class attributes for constants: `MAX_SIZE = 18`
- If a static method doesn't relate to the class, move it out

QUICK SUMMARY

- instance — `self`, per object
- `@classmethod` — `cls`, per class
- `@staticmethod` — no implicit arg
- class attrs — shared

Properties & Access Control

CONTROLLED ATTRIBUTE ACCESS

A **property** looks like a plain attribute to callers but runs methods behind the scenes, so you can validate values, compute results, or make attributes read-only without changing the public interface. Python has no `private` keyword: a single underscore marks something internal by convention, and a double underscore triggers **name mangling** to avoid clashes in subclasses.

NOTES

- Start with plain attributes; add a property later if needed
- Store the real value under `_name`
- A property without a setter is read-only
- `__slots__` blocks new attributes and saves memory
- `functools.cached_property` computes once, then caches

UNDER THE HOOD

- `property` is a descriptor with `__get__` / `__set__`
- Data descriptors win over the instance `__dict__`
- `__slots__` swaps `__dict__` for fixed slots

VALIDATED AND COMPUTED PROPERTIES

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius          # runs the setter
    @property
    def celsius(self):
        return self._celsius
    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value
    @property
    def fahrenheit(self):               # computed, read-only
        return self._celsius * 9 / 5 + 32
t = Temperature(20)
t.fahrenheit                            # 68.0
t.celsius = -300                        # ValueError
t.fahrenheit = 50                       # AttributeError (no setter)
```

CODE EXPLAINED

- → `@property` — getter; accessed without `()`
- → `@celsius.setter` — runs on assignment
- → `self._celsius` — the stored value
- → `__init__` uses the setter, so validation always applies

PYTHON VERSIONS

- **3.8** — `functools.cached_property`

THE PYTHONIC WAY

- ✗ ~~`get_x()` / `set_x()` up front~~ → ✓ **plain attribute; add `@property` later**
- ✗ ~~Computing an expensive value every access~~ → ✓ `@cached_property`
- ✗ ~~`__private` for every attribute~~ → ✓ `_internal` **convention**

NAMING CONVENTIONS

NAME	MEANING
`name`	Public
`_name`	Internal (convention)
`__name`	Mangled to _Class__name
`__name__`	Python special name
`name_`	Avoids keyword clash

PROPERTY FORMS

CODE	DOES
`@property`	Getter
`@x.setter`	Setter
`@x.deleter`	Deleter
`property(get, set)`	Function form
`@cached_property`	Compute once

__SLOTS__

EFFECT	DETAIL
`__slots__ = ("x", "y")`	Fixed attribute names
`No __dict__`	Less memory per object
`Typos`	AttributeError, not silent
`Inheritance`	Subclasses need their own

⚠ COMMON MISTAKES

- Setter assigns `self.celsius` — infinite recursion; use `self._celsius`
- Setter name not matching the property — creates a separate property
- Expensive work in a getter — callers assume attribute access is cheap
- Treating `_name` as enforced privacy — it's only a convention

✓ TIPS & BEST PRACTICES

- Don't write getters/setters up front; Python lets you add them later
- Use `cached_property` for costly values that never change
- Add `__slots__` when creating millions of small objects
- Expose read-only data as a property without a setter

QUICK SUMMARY

- `@property` — method as attribute
- `@x.setter` — validate on assignment
- `_name` — internal by convention
- `__name` — name mangling

Inheritance

EXTENDING CLASSES

Inheritance lets a subclass reuse and extend a base class: `class Dog(Animal):`. The subclass gets every attribute and method of the parent, can **override** methods by redefining them, and can call the parent's version with `super()`. Every class ultimately inherits from `object`. Use inheritance for true "is-a" relationships.

NOTES

- Method lookup checks the subclass first, then parents
- `super()` reaches the next class in the MRO (Sheet 62)
- `typing.override` (3.12+) marks intended overrides
- For "has-a" relationships, use composition instead

SUBCLASS, OVERRIDE, SUPER()

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
    def describe(self):
        return f"{self.name} says {self.speak()}"
class Dog(Animal):                      # Dog inherits from Animal
    def __init__(self, name, breed):
        super().__init__(name)          # run parent setup
        self.breed = breed
    def speak(self):                    # override
        return "Woof"
d = Dog("Rex", "lab")
d.describe()                            # 'Rex says Woof'
isinstance(d, Animal)                   # True
issubclass(Dog, Animal)                 # True
```

CODE EXPLAINED

- → `class Dog(Animal)` — Animal is the base class
- → `super().__init__(name)` — reuse the parent's setup
- → `speak()` override — `describe()` now calls Dog's version
- → `isinstance` — a Dog is also an Animal

PYTHON VERSIONS

- **3.0** — zero-argument `super()`
- **3.12** — `@typing.override`

THE PYTHONIC WAY

- ✗ ~~`Parent.__init__(self, ...)`~~ → ✓ `super().__init__(...)`
- ✗ ~~Inheriting to reuse one helper~~ → ✓ **composition:** `self.helper = Helper()`
- ✗ ~~`type(a) == Animal`~~ → ✓ `isinstance(a, Animal)`

TERMS

TERM	MEANING
`base / parent`	Class inherited from
`subclass / child`	Class that inherits
`override`	Redefine a method
`extend`	Override + call super()
`object`	Root of every class

SUPER() USES

CODE	CALLS
`super().__init__(...)`	Parent initializer
`super().method()`	Parent's version
`return super().x() + 1`	Extend result
`super().__repr__()`	Parent repr

RELATIONSHIP CHECKS

CALL	ANSWERS
`isinstance(o, C)`	o is a C (or subclass)?
`issubclass(A, B)`	A derives from B?
`C.__bases__`	Direct parents
`C.__mro__`	Full lookup order
`type(o) is C`	Exactly C?

⚠ COMMON MISTAKES

- Skipping `super().__init__()` — parent attributes never get set
- Changing an override's signature — breaks code that treats it like the parent
- Deep hierarchies (4+ levels) — hard to follow; flatten
- Inheriting just to reuse a helper — use composition

✓ TIPS & BEST PRACTICES

- Ask "is a Dog an Animal?" before inheriting
- Keep hierarchies one or two levels deep
- Mark overrides with `@typing.override` (3.12+)
- Put shared behavior in the base; specifics in subclasses

QUICK SUMMARY

- `class B(A)` — B inherits A
- override — redefine a method
- `super()` — call parent version
- `isinstance` — respects inheritance

Multiple Inheritance & MRO

SEVERAL PARENTS

A class can inherit from several bases: `class D(B, C):`. Python decides lookup order with the **Method Resolution Order (MRO)**, computed by C3 linearization: a class comes before its parents, and parents keep their listed left-to-right order. `super()` calls the **next class in the MRO**, not necessarily the parent, which lets cooperative methods run every class exactly once.

NOTES

- Inspect order with `Class.__mro__` or `Class.mro()`
- Each class appears once, `object` last
- An impossible order raises `TypeError` at class creation
- **Mixins** add one small ability to many classes
- Prefer composition when the hierarchy gets tangled

UNDER THE HOOD

- C3 linearization builds the MRO at class creation
- The result is stored in `Cls.__mro__`
- `super()` finds the next class after the current one

DIAMOND AND MIXIN

```
class A:
    def hello(self): return "A"
class B(A):
    def hello(self): return "B>" + super().hello()
class C(A):
    def hello(self): return "C>" + super().hello()
class D(B, C):
    def hello(self): return "D>" + super().hello()
D().hello()                             # 'D>B>C>A'
[k.__name__ for k in D.__mro__]         # ['D', 'B', 'C', 'A', 'object']

class JsonMixin:                        # adds one ability
    def to_json(self):
        import json
        return json.dumps(vars(self))
class User(JsonMixin):
    def __init__(self, name): self.name = name
User("Ana").to_json()                   # '{"name": "Ana"}'
```

CODE EXPLAINED

→ diamond — B and C share parent A

→ B's `super()` goes to C, not A — that's the MRO

→ A runs once, even with two paths to it

→ `JsonMixin` — reusable behavior, no state of its own

THE PYTHONIC WAY

✗ ~~Calling each base's `__init__` by name~~ → ✓ **cooperative `super().__init__(**kw)`**

✗ ~~Big multi-base hierarchies~~ → ✓ **one base + small mixins**

✗ ~~Guessing method lookup~~ → ✓ `print(Cls.__mro__)`

MRO RULES

RULE	EFFECT
`Child first`	Subclass before bases
`Left to right`	Order bases are listed
`Once each`	No duplicates
`object last`	Always the end
`Conflict`	TypeError at definition

COOPERATIVE SUPER()

PRACTICE	WHY
`Every class calls super()`	Chain reaches all
`Accept **kwargs`	Pass extra args along
`Same signature`	Any order works
`No Parent.method(self)`	Avoids double calls

MIXIN GUIDELINES

GUIDELINE	EXAMPLE
`Suffix with Mixin`	JsonMixin
`One ability each`	Serialize, log, compare
`Avoid __init__`	Or make it cooperative
`List before the base`	class U(Mixin, Base)

⚠ COMMON MISTAKES

- Assuming `super()` means "my parent" — it means "next in MRO"
- Calling `A.__init__(self)` directly in a diamond — A runs twice
- Base order that contradicts another class's order — `TypeError`
- Mixins with their own required state — fragile combinations

✓ TIPS & BEST PRACTICES

- Print `Cls.__mro__` whenever lookup surprises you
- Keep multiple inheritance to mixins plus one real base
- Use `**kwargs` passthrough in cooperative `__init__` methods
- Prefer composition for anything more complex

QUICK SUMMARY

- `class D(B, C)` — multiple bases
- MRO — C3 lookup order
- `super()` — next in MRO
- mixins — small add-on behaviors

Polymorphism & Duck Typing

SAME INTERFACE, DIFFERENT BEHAVIOR

Polymorphism means code can call the same method on different types and get type-appropriate behavior. Python does this through method overriding and, more broadly, **duck typing**: if an object has the method you need, it works, regardless of its class. `typing.Protocol` lets type checkers verify that structural contract without inheritance.

NOTES

- "If it walks like a duck and quacks like a duck…"
- Python checks for methods at call time, not class membership
- **EAFP**: try it and catch the error
- **LBYL**: check first with `hasattr` / `isinstance`
- Operator overloading is polymorphism too (Sheet 66)

OVERRIDING, DUCK TYPING, PROTOCOL

```
class Circle:
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2
class Square:
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2
shapes = [Circle(1), Square(2)]
[round(s.area(), 2) for s in shapes]   # [3.14, 4]
class FakeFile:                        # not a file, but has write()
    def write(self, text): print("LOG:", text)
def save(out, text): out.write(text)
save(FakeFile(), "hi")                 # LOG: hi
from typing import Protocol
class HasArea(Protocol):
    def area(self) -> float: ...
def total_area(items: list[HasArea]) -> float:
    return sum(i.area() for i in items)
total_area(shapes)                     # 7.14159
```

CODE EXPLAINED

- → same call `s.area()` — each class answers its own way
- → `FakeFile` — accepted because it has `write()`
- → `Protocol` — describes the needed methods
- → no inheritance needed — Circle never mentions HasArea

PYTHON VERSIONS

- **3.4** — `functools.singledispatch`
- **3.8** — `typing.Protocol` structural typing

THE PYTHONIC WAY

- ✗ ~~`if isinstance(x, Duck): x.quack()`~~ → ✓ **just call** `x.quack()`
- ✗ ~~`if hasattr(f, "read"): f.read()`~~ → ✓ `try: f.read()` + `except AttributeError`
- ✗ ~~`def f(xs: list)`~~ ~~for read-only input~~ → ✓ `def f(xs: Iterable)`

FORMS OF POLYMORPHISM

MECHANISM	EXAMPLE
Overriding	Dog.speak() vs Cat.speak()
Duck typing	Anything with .write()
Operator overloading	__add__ on Vec
singledispatch	Function by argument type
Protocol	Typed duck typing

EAFP VS LBYL

EAFP (PREFERRED)	LBYL
try: x.read()	if hasattr(x, "read"):
except AttributeError:	x.read()
One lookup	Two lookups
No race conditions	State can change

BUILT-IN PROTOCOLS

BEHAVIOR	NEEDS
Iterable	__iter__
Sized	__len__
Callable	__call__
Context manager	__enter__ / __exit__
File-like	read() / write()

⚠ COMMON MISTAKES

- Checking `type(x) == list` — rejects tuples and other sequences
- Long `isinstance` chains — use overriding or `singledispatch`
- Catching `AttributeError` around a big block — hides real bugs
- Overrides with incompatible signatures — callers break

✓ TIPS & BEST PRACTICES

- Accept the most general type you can: any iterable, not just lists
- Use `Protocol` to document duck-typed parameters
- Use `functools.singledispatch` for type-based functions
- Keep EAFP `try` blocks to a single operation

QUICK SUMMARY

- polymorphism — one call, many behaviors
- duck typing — methods matter, not class
- EAFP — try, then handle
- `Protocol` — structural type hints

Abstract Classes & Interfaces

DEFINING REQUIRED METHODS

An **abstract base class** (ABC) defines methods that subclasses **must** implement. Subclass `abc.ABC` and mark methods with `@abstractmethod`; Python then refuses to instantiate the base or any subclass that leaves one unimplemented. ABCs can also provide concrete shared methods. For interfaces checked by structure rather than inheritance, use `typing.Protocol`.

NOTES

- The check happens at instantiation, not at class definition
- Abstract methods can still have a body for `super()` calls
- Combine `@property` with `@abstractmethod` for required attributes
- `collections.abc` has ready-made ABCs
- `@runtime_checkable` enables `isinstance` with Protocols

ABC AND PROTOCOL

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    def describe(self):                # concrete, inherited
        return f"{type(self).__name__}: {self.area():.1f}"
class Rect(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h
Shape()                                # TypeError: can't instantiate
Rect(2, 3).describe()                  # 'Rect: 6.0'
class Bad(Shape): pass
Bad()                                  # TypeError: abstract method 'area'
from typing import Protocol, runtime_checkable
@runtime_checkable
class Closeable(Protocol):
    def close(self) -> None: ...
isinstance(open(__file__), Closeable)  # True (structural match)
```

CODE EXPLAINED

- → `Shape(ABC)` — marks the class as abstract
- → `@abstractmethod` — subclasses must override
- → `describe()` — shared code that relies on `area()`
- → `Protocol` — matches any class with `close()`

PYTHON VERSIONS

- **3.4** — `abc.ABC` helper base class
- **3.8** — `typing.Protocol` and `@runtime_checkable`

THE PYTHONIC WAY

- ✗ ~~Base method with `raise NotImplementedError`~~ → ✓ `@abstractmethod`
- ✗ ~~Forcing inheritance on outside classes~~ → ✓ `typing.Protocol`
- ✗ ~~Writing `__len__`, `__iter__`, … by hand~~ → ✓ **subclass** `collections.abc.Sequence`

ABC TOOLS

TOOL	USE
`ABC`	Base class to inherit
`@abstractmethod`	Required method
`@property + @abstractmethod`	Required attribute
`Shape.register(C)`	Virtual subclass
`ABCMeta`	Metaclass behind ABC

ABC VS PROTOCOL

ABC	PROTOCOL
`Nominal (inherit)`	Structural (match)
`Enforced at runtime`	Checked by mypy/pyright
`Can share code`	Interface only
`You own the classes`	Works with third-party classes

COLLECTIONS.ABC

ABC	REQUIRES
`Iterable`	`__iter__`
`Sequence`	`__getitem__`, `__len__`
`Mapping`	`__getitem__`, `__iter__`, `__len__`
`Callable`	`__call__`
`Hashable`	`__hash__`

⚠ COMMON MISTAKES

- Forgetting to inherit from `ABC` — `@abstractmethod` is ignored
- Misspelling the override name — the class stays abstract
- Using ABCs for one-implementation code — needless ceremony
- Expecting `Protocol` to enforce anything at runtime without `@runtime_checkable`

✓ TIPS & BEST PRACTICES

- Inherit from `collections.abc.Mapping` etc. to get mixin methods free
- Use `Protocol` when you can't change the implementing classes
- Raise `NotImplementedError` only for optional hooks
- Keep interfaces small — two or three methods

QUICK SUMMARY

- `ABC` — abstract base class
- `@abstractmethod` — must override
- can't instantiate — until all are defined
- `Protocol` — structural interfaces

Dunder Methods — Basics

REPRESENTATION & COMPARISON

Dunder (double-underscore) methods let your classes plug into Python's built-in behavior. `__repr__` and `__str__` control how objects display, `__eq__` and `__lt__` define comparisons, and `__hash__` lets objects go in sets and act as dict keys. `@functools.total_ordering` derives the remaining comparisons from `__eq__` plus one ordering method.

NOTES

- `print()` uses `__str__`, falling back to `__repr__`
- Defining `__eq__` sets `__hash__` to `None` unless you define it
- Equal objects must have equal hashes
- Return `NotImplemented` for types you don't handle
- `@dataclass(order=True)` generates comparisons

UNDER THE HOOD

- `a == b` tries `a.__eq__(b)`, then `b.__eq__(a)`
- Both `NotImplemented` → falls back to `a is b`
- Dicts and sets use `__hash__` first, `__eq__` second

A COMPARABLE, HASHABLE CLASS

```python
from functools import total_ordering
@total_ordering
class Version:
    def __init__(self, major, minor):
        self.major, self.minor = major, minor
    def _key(self): return (self.major, self.minor)
    def __repr__(self): return f"Version({self.major}, {self.minor})"
    def __str__(self): return f"v{self.major}.{self.minor}"
    def __eq__(self, other):
        if not isinstance(other, Version): return NotImplemented
        return self._key() == other._key()
    def __lt__(self, other): return self._key() < other._key()
    def __hash__(self): return hash(self._key())
v = Version(1, 2)
str(v), repr(v)                    # ('v1.2', 'Version(1, 2)')
v == Version(1, 2), v < Version(2, 0)   # (True, True)
v >= Version(1, 0)                 # True  (via total_ordering)
len({v, Version(1, 2)})            # 1  (equal + same hash)
```

CODE EXPLAINED

- → `__repr__` — unambiguous, for developers
- → `__str__` — friendly, for users
- → `NotImplemented` — lets Python try the other operand
- → `_key()` — one tuple drives eq, lt, and hash

PYTHON VERSIONS

- **3.7** — `@dataclass(order=True)` writes the comparisons

THE PYTHONIC WAY

- ✗ ~~`__str__` only~~ → ✓ `__repr__` **first,** `__str__` **if needed**
- ✗ ~~Writing all six comparisons~~ → ✓ `@total_ordering` **or** `@dataclass(order=True)`
- ✗ ~~`return False` for a foreign type~~ → ✓ `return NotImplemented`

DISPLAY

METHOD	TRIGGERED BY
`__repr__`	repr(x), REPL, containers
`__str__`	str(x), print(x)
`__format__`	f"{x:spec}"
`__bytes__`	bytes(x)

COMPARISON & HASHING

METHOD	OPERATOR
`__eq__`	==
`__ne__`	!= (auto from __eq__)
`__lt__`	<
`__le__`	<=
`__gt__`	>
`__ge__`	>=
`__hash__`	hash(), sets, dict keys

RULES

RULE	WHY
`a == b ⇒ hash(a) == hash(b)`	Sets and dicts rely on it
`Hash only immutable fields`	Hash must not change
`Return NotImplemented`	Not False, for unknown types
`Use total_ordering`	Write 2 methods, get 6

⚠ COMMON MISTAKES

- Defining `__eq__` only — instances become unhashable
- Hashing mutable fields — objects get lost inside sets
- Returning `False` for foreign types — blocks the reflected comparison
- `__repr__` returning non-str — `TypeError`

✓ TIPS & BEST PRACTICES

- Make `__repr__` look like the constructor call
- Compare via a tuple key to keep methods consistent
- Use `@dataclass(frozen=True, order=True)` for value objects
- Define `__str__` only if users need a friendlier form

QUICK SUMMARY

- `__repr__` / `__str__` — display
- `__eq__` / `__lt__` — comparisons
- `__hash__` — sets and dict keys
- `@total_ordering` — fill the rest

Dunder Methods — Operations

OPERATORS & CONTAINERS

Dunder methods also let your objects use **operators** (+, *, `abs()`) and behave like **containers** (`len()`, indexing, `in`, iteration) or **functions** (`obj()`). When `a + b` fails on `a`, Python tries the **reflected** method `b.__radd__(a)`. In-place methods like `__iadd__` power +=; without one, Python falls back to `__add__`.

NOTES

- Return a **new** object from arithmetic methods
- Return `NotImplemented` for unsupported operand types
- `__getitem__` alone makes objects iterable and sliceable
- `__call__` makes `obj()` work

UNDER THE HOOD

- `a + b` calls `a.__add__(b)`, then `b.__radd__(a)`
- Special methods are looked up on the type, not the instance
- `x += y` tries `__iadd__` before `__add__`

A VECTOR AND A CONTAINER

```
class Vec:
    def __init__(self, x, y): self.x, self.y = x, y
    def __repr__(self): return f"Vec({self.x}, {self.y})"
    def __add__(self, o): return Vec(self.x + o.x, self.y + o.y)
    def __mul__(self, k): return Vec(self.x * k, self.y * k)
    def __rmul__(self, k): return self * k      # handles 3 * v
    def __abs__(self): return (self.x**2 + self.y**2) ** 0.5
v = Vec(3, 4)
v + Vec(1, 1), v * 2, 2 * v   # (Vec(4, 5), Vec(6, 8), Vec(6, 8))
abs(v)                        # 5.0
class Playlist:
    def __init__(self, songs): self.songs = list(songs)
    def __len__(self): return len(self.songs)
    def __getitem__(self, i): return self.songs[i]
    def __contains__(self, s): return s in self.songs
p = Playlist(["A", "B", "C"])
len(p), p[1], p[-2:], "B" in p  # (3, 'B', ['B', 'C'], True)
```

CODE EXPLAINED

→ `__add__` — `v + w`

→ `__rmul__` — `2 * v` after `int` gives up

→ `__getitem__` — indexing, slicing, and iteration

→ `p[-2:]` — slices arrive as a `slice` object

THE PYTHONIC WAY

✗ ~~`v.add(w)`~~ → ✓ `v + w` **via** `__add__`

✗ ~~`obj.size()`~~ → ✓ `len(obj)` **via** `__len__`

✗ ~~`obj.has(x)`~~ → ✓ `x in obj` **via** `__contains__`

ARITHMETIC

METHOD	OPERATOR
`__add__ / __radd__`	+
`__sub__ / __rsub__`	-
`__mul__ / __rmul__`	*
`__truediv__`	/
`__floordiv__`	//
`__mod__`	%
`__pow__`	**
`__neg__ / __abs__`	-x / abs(x)

IN-PLACE & UNARY

METHOD	TRIGGERED BY
`__iadd__`	+=
`__isub__`	-=
`__imul__`	*=
`__bool__`	bool(x), if x
`__int__ / __float__`	int(x) / float(x)
`__round__`	round(x)

CONTAINER & CALLABLE

METHOD	TRIGGERED BY
`__len__`	len(x)
`__getitem__`	x[k]
`__setitem__`	x[k] = v
`__delitem__`	del x[k]
`__contains__`	k in x
`__iter__`	for i in x
`__call__`	x()

⚠ COMMON MISTAKES

- Mutating `self` in `__add__` — `a + b` shouldn't change `a`
- Forgetting `__rmul__` — `2 * v` raises `TypeError`
- `__len__` returning a non-int or negative — `TypeError` / `ValueError`
- Raising `TypeError` instead of returning `NotImplemented`

✓ TIPS & BEST PRACTICES

- Implement `__iter__` with `yield` for clean iteration
- Subclass `collections.abc.Sequence` to get `index`, `count`, `reversed` free
- Keep operators intuitive; don't overload `+` for unrelated actions
- Use `__call__` for configurable function objects

QUICK SUMMARY

- `__add__` & friends — operators
- `__r*__` — reflected operands
- `__len__` / `__getitem__` — container behavior
- `__call__` — callable objects

Context Managers

SETUP AND GUARANTEED CLEANUP

A **context manager** runs setup code when a `with` block starts and cleanup code when it ends, even if the block raises an exception. Files, locks, database transactions, and temporary changes all use this pattern. Write one as a class with `__enter__` / `__exit__`, or as a generator decorated with `@contextlib.contextmanager`.

NOTES

- `__enter__`'s return value is bound to the `as` name
- `__exit__` receives the exception info, or three `None`s
- Returning `True` from `__exit__` suppresses the exception
- Several managers: `with a() as x, b() as y:`

UNDER THE HOOD

- `with` calls `__enter__` and guarantees `__exit__`
- `@contextmanager` runs a generator: before `yield` = enter
- After `yield` = exit; exceptions are thrown in at `yield`

CLASS-BASED AND GENERATOR-BASED

```python
import os, time
from contextlib import contextmanager
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self                    # bound to `as t`
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False                   # don't suppress errors
with Timer() as t:
    sum(range(10**6))
t.elapsed                              # 0.012 (seconds)
@contextmanager
def cd(path):
    old = os.getcwd(); os.chdir(path)  # setup
    try: yield                         # the with-body runs here
    finally: os.chdir(old)             # cleanup
```

CODE EXPLAINED

- → `__enter__` — setup; returns the `as` value
- → `__exit__` — cleanup; runs on success or error
- → `@contextmanager` — code before `yield` is setup, after is cleanup
- → `try/finally` around `yield` — cleanup survives errors

PYTHON VERSIONS

- **3.7** — `asynccontextmanager` and `nullcontext`
- **3.10** — parenthesized multi-manager `with ( ... ):`
- **3.11** — `contextlib.chdir()`

THE PYTHONIC WAY

✗ ~~`try` / `finally` around setup + cleanup~~ → ✓ `with resource:`

✗ ~~A class with `__enter__` / `__exit__` for a tiny job~~ → ✓ `@contextmanager`

✗ ~~`try: ... except FileNotFoundError: pass`~~ → ✓ `with suppress(FileNotFoundError):`

PROTOCOL

STEP	METHOD
`Enter with`	`__enter__()`
`Bind as`	Its return value
`Leave normally`	`__exit__(None, None, None)`
`Leave by error`	`__exit__(type, exc, tb)`
`Suppress error`	`__exit__` returns True

CONTEXTLIB

TOOL	USE
`@contextmanager`	Generator → manager
`suppress(E)`	Ignore exception E
`closing(obj)`	Call .close() at end
`ExitStack()`	Dynamic number of managers
`nullcontext()`	Optional manager placeholder

COMMON MANAGERS

CODE	HANDLES
`open(path)`	File closing
`threading.Lock()`	Acquire / release
`sqlite3 connection`	Commit / rollback
`TemporaryDirectory()`	Delete when done

⚠ COMMON MISTAKES

- Opening files without `with` — they stay open if an error occurs
- No `try/finally` around `yield` — cleanup skipped on exceptions
- Returning `True` from `__exit__` by accident — errors vanish
- Using a file after its `with` block — it's closed

✓ TIPS & BEST PRACTICES

- Use `with` for anything that must be released
- Prefer `@contextmanager` for short setup/cleanup pairs
- Use `ExitStack` when the number of resources varies

QUICK SUMMARY

- `with` — setup + guaranteed cleanup
- `__enter__` / `__exit__` — the protocol
- `@contextmanager` — generator form
- `suppress` — ignore expected errors

Reading Files

OPENING AND READING

`open(path)` returns a file object; the default mode `"r"` reads text. Always open files in a `with` block so they close automatically. `read()` returns the whole file as one string, `readline()` one line, `readlines()` a list of lines, and iterating the file object streams one line at a time. Pass `encoding="utf-8"` explicitly, because the default depends on the operating system.

NOTES

- Lines keep their trailing `\n`
- Iterating `for line in f` uses little memory
- `"rb"` mode returns `bytes`, not `str`
- A file object is consumed as you read; `seek(0)` rewinds
- `Path.read_text()` reads a whole file in one call

READING PATTERNS

```
with open("notes.txt", encoding="utf-8") as f:
    text = f.read()                     # whole file as one str
with open("notes.txt", encoding="utf-8") as f:
    first = f.readline()                # 'line 1\n' (keeps \n)
    rest = f.readlines()                # ['line 2\n', 'line 3\n']
with open("big.log", encoding="utf-8") as f:
    for line in f:                      # streams line by line
        if "ERROR" in line:
            print(line.rstrip("\n"))
with open("notes.txt", encoding="utf-8") as f:
    lines = f.read().splitlines()       # no trailing \n
with open("photo.jpg", "rb") as f:
    header = f.read(4)                  # b'\xff\xd8\xff\xe0'
from pathlib import Path
Path("notes.txt").read_text(encoding="utf-8")  # one-liner
```

CODE EXPLAINED

- → `with` — closes the file even if an error occurs
- → `readline()` then `readlines()` — continues where it left off
- → `for line in f` — best for large files
- → `splitlines()` — lines without newline characters
- → `"rb"` — raw bytes for images and other binary data

PYTHON VERSIONS

- **3.10** — `EncodingWarning` flags a missing `encoding=`
- **3.15** — UTF-8 mode on by default (PEP 686)

THE PYTHONIC WAY

- ✗ ~~`f = open(p)` … `f.close()`~~ → ✓ `with open(p, encoding="utf-8") as f:`
- ✗ ~~`for line in f.readlines():`~~ → ✓ `for line in f:`
- ✗ ~~`open` + `read` for a small file~~ → ✓ `Path(p).read_text(encoding="utf-8")`

READ METHODS

METHOD	RETURNS
`f.read()`	Entire file
`f.read(n)`	Up to n chars/bytes
`f.readline()`	Next line
`f.readlines()`	List of lines
`for line in f`	Lines, lazily
`f.seek(0)`	Rewind to start

OPEN OPTIONS

ARGUMENT	MEANING
`"r"`	Read text (default)
`"rb"`	Read bytes
`"r+"`	Read and write
`encoding="utf-8"`	Text encoding
`errors="replace"`	Swap bad bytes for `&#65533;`

READ ERRORS

EXCEPTION	CAUSE
`FileNotFoundError`	Path doesn't exist
`PermissionError`	No read access
`IsADirectoryError`	Path is a folder
`UnicodeDecodeError`	Wrong encoding

⚠ COMMON MISTAKES

- Opening without `with` — the file may stay open after errors
- Omitting `encoding` — works on your machine, fails on another
- `read()` on a multi-GB file — loads it all into memory
- Comparing lines without stripping `\n` — `"yes\n" != "yes"`

✓ TIPS & BEST PRACTICES

- Stream big files with `for line in f`
- Use `Path(...).read_text()` for small files
- Pass `errors="replace"` when reading messy text
- Build paths relative to the script with `Path(__file__).parent`

QUICK SUMMARY

- `open(path, encoding=)` — open text
- `with` — auto-close
- `read()` / `readline()` / `readlines()`
- `for line in f` — stream lines

Writing & Appending Files

'w' · 'a' · 'x' · write() · writelines() · print(file=)

CREATING AND UPDATING FILES

Open a file with mode "w" to write (creating it or **erasing** existing content), "a" to append to the end, or "x" to create it only if it doesn't exist yet. `write()` takes a string and doesn't add newlines; `writelines()` writes many strings; `print(..., file=f)` formats and adds a newline for you. Use "b" modes for bytes.

NOTES

- `"w"` truncates the file immediately on open
- `write()` returns the number of characters written
- Data is buffered until the file is flushed or closed
- `"x"` raises `FileExistsError` instead of overwriting
- Write CSV files with `newline=""` (Sheet 72)

WRITING PATTERNS

```
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("Hello\n")                  # returns 6
    f.write(f"Total: {42}\n")           # write() needs a str
with open("out.txt", "a", encoding="utf-8") as f:
    f.write("appended line\n")          # added at the end
lines = ["a", "b", "c"]
with open("list.txt", "w", encoding="utf-8") as f:
    f.writelines(f"{x}\n" for x in lines)  # add \n yourself
with open("report.txt", "w", encoding="utf-8") as f:
    print("Name", "Score", sep="\t", file=f)  # print to a file
with open("new.txt", "x", encoding="utf-8") as f:
    f.write("fresh")                    # FileExistsError if present
with open("data.bin", "wb") as f:
    f.write(bytes([0, 255]))            # binary needs bytes
from pathlib import Path
Path("note.txt").write_text("hi\n", encoding="utf-8")
```

CODE EXPLAINED

- → `"w"` — start fresh; old content is gone
- → `"a"` — keep content, add to the end
- → `writelines()` — no separators added
- → `print(file=f)` — handy for formatted rows
- → `"wb"` — write `bytes` objects

PYTHON VERSIONS

- **3.3** — `"x"` exclusive-create mode

THE PYTHONIC WAY

✗ ~~`f.write(str(a) + "\n")`~~ → ✓ `print(a, file=f)`

✗ ~~Checking `exists()` before~~ `"w"` → ✓ `open(p, "x")`

✗ ~~Writing straight over the live file~~ → ✓ **temp file + `os.replace()`**

WRITE MODES

MODE	EFFECT
`"w"`	Create or truncate
`"a"`	Create or append
`"x"`	Create; fail if exists
`"w+"`	Truncate, read + write
`"a+"`	Append + read
`"wb" / "ab"`	Binary versions

WRITE METHODS

CALL	DOES
`f.write(s)`	Write one string
`f.writelines(it)`	Write many strings
`print(…, file=f)`	Formatted line
`f.flush()`	Push buffer to OS
`Path.write_text(s)`	Whole file at once

SAFE WRITES

TECHNIQUE	WHY
`Write temp, then os.replace()`	Atomic swap
`"x" mode`	Never overwrite
`os.fsync(f.fileno())`	Force to disk
`with block`	Flush + close

⚠ COMMON MISTAKES

- Opening an existing file with `"w"` — its content is erased instantly
- `f.write(42)` — `TypeError`; convert with `str()` or an f-string
- Expecting `writelines()` to add newlines — it doesn't
- Not closing the file — buffered data can be lost

✓ TIPS & BEST PRACTICES

- Write to a temp file and `os.replace()` it for crash safety
- Use `print(..., file=f)` for quick reports
- Use `"x"` mode for files that must never be overwritten
- `tempfile.NamedTemporaryFile` for scratch files

QUICK SUMMARY

- `"w"` — overwrite; `"a"` — append
- `"x"` — create only
- `write()` — str only, no newline
- `print(file=f)` — formatted output

File Paths & Operations

WORKING WITH PATHS

`pathlib.Path` represents file-system paths as objects. Join parts with the `/` operator, read pieces like `.name` and `.suffix`, and call methods to test, create, list, rename, and delete. It works the same on Windows, macOS, and Linux. The older `os.path` functions still work but operate on plain strings.

PATHLIB ESSENTIALS

```
from pathlib import Path
p = Path("data") / "2026" / "report.csv"   # join with /
p.name, p.stem, p.suffix           # ('report.csv', 'report', '.csv')
p.parent                           # Path('data/2026')
p.with_suffix(".json")             # Path('data/2026/report.json')
Path.cwd(), Path.home()            # current folder, home folder
p.exists(), p.is_file(), p.is_dir()
p.parent.mkdir(parents=True, exist_ok=True)  # create folders
p.resolve()                        # absolute path
list(Path(".").glob("*.py"))       # matches in this folder
list(Path(".").rglob("*.csv"))     # recursive search
p.stat().st_size                   # size in bytes
p.rename("old.csv")                # move / rename
Path("old.csv").unlink(missing_ok=True)  # delete file
import os
os.path.join("data", "x.txt")      # 'data/x.txt'  (string style)
```

NOTES

- Relative paths resolve against the current working directory
- `Path(__file__).parent` is the script's own folder
- `glob()` patterns: `*`, `?`, `**` for recursion
- Use `shutil` for copying and deleting whole trees
- Most functions that take paths accept `Path` objects

CODE EXPLAINED

- → `/` — builds paths with the right separator
- → `stem` / `suffix` — name without / only the extension
- → `mkdir(parents=True, exist_ok=True)` — like `mkdir -p`
- → `rglob()` — searches subfolders too
- → `unlink(missing_ok=True)` — no error if already gone

PYTHON VERSIONS

- **3.4** — `pathlib` added
- **3.12** — `Path.walk()`
- **3.14** — `Path.copy()` and `Path.move()`

THE PYTHONIC WAY

- ✗ ~~`os.path.join(base, "data", f)`~~ → ✓ `Path(base) / "data" / f`
- ✗ ~~`open("data.txt")` relative to cwd~~ → ✓ `Path(__file__).parent / "data.txt"`
- ✗ ~~`os.listdir()` + `endswith`~~ → ✓ `Path(d).glob("*.txt")`

PATH PARTS

ATTRIBUTE	FOR /A/B/C.TAR.GZ
`name`	c.tar.gz
`stem`	c.tar
`suffix`	.gz
`suffixes`	['.tar', '.gz']
`parent`	/a/b
`parts`	('/', 'a', 'b', 'c.tar.gz')

OPERATIONS

METHOD	DOES
`exists()`	Path exists?
`mkdir()`	Create folder
`iterdir()`	List folder
`glob(pat)`	Match files
`rename(t)`	Move
`unlink()`	Delete file
`rmdir()`	Delete empty folder

OS.PATH → PATHLIB

OS / SHUTIL	PATHLIB
`os.path.join(a, b)`	Path(a) / b
`os.path.exists(p)`	p.exists()
`os.path.basename(p)`	p.name
`os.listdir(d)`	d.iterdir()
`os.makedirs(d)`	d.mkdir(parents=True)
`shutil.copy(a, b)`	p.copy(b) (3.14+)

⚠ COMMON MISTAKES

- Building paths with `+ "/" +` — breaks on Windows
- Assuming relative paths start at the script — they start at the cwd
- `mkdir()` on an existing folder without `exist_ok=True` — `FileExistsError`
- `rmdir()` on a non-empty folder — use `shutil.rmtree()`

✓ TIPS & BEST PRACTICES

- Anchor data files with `Path(__file__).parent / "data"`
- Use `shutil.copy2`, `move`, and `rmtree` for bigger jobs
- `sorted(p.glob("*.txt"))` for predictable order
- Convert to a string with `str(p)` only when an API requires it

QUICK SUMMARY

- `Path(a) / b` — join
- `.name` `.stem` `.suffix` — parts
- `exists` `mkdir` `glob` — common ops
- `os.path` — older string API

JSON

JAVASCRIPT OBJECT NOTATION

The `json` module converts between Python objects and JSON text. `dumps()` and `loads()` work with strings; `dump()` and `load()` work with open files. Dicts become JSON objects, lists become arrays, and `None`, `True`, `False` become `null`, `true`, `false`. Types JSON lacks, like `datetime` and `set`, need a `default=` function.

NOTES

- The `s` in `dumps` / `loads` means "string"
- JSON object keys are always strings
- Tuples are written as arrays and load back as lists
- JSON requires double quotes
- Invalid input raises `json.JSONDecodeError`

ROUND-TRIP EXAMPLE

```
import json
data = {"name": "Ana", "age": 30, "tags": ["admin"], "active": True}
s = json.dumps(data)                # '{"name": "Ana", "age": 30, ...}'
json.dumps(data, indent=2)          # pretty-printed
json.dumps({"b": 1, "a": 2}, sort_keys=True)  # '{"a": 2, "b": 1}'
json.loads('{"x": [1, 2], "ok": null}')  # {'x': [1, 2], 'ok': None}
with open("user.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)    # write to a file
with open("user.json", encoding="utf-8") as f:
    loaded = json.load(f)           # read from a file
loaded == data                      # True
from datetime import date
json.dumps({"d": date(2026, 10, 1)}, default=str)  # '{"d": "2026-10-01"}'
json.dumps({"city": "Zürich"}, ensure_ascii=False) # keeps ü
json.loads("{'a': 1}")              # JSONDecodeError (single quotes)
```

CODE EXPLAINED

- → `indent=2` — readable output
- → `sort_keys=True` — stable output for diffs
- → `dump` / `load` — file versions
- → `default=str` — fallback for unknown types
- → `ensure_ascii=False` — write non-ASCII characters as-is

THE PYTHONIC WAY

- ✗ ~~`json.loads(f.read())`~~ → ✓ `json.load(f)`
- ✗ ~~`str(d)` to save a dict~~ → ✓ `json.dumps(d, indent=2)`
- ✗ ~~Manual dict building for a dataclass~~ → ✓ `json.dumps(asdict(obj))`

FUNCTIONS

FUNCTION	CONVERTS
`json.dumps(obj)`	Python → str
`json.loads(s)`	str → Python
`json.dump(obj, f)`	Python → file
`json.load(f)`	file → Python

TYPE MAPPING

PYTHON	JSON
`dict`	object
`list, tuple`	array
`str`	string
`int, float`	number
`True / False`	true / false
`None`	null

USEFUL OPTIONS

OPTION	EFFECT
`indent=2`	Pretty print
`sort_keys=True`	Ordered keys
`default=fn`	Handle other types
`ensure_ascii=False`	Keep Unicode
`separators=(",", ":")`	Compact output

⚠ COMMON MISTAKES

- Mixing up `dump` and `dumps` — one needs a file, one returns a string
- Expecting int keys back — `{1: "a"}` loads as `{"1": "a"}`
- Writing Python-style text (`'single quotes'`, `True`) — invalid JSON
- Dumping `datetime` or `set` — `TypeError: not JSON serializable`

✓ TIPS & BEST PRACTICES

- Validate and pretty-print from the shell: `python -m json.tool file.json`
- Catch `json.JSONDecodeError` when parsing outside input
- Convert sets with `default=list` or `sorted(s)` first
- Use `dataclasses.asdict()` to serialize dataclasses

QUICK SUMMARY

- `dumps` / `loads` — strings
- `dump` / `load` — files
- `indent=` — pretty output
- `default=` — custom types

CSV Files

COMMA-SEPARATED VALUES

The `csv` module reads and writes delimited text correctly, handling quotes, embedded commas, and line breaks inside fields. `csv.reader` yields each row as a list of strings and `csv.writer` writes lists. `DictReader` and `DictWriter` use the header row so you can work with column names. Always open CSV files with `newline=""`.

NOTES

- Every value read is a `str`; convert it yourself
- `newline=""` prevents blank lines on Windows
- Excel files saved as CSV may need `encoding="utf-8-sig"`
- `delimiter=";"` or `"\t"` for other separators
- For analysis, `pandas.read_csv()` is often easier

READ & WRITE CSV

```
import csv
rows = [["name", "score"], ["Ana", 91], ["Bo", 78]]
with open("scores.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows(rows)          # handles quoting
with open("scores.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)                  # ['name', 'score']
    for name, score in reader:
        print(name, int(score))            # values arrive as str
with open("scores.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["score"])   # Ana 91
people = [{"name": "Cy", "score": 85}]
with open("more.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["name", "score"])
    w.writeheader(); w.writerows(people)
```

CODE EXPLAINED

- → `writerows()` — write many rows at once
- → `next(reader)` — pull off the header row
- → `int(score)` — convert numeric columns
- → `DictReader` — each row is a dict keyed by header
- → `writeheader()` — write the field names first

THE PYTHONIC WAY

- ✗ ~~`line.split(",")`~~ → ✓ `csv.reader(f)`
- ✗ ~~`row[3]`~~ ~~magic indexes~~ → ✓ `csv.DictReader` → `row["email"]`
- ✗ ~~`open(p, "w")`~~ ~~for CSV~~ → ✓ `open(p, "w", newline="")`

READERS & WRITERS

OBJECT	USE
`csv.reader(f)`	Rows as lists
`csv.writer(f)`	Write lists
`csv.DictReader(f)`	Rows as dicts
`csv.DictWriter(f, fieldnames)`	Write dicts
`w.writerow(row)`	One row
`w.writeheader()`	Header (DictWriter)

OPTIONS

OPTION	EFFECT
`delimiter=";"`	Field separator
`quotechar='"'`	Quote character
`quoting=csv.QUOTE_ALL`	Quote every field
`skipinitialspace=True`	Ignore space after ,
`dialect="excel-tab"`	Tab-separated

GOTCHAS

ISSUE	FIX
Blank lines on Windows	newline=""
\ufeff in first header	encoding="utf-8-sig"
Numbers as text	Convert with int()/float()
Extra dict keys	extrasaction="ignore"

⚠ COMMON MISTAKES

- Leaving out `newline=""` — doubled line breaks on Windows
- Splitting lines with `.split(",")` — breaks on quoted commas
- Forgetting that every value is a string
- `DictWriter` given dicts with unexpected keys — `ValueError`

✓ TIPS & BEST PRACTICES

- Use `utf-8-sig` when the file will be opened in Excel
- Detect the format with `csv.Sniffer().sniff(sample)`
- Stream big files row by row; don't load them all
- Switch to `pandas` for filtering, grouping, and joins

QUICK SUMMARY

- `csv.reader` / `writer` — lists
- `DictReader` / `DictWriter` — dicts
- `newline=""` — always
- values — strings; convert yourself

Importing & Modules

USING CODE FROM OTHER FILES

Every `.py` file is a **module**. `import` loads a module once, caches it in `sys.modules`, and binds a name to it. `from module import name` pulls specific names in, and `as` sets an alias. Inside packages, **relative imports** with dots refer to sibling modules. The `if __name__ == "__main__":` guard runs code only when a file is executed directly.

NOTES

- A module runs top to bottom on first import only
- Python searches `sys.path` in order
- Relative imports work only inside packages
- `__all__` limits what `from m import *` exports
- `importlib.reload(m)` re-runs a module (REPL only)

UNDER THE HOOD

- Loaded modules are cached in `sys.modules`
- A second import is just a dict lookup
- Bytecode is cached as `__pycache__/mod.cpython-3XX.pyc`

IMPORT FORMS

```
import math                          # bind the module
math.sqrt(16)                        # 4.0
import numpy as np                   # module alias
from math import pi, floor           # specific names
floor(pi)                            # 3
from os.path import join as pjoin    # alias one name
from math import *                   # every public name (avoid)
import importlib
json_mod = importlib.import_module("json")   # by string
# inside mypkg/reader.py:
from . import utils                  # sibling module
from .utils import clean             # name from a sibling
from ..core import Engine            # parent package
if __name__ == "__main__":           # only when run directly
    main()
import sys
sys.path[:2]                         # first search locations
```

CODE EXPLAINED

- → `import x` — use as `x.name`
- → `from x import y` — use `y` directly
- → `.` / `..` — current / parent package
- → `__name__` — `"__main__"` when run as a script, else module name

PYTHON VERSIONS

- **3.3** — implicit namespace packages
- **3.7** — module-level `__getattr__` (PEP 562)

THE PYTHONIC WAY

- ✗ ~~`from module import *`~~ → ✓ `from module import name`
- ✗ ~~`a.b.c.f()` spelled out everywhere~~ → ✓ `from a.b.c import f`
- ✗ ~~`sys.path.append("..")`~~ → ✓ **install the package or use** `python -m`

IMPORT FORMS

SYNTAX	BINDS
`import m`	m
`import m as x`	x
`from m import a`	a
`from m import a as b`	b
`from . import m`	Sibling m
`from m import *`	All public names

SEARCH ORDER

STEP	LOCATION
1	sys.modules cache
2	Built-in modules
3	Script's folder
4	PYTHONPATH entries
5	Standard library
6	site-packages

MODULE ATTRIBUTES

ATTRIBUTE	HOLDS
`__name__`	Module name
`__file__`	Source path
`__doc__`	Docstring
`__all__`	Public names
`dir(m)`	All names

⚠ COMMON MISTAKES

- Naming your file `random.py` or `json.py` — shadows the real module
- Running a package file directly with relative imports — `ImportError`; use `python -m pkg.mod`
- Circular imports — two modules importing each other at top level
- `from x import *` — unclear origins and name clashes

✓ TIPS & BEST PRACTICES

- Order imports: standard library, third-party, local
- Run package code with `python -m package.module`
- Import heavy modules inside the function that needs them
- Let `ruff` or `isort` sort imports automatically

QUICK SUMMARY

- `import m` — whole module
- `from m import x` — one name
- `.` imports — inside packages
- `__main__` guard — script-only code

Creating Packages

ORGANIZING AND SHIPPING CODE

A **package** is a folder of modules, usually marked with an `__init__.py` file that runs when the package is imported. Modern projects describe themselves in `pyproject.toml` (name, version, dependencies, entry points) and keep code under `src/` to avoid accidental imports from the working folder. Build a wheel to share it, or install it in editable mode while developing.

NOTES

- `__init__.py` can be empty, or re-export the public API
- Folders without `__init__.py` become namespace packages
- `__main__.py` makes `python -m mypkg` work
- Editable installs pick up code changes immediately

__INIT__.PY AND PYPROJECT.TOML

```
# src/mypkg/__init__.py
"""mypkg: tools for tidy data."""
from .core import load, save        # re-export the public API
__all__ = ["load", "save"]
__version__ = "0.1.0"
# pyproject.toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
[project]
name = "mypkg"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]
[project.scripts]
mypkg = "mypkg.cli:main"            # command-line entry point
```

CODE EXPLAINED

- → re-export — users write `from mypkg import load`
- → `[build-system]` — which tool builds the package
- → `[project]` — metadata and dependencies
- → `[project.scripts]` — installs a `mypkg` command

PYTHON VERSIONS

- **3.3** — namespace packages need no `__init__.py`
- **3.11** — `tomllib` reads `pyproject.toml`

THE PYTHONIC WAY

- ✗ ~~`setup.py` for new projects~~ → ✓ `pyproject.toml`
- ✗ ~~Package code at the repo root~~ → ✓ `src/` **layout**
- ✗ ~~`from pkg.sub.mod import Thing` for users~~ → ✓ **re-export:** `from pkg import Thing`

SRC LAYOUT

PATH	CONTAINS
`pyproject.toml`	Project metadata
`README.md`	Description
`src/mypkg/__init__.py`	Package marker
`src/mypkg/core.py`	Code
`src/mypkg/__main__.py`	python -m entry
`tests/`	Test files

COMMANDS

COMMAND	DOES
`pip install -e .`	Editable install
`python -m build`	Make sdist + wheel
`twine upload dist/*`	Publish to PyPI
`uv build`	Fast build alternative
`python -m mypkg`	Run __main__.py

TERMS

TERM	MEANING
`module`	One .py file
`package`	Folder of modules
`namespace pkg`	No __init__.py (PEP 420)
`wheel (.whl)`	Built, installable archive
`sdist`	Source archive

⚠ COMMON MISTAKES

- Flat layout with tests importing local folders instead of the installed package
- Heavy work in `__init__.py` — slows every import
- Forgetting to bump `version` before publishing — upload rejected
- Listing dev tools (pytest, ruff) as runtime `dependencies`

✓ TIPS & BEST PRACTICES

- Use the `src/` layout for anything you'll distribute
- Put dev tools under `[project.optional-dependencies]` or dependency groups
- Keep the public API small and re-exported from `__init__.py`
- Test publishing on TestPyPI first

QUICK SUMMARY

- package — folder + `__init__.py`
- `pyproject.toml` — project config
- `src/` layout — safer imports
- `pip install -e .` — develop in place

pip & Package Management

INSTALLING THIRD-PARTY PACKAGES

`pip` installs packages from the Python Package Index (PyPI). Run it as `python -m pip` so it installs into the interpreter you actually use. Create a **virtual environment** per project so each project has its own isolated packages and versions. Record dependencies in `requirements.txt` for apps or `pyproject.toml` for packages.

NOTES

- `venv` ships with Python; no install needed
- Activate the venv before installing
- `pip freeze` lists exact installed versions
- Never use `sudo pip install`
- `uv` is a much faster pip/venv-compatible tool

PIP & VENV COMMANDS

```
python -m venv .venv                        # create a virtual env
source .venv/bin/activate                   # macOS / Linux
.venv\Scripts\activate                      # Windows
python -m pip install requests              # latest version
python -m pip install "requests>=2.31,<3"   # version range
python -m pip install -U requests           # upgrade
python -m pip uninstall requests
python -m pip list                          # installed packages
python -m pip list --outdated               # upgrades available
python -m pip show requests                 # version, location, deps
python -m pip freeze > requirements.txt     # pin exact versions
python -m pip install -r requirements.txt   # reproduce env
python -m pip install -e .                  # editable project install
deactivate                                  # leave the venv
```

CODE EXPLAINED

- → `python -m pip` — ties pip to this interpreter
- → `-U` — upgrade to the newest allowed version
- → `freeze` — exact pins for reproducible installs
- → `-r` — install everything listed in a file
- → `-e .` — install your own project in editable mode

PYTHON VERSIONS

- **3.4** — `ensurepip` bundles pip
- **pip 23** — refuses installs into PEP 668 system Pythons

THE PYTHONIC WAY

- ✗ ~~Unpinned~~ ~~`requirements.txt`~~ ~~for an app~~ → ✓ **pinned versions or a lockfile**
- ✗ ~~`pip install`~~ ~~in a global Python~~ → ✓ **venv per project**
- ✗ ~~`pip`~~ ~~for CLI tools~~ → ✓ `pipx` **or** `uv tool install`

VERSION SPECIFIERS

SPEC	MEANS
`==2.31.0`	Exactly
`>=2.31`	At least
`<3`	Below
`~=2.31`	≥2.31, <3.0
`!=2.30`	Not this one

DEPENDENCY FILES

FILE	BEST FOR
`requirements.txt`	App installs (pinned)
`pyproject.toml`	Package dependencies
`uv.lock / poetry.lock`	Exact lock files
`.python-version`	Interpreter version

TOOLS

TOOL	ROLE
`pip`	Install packages
`venv`	Isolated environments
`pipx`	Install CLI apps globally
`uv`	Fast all-in-one manager
`poetry`	Deps + packaging
`conda`	Non-Python deps too

⚠ COMMON MISTAKES

- `pip` pointing at a different Python — use `python -m pip`
- Installing without activating the venv — packages land globally
- `sudo pip install` — can break the system Python
- Committing `.venv/` to git — add it to `.gitignore`

✓ TIPS & BEST PRACTICES

- One virtual environment per project
- Pin versions for apps; use ranges for libraries
- Run `python -m pip check` to find broken dependencies
- Use `pipx install ruff` for tools you run everywhere

QUICK SUMMARY

- `python -m venv .venv` — isolate
- `pip install` / `uninstall` — manage
- `pip freeze` — pin versions
- `-r requirements.txt` — reproduce

Math & Random Modules

NUMBERS AND RANDOMNESS

`math` provides fast functions for real numbers: roots, rounding, logarithms, trigonometry, and combinatorics, plus constants like `pi`. `random` generates pseudo-random numbers and choices; call `random.seed()` to make results repeatable. For passwords and tokens, use the `secrets` module instead, because `random` is predictable.

NOTES

- `math` works on floats; use `cmath` for complex numbers
- Trig functions use radians
- `randint(a, b)` includes b; `randrange(a, b)` excludes b
- `shuffle()` changes the list in place
- `statistics` handles mean, median, stdev

MATH AND RANDOM

```
import math, random
math.sqrt(16), math.isqrt(17)           # (4.0, 4)
math.floor(-2.5), math.ceil(2.1)        # (-3, 3)
math.pi, math.e, math.tau, math.inf     # constants
math.log(100, 10), math.log2(8)         # (2.0, 3.0)
math.sin(math.pi / 2)                   # 1.0
math.degrees(math.pi)                   # 180.0
math.gcd(12, 18), math.lcm(4, 6)        # (6, 12)
math.factorial(5), math.comb(5, 2)      # (120, 10)
math.hypot(3, 4)                        # 5.0
random.seed(42)                         # repeatable sequence
random.random()                         # float in [0.0, 1.0)
random.randint(1, 6)                    # 1..6 inclusive
random.choice(["red", "green"])         # one item
random.sample(range(100), 3)            # 3 unique items
deck = list(range(10)); random.shuffle(deck)  # in place
```

CODE EXPLAINED

- → `isqrt` — exact integer square root
- → `log(x, base)` — any base; `log2`, `log10` are faster
- → `comb(n, k)` — n choose k
- → `seed(42)` — same numbers every run
- → `sample()` — no repeats; `choices()` allows repeats

PYTHON VERSIONS

- **3.6** — `secrets` module and `random.choices()`
- **3.8** — `math.prod()`, `math.isqrt()`, `math.dist()`

THE PYTHONIC WAY

✗ ~~`random` for tokens~~ → ✓ `secrets.token_hex()`

✗ ~~`reduce(operator.mul, xs)`~~ → ✓ `math.prod(xs)`

✗ ~~`xs[random.randint(0, len(xs) - 1)]`~~ → ✓ `random.choice(xs)`

MATH

FUNCTION	RESULT
`math.prod([2, 3, 4])`	24
`math.trunc(-2.7)`	-2
`math.fsum([.1] * 10)`	1.0
`math.isclose(a, b)`	Near-equal?
`math.dist(p, q)`	Euclidean distance
`math.perm(5, 2)`	20

RANDOM

FUNCTION	RETURNS
`random()`	[0, 1) float
`uniform(a, b)`	Float in range
`randrange(0, 10, 2)`	Even 0–8
`choices(xs, k=5)`	With repeats
`gauss(mu, sigma)`	Normal sample

SECURE & STATS

CALL	USE
`secrets.token_hex(16)`	Random token
`secrets.choice(xs)`	Secure pick
`secrets.randbelow(n)`	Secure int
`statistics.mean(xs)`	Average
`statistics.median(xs)`	Middle value

⚠ COMMON MISTAKES

- Using `random` for passwords or tokens — use `secrets`
- `math.sqrt(-1)` — `ValueError`; use `cmath.sqrt`
- `deck = random.shuffle(deck)` — `deck` becomes `None`
- Passing degrees to `math.sin()` — convert with `math.radians()`

✓ TIPS & BEST PRACTICES

- Seed random in tests for reproducible results
- `random.Random(seed)` gives an independent generator
- Use `math.isclose()` instead of `==` for floats
- Use `math.prod()` instead of `reduce(mul, ...)`

QUICK SUMMARY

- `math` — real-number functions
- `random` — pseudo-random values
- `seed()` — repeatable runs
- `secrets` — security-safe randomness

Dates, Times & Timezones

WORKING WITH TIME

`datetime` provides `date`, `time`, `datetime`, and `timedelta`. A datetime is **naive** (no time zone) or **aware** (has `tzinfo`). Do arithmetic with `timedelta`, format with `strftime()`, and parse with `strptime()` or `fromisoformat()`. `zoneinfo` (3.9+) supplies real time zones with daylight-saving rules.

NOTES

- Naive and aware datetimes can't be compared
- `weekday()` : Monday is 0; `isoweekday()` : Monday is 1
- `timedelta` has days, seconds, microseconds — no months
- `datetime.utcnow()` is deprecated (3.12); use `now(timezone.utc)`
- Windows may need `pip install tzdata` for `zoneinfo`

DATETIME ESSENTIALS

```
from datetime import datetime, date, timedelta, timezone
from zoneinfo import ZoneInfo
today = date.today()                      # date(2026, 10, 1)
now = datetime.now()                      # naive local time
utc = datetime.now(timezone.utc)          # aware, UTC
d = datetime(2026, 10, 1, 14, 30)
d.year, d.month, d.weekday()              # (2026, 10, 3)  Mon=0
d + timedelta(days=7, hours=2)            # 2026-10-08 16:30
(date(2026, 12, 25) - date(2026, 10, 1)).days   # 85
d.strftime("%Y-%m-%d %H:%M")              # '2026-10-01 14:30'
d.strftime("%a %b %d")                    # 'Thu Oct 01'
datetime.strptime("01/10/2026", "%d/%m/%Y")  # parse text
datetime.fromisoformat("2026-10-01T14:30")   # ISO 8601
d.isoformat()                             # '2026-10-01T14:30:00'
ny = d.replace(tzinfo=ZoneInfo("America/New_York"))
ny.astimezone(ZoneInfo("Asia/Tokyo"))  # 2026-10-02 03:30+09:00
```

CODE EXPLAINED

- → `timedelta` — add or subtract durations
- → date - date — a `timedelta` ; `.days` gives the count
- → `strftime` — datetime → text; `strptime` — text → datetime
- → `replace(tzinfo=...)` — attach a zone to a naive time
- → `astimezone()` — convert between zones

PYTHON VERSIONS

- **3.9** — `zoneinfo` time zones
- **3.11** — `fromisoformat()` parses most ISO 8601
- **3.12** — `datetime.utcnow()` deprecated

THE PYTHONIC WAY

- ✗ ~~`datetime.utcnow()`~~ → ✓ `datetime.now(timezone.utc)`
- ✗ ~~`strptime(s, "%Y-%m-%d")`~~ ~~for ISO~~ → ✓ `date.fromisoformat(s)`
- ✗ ~~`time.time()`~~ ~~to time code~~ → ✓ `time.perf_counter()`

FORMAT CODES

CODE	EXAMPLE
`%Y`	2026
`%m / %d`	10 / 01
`%H:%M:%S`	14:30:00
`%I %p`	02 PM
`%a / %A`	Thu / Thursday
`%b / %B`	Oct / October
`%j`	274 (day of year)

CLASSES

CLASS	HOLDS
`date`	Year, month, day
`time`	Hour … microsecond
`datetime`	Date + time
`timedelta`	Duration
`timezone.utc`	Fixed UTC offset
`ZoneInfo("Europe/Paris")`	Real zone + DST

GOOD PRACTICE

DO	WHY
`Store UTC`	No DST ambiguity
`Use ISO 8601`	Sortable, parseable
`Convert at display`	Local time for users
`time.monotonic()`	Measure durations

⚠ COMMON MISTAKES

- Comparing naive and aware datetimes — `TypeError`
- Adding months with `timedelta(days=30)` — wrong lengths
- Format string not matching input in `strptime` — `ValueError`
- Using wall-clock `datetime.now()` to time code — use `perf_counter()`

✓ TIPS & BEST PRACTICES

- Use `dateutil.relativedelta` for month/year arithmetic
- Keep datetimes aware and in UTC internally
- f-strings accept format codes: `f"{d:%Y-%m-%d}"`
- Parse ISO strings with `fromisoformat()` — fast and strict

QUICK SUMMARY

- `date` / `datetime` / `timedelta`
- `strftime` / `strptime` — format / parse
- naive vs aware — know which you have
- `ZoneInfo` — real time zones

OS & System Modules

TALKING TO THE OPERATING SYSTEM

`os` reads and changes the environment your program runs in: the working directory, environment variables, and processes. `sys` exposes the interpreter itself: command-line arguments, version, platform, and exit codes. `shutil` performs high-level file operations, and `subprocess.run()` launches other programs and captures their output.

NOTES

- `os.environ` changes affect this process and its children only
- `sys.argv[0]` is the script name
- Exit status 0 means success
- Pass commands to `subprocess` as a list, not a string
- For real CLIs, use `argparse` instead of raw `sys.argv`

OS, SYS, SHUTIL, SUBPROCESS

```
import os, sys, shutil, subprocess
os.getcwd()                             # current folder
os.environ.get("HOME")                  # env var or None
os.environ["APP_MODE"] = "dev"          # set for this process
os.listdir(".")                         # names in folder
os.cpu_count()                          # e.g. 8
sys.argv                                # ['script.py', 'arg1', ...]
sys.version_info >= (3, 10)             # True
sys.platform                            # 'linux', 'darwin', 'win32'
shutil.copy2("a.txt", "b.txt")          # copy with metadata
shutil.rmtree("build")                  # delete a folder tree
shutil.which("git")                     # '/usr/bin/git' or None
r = subprocess.run(["git", "status"], capture_output=True,
                   text=True, check=True)
r.returncode, r.stdout[:15]             # (0, 'On branch main\n')
sys.exit(1)                             # exit with status 1
```

CODE EXPLAINED

- → `environ.get()` — safe; `environ[k]` raises `KeyError`
- → `sys.platform` — branch on OS
- → `shutil.which()` — find a program on PATH
- → `capture_output` + `text` — get stdout as a str
- → `check=True` — raise if the command fails

PYTHON VERSIONS

- **3.5** — `subprocess.run()`
- **3.7** — `capture_output=True` and `text=True`

THE PYTHONIC WAY

- ✗ ~~`os.system("ls " + d)`~~ → ✓ `subprocess.run(["ls", d], check=True)`
- ✗ ~~`os.environ["PORT"]`~~ ~~for optional~~ → ✓ `os.getenv("PORT", "8000")`
- ✗ ~~`os.path`~~ ~~string juggling~~ → ✓ `pathlib.Path`

OS

CALL	DOES
`os.getcwd()`	Working directory
`os.chdir(p)`	Change directory
`os.environ`	Env variables
`os.getpid()`	Process ID
`os.walk(top)`	Walk a folder tree
`os.replace(a, b)`	Atomic rename

SYS

NAME	HOLDS
`sys.argv`	Command-line args
`sys.exit(code)`	Exit program
`sys.path`	Import search path
`sys.stdin / stdout / stderr`	Standard streams
`sys.executable`	Python binary path

SUBPROCESS.RUN()

ARGUMENT	EFFECT
`capture_output=True`	Collect stdout/stderr
`text=True`	str, not bytes
`check=True`	Raise on failure
`cwd=path`	Run in folder
`timeout=30`	Kill after 30 s
`env={...}`	Custom environment

⚠ COMMON MISTAKES

- `shell=True` with user input — command injection risk
- `os.environ["KEY"]` for optional settings — `KeyError` when unset
- Calling `sys.exit()` inside library code — kills the caller's program
- Relying on the working directory — scripts break when run elsewhere

✓ TIPS & BEST PRACTICES

- Use `argparse` for flags and help text
- Use `pathlib` instead of `os.path` for paths
- Set `timeout=` on subprocess calls that might hang
- Read config from env vars with defaults: `os.getenv("PORT", "8000")`

QUICK SUMMARY

- `os` — environment and files
- `sys` — interpreter and argv
- `shutil` — copy, move, delete trees
- `subprocess.run()` — run commands

functools Module

FUNCTION TOOLS

`functools` contains higher-order tools for working with functions. `partial()` pre-fills arguments, `reduce()` folds a sequence into one value, `cache` and `lru_cache` memoize results, `wraps` preserves metadata in decorators, `singledispatch` picks an implementation by argument type, and `cached_property` computes an attribute once.

NOTES

- Cached functions need hashable arguments
- `cache` is `lru_cache(maxsize=None)` (3.9+)
- Caching methods keeps `self` alive; be careful
- `total_ordering` is covered on Sheet 65
- `wraps` is covered on Sheet 44

FUNCTOOLS IN ACTION

```
from functools import partial, reduce, cache, lru_cache, singledispatch
int_from_bin = partial(int, base=2)
int_from_bin("1010")                    # 10
reduce(lambda a, b: a * b, [1, 2, 3, 4], 1)   # 24
@cache                                  # unbounded memo
def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
fib(80)                                 # 23416728348467685, instantly
@lru_cache(maxsize=128)                 # keeps 128 recent results
def lookup(key): ...
lookup.cache_info()                     # CacheInfo(hits=..., misses=...)
lookup.cache_clear()                    # empty the cache
@singledispatch
def show(x): return f"obj {x}"
@show.register
def _(x: int): return f"int {x}"
show(5), show("a")                      # ('int 5', 'obj a')
```

CODE EXPLAINED

- → `partial(int, base=2)` — a new function with base fixed
- → `reduce(f, xs, initial)` — initial guards empty input
- → `@cache` — recursion becomes linear time
- → `cache_info()` — check hit rates
- → `@show.register` — chosen by the type hint

PYTHON VERSIONS

- **3.8** — `cached_property` and `singledispatchmethod`
- **3.9** — `@cache`

THE PYTHONIC WAY

- ✗ ~~Memo `dict` + manual lookup~~ → ✓ `@cache`
- ✗ ~~`lambda x: f(x, 10)`~~ → ✓ `partial(f, b=10)`
- ✗ ~~`if isinstance(...)` dispatch chain~~ → ✓ `@singledispatch`

FUNCTIONS

TOOL	DOES
`partial(f, *a, **kw)`	Pre-fill args
`reduce(f, xs, init)`	Fold to one value
`cache`	Unbounded memo
`lru_cache(n)`	Bounded memo
`wraps(f)`	Copy metadata
`singledispatch`	Dispatch on type
`cached_property`	Compute once per object

CACHE API

CALL	DOES
`f.cache_info()`	Hits, misses, size
`f.cache_clear()`	Reset
`typed=True`	1 and 1.0 cached apart
`maxsize=None`	No limit

PARTIAL VS LAMBDA

PARTIAL	LAMBDA
`partial(f, x=1)`	lambda *a: f(*a, x=1)
`Inspectable .args`	Opaque
`Picklable`	Not picklable
`Fixed args only`	Any expression

⚠ COMMON MISTAKES

- Caching a function that takes a list — `TypeError: unhashable`
- Caching functions with side effects or time-dependent results
- `@lru_cache` on methods — instances never get garbage-collected
- `reduce()` on an empty sequence without `initial` — `TypeError`

✓ TIPS & BEST PRACTICES

- Use `@cache` for pure recursive functions
- Use `partial` for callbacks that need fixed arguments
- Prefer `sum`, `max`, `math.prod` over `reduce`
- Use `cached_property` for expensive per-instance values

QUICK SUMMARY

- `partial` — pre-filled functions
- `cache` / `lru_cache` — memoize
- `reduce` — fold
- `singledispatch` — type-based dispatch

unittest Testing Framework

BUILT-IN TEST FRAMEWORK

`unittest` ships with Python and organizes tests as methods on classes that subclass `unittest.TestCase`. Each method whose name starts with `test` is one test. `setUp()` runs before every test and `tearDown()` after it. Assertion methods report clear failure messages, and `python -m unittest discover` finds and runs every test file.

NOTES

- Test files are named `test_*.py` for discovery
- Each test gets a fresh `setUp()` state
- Tests run in alphabetical order; don't depend on it
- `unittest.mock` replaces dependencies with fakes
- pytest can run unittest-style tests unchanged

A TESTCASE

```
import unittest
from shop import Cart                    # code under test
class TestCart(unittest.TestCase):
    def setUp(self):                     # before each test
        self.cart = Cart()
    def test_starts_empty(self):
        self.assertEqual(len(self.cart), 0)
    def test_add_item(self):
        self.cart.add("pen", 1.50)
        self.assertAlmostEqual(self.cart.total(), 1.50)
        self.assertIn("pen", self.cart)
    def test_negative_price(self):
        with self.assertRaises(ValueError):
            self.cart.add("pen", -1)
    @unittest.skip("not ready")
    def test_discounts(self): ...
if __name__ == "__main__":
    unittest.main()                      # python test_cart.py
```

CODE EXPLAINED

- → `TestCase` — base class for a group of tests
- → `setUp` — build fresh fixtures for every test
- → `assertAlmostEqual` — safe float comparison
- → `assertRaises` as a context manager — expects an error
- → `@unittest.skip` — temporarily disable a test

PYTHON VERSIONS

- **3.8** — `IsolatedAsyncioTestCase`
- **3.11** — `enterContext()` and `enterClassContext()`

THE PYTHONIC WAY

✗ ~~`assertTrue(a == b)`~~ → ✓ `assertEqual(a, b)`

✗ ~~`try` / ~~`except`~~ / ~~`fail()`~~ → ✓ `with self.assertRaises(ValueError):`

✗ ~~Copy-paste test per input~~ → ✓ `with self.subTest(n=n):`

ASSERTIONS

METHOD	CHECKS
`assertEqual(a, b)`	a == b
`assertTrue(x)`	bool(x) is True
`assertIsNone(x)`	x is None
`assertIn(a, b)`	a in b
`assertRaises(E)`	E is raised
`assertAlmostEqual(a, b)`	Equal to 7 places
`assertCountEqual(a, b)`	Same items, any order

SETUP & TEARDOWN

METHOD	RUNS
`setUp()`	Before each test
`tearDown()`	After each test
`setUpClass()`	Once per class
`tearDownClass()`	Once per class (end)
`addCleanup(f)`	After test, even on failure

RUNNING TESTS

COMMAND	RUNS
`python -m unittest`	Discover in cwd
`… discover -s tests`	From tests/
`… -v`	Verbose output
`… -k add`	Names matching "add"
`… test_cart.TestCart`	One class

⚠ COMMON MISTAKES

- Naming a method `check_add` — not collected; must start with `test`
- Tests relying on each other's state or order
- `assertTrue(a == b)` — failure message hides both values; use `assertEqual`
- Comparing floats with `assertEqual`

✓ TIPS & BEST PRACTICES

- Use `self.subTest(i=i)` to loop over cases without stopping at the first failure
- Patch dependencies with `unittest.mock.patch("module.name")`
- Keep each test focused on one behavior
- Run single tests by dotted name while debugging

QUICK SUMMARY

- `TestCase` — test container
- `test_*` methods — tests
- `setUp` / `tearDown` — fixtures
- `assert*` — checks with messages

pytest Testing Framework

THE POPULAR TEST RUNNER

`pytest` is a third-party test framework (`pip install pytest`) that uses plain functions and plain `assert` statements; it rewrites asserts to show detailed failure output. **Fixtures** supply setup by naming them as test parameters. `@pytest.mark.parametrize` runs one test over many inputs, and `pytest.raises` checks for exceptions.

NOTES

- Collects `test_*.py` files and `test_*` functions
- Shared fixtures live in `conftest.py`
- Fixtures can `yield` to run teardown afterward
- Built-in fixtures: `tmp_path`, `capsys`, `monkeypatch`
- Configure in `pyproject.toml` under `[tool.pytest.ini_options]`

FIXTURES, RAISES, PARAMETRIZE

```
# test_cart.py   →  run with: pytest -v
import pytest
from shop import Cart
@pytest.fixture
def cart():                              # fresh object per test
    return Cart()
def test_add_item(cart):                 # fixture injected by name
    cart.add("pen", 1.50)
    assert cart.total() == pytest.approx(1.50)
    assert "pen" in cart
def test_negative_price(cart):
    with pytest.raises(ValueError, match="price"):
        cart.add("pen", -1)
@pytest.mark.parametrize("qty, expected", [(1, 1.5), (3, 4.5), (0, 0)])
def test_quantities(cart, qty, expected):
    cart.add("pen", 1.50, qty=qty)
    assert cart.total() == pytest.approx(expected)
```

CODE EXPLAINED

- → `@pytest.fixture` — reusable setup
- → `cart` parameter — pytest passes the fixture's return value
- → `pytest.approx` — tolerant float comparison
- → `match=` — regex checked against the error message
- → `parametrize` — three tests from one function

THE PYTHONIC WAY

- ✗ ~~`self.assertEqual(a, b)`~~ → ✓ **plain** `assert a == b`
- ✗ ~~Copy-paste tests per input~~ → ✓ `@pytest.mark.parametrize`
- ✗ ~~Setup code in every test~~ → ✓ **a** `@pytest.fixture`

COMMAND LINE

COMMAND	DOES
`pytest`	Run all tests
`pytest -v`	Verbose
`pytest -k add`	Name filter
`pytest -x`	Stop at first failure
`pytest --lf`	Rerun last failures
`pytest -s`	Show print output

FIXTURE OPTIONS

FEATURE	USE
`scope="module"`	Share across a file
`scope="session"`	Share across the run
`yield value`	Teardown after yield
`autouse=True`	Apply to every test
`tmp_path`	Temporary folder
`monkeypatch`	Patch attrs / env vars

MARKERS

MARKER	EFFECT
`@pytest.mark.skip`	Skip test
`@pytest.mark.skipif(c)`	Skip if c
`@pytest.mark.xfail`	Expected to fail
`@pytest.mark.parametrize`	Many inputs
`@pytest.mark.slow`	Custom tag (-m slow)

⚠ COMMON MISTAKES

- File or function names without the `test_` prefix — silently skipped
- Module-scoped fixtures that tests mutate — state leaks between tests
- Comparing floats with `==` — use `pytest.approx`
- `pytest.raises` without `match=` — passes on the wrong error message

✓ TIPS & BEST PRACTICES

- Put shared fixtures in `conftest.py`
- Use `tmp_path` instead of writing to real folders
- Add `pytest-cov` and run `pytest --cov=mypkg`
- Register custom markers to avoid warnings

QUICK SUMMARY

- plain `assert` — detailed failures
- fixtures — setup by parameter name
- `parametrize` — many cases, one test
- `pytest.raises` — expect errors

Logging

RECORDING WHAT HAPPENS

The `logging` module records messages with a **level**, a timestamp, and the module they came from, and sends them to handlers such as the console or a file. Unlike `print()`, you can turn detail up or down without editing code. Create one logger per module with `getLogger(__name__)` and configure output once at program start.

NOTES

- Default level is WARNING; DEBUG and INFO are hidden
- `basicConfig()` only works the first time (unless `force=True`)
- Pass values as arguments: `log.info("x=%s", x)`
- Libraries should log but never call `basicConfig()`

CONFIGURE AND LOG

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(name)s: %(message)s",
    datefmt="%H:%M:%S",
)
log = logging.getLogger(__name__)       # one logger per module
log.debug("hidden")                     # below INFO: not shown
log.info("server started on %s", 8080)
# 14:30:05 INFO     __main__: server started on 8080
log.warning("disk at %d%%", 91)
try:
    1 / 0
except ZeroDivisionError:
    log.exception("math failed")        # ERROR + traceback
fh = logging.FileHandler("app.log", encoding="utf-8")
fh.setLevel(logging.WARNING)
log.addHandler(fh)                      # warnings also go to a file
```

CODE EXPLAINED

- → `basicConfig` — sets the root level and format
- → `getLogger(__name__)` — logger named after the module
- → `%s` args — formatted only if the message is emitted
- → `log.exception()` — call inside `except`
- → handler level — filters what reaches that destination

PYTHON VERSIONS

- **3.2** — `{}` / `$` format styles and `dictConfig()`
- **3.8** — `basicConfig(force=True)`

THE PYTHONIC WAY

✗ ~~`print("DEBUG:", x)`~~ → ✓ `log.debug("x=%s", x)`

✗ ~~`log.info(f"user {u}")`~~ → ✓ `log.info("user %s", u)`

✗ ~~`logging.info(...)`~~ ~~in modules~~ → ✓ `log = logging.getLogger(__name__)`

LEVELS

LEVEL	VALUE	USE
`DEBUG`	10	Diagnostic detail
`INFO`	20	Normal events
`WARNING`	30	Unexpected, still working
`ERROR`	40	Operation failed
`CRITICAL`	50	Program can't continue

FORMAT FIELDS

FIELD	SHOWS
`%(asctime)s`	Timestamp
`%(levelname)s`	Level name
`%(name)s`	Logger name
`%(message)s`	Message
`%(funcName)s`	Function
`%(lineno)d`	Line number

COMPONENTS

PART	ROLE
`Logger`	Entry point you call
`Handler`	Destination
`Formatter`	Line layout
`Filter`	Fine-grained selection
`RotatingFileHandler`	Size-capped log files

⚠ COMMON MISTAKES

- Using `print()` for diagnostics — can't be filtered or redirected
- `log.info(f"x={x}")` — formats even when the level is off
- Calling `basicConfig()` after something else configured logging — no effect
- `log.error(e)` in `except` — loses the traceback; use `log.exception()`

✓ TIPS & BEST PRACTICES

- Configure once in `main()`; just call `getLogger(__name__)` elsewhere
- Use `RotatingFileHandler` for long-running services
- Use `logging.config.dictConfig()` for larger setups
- Quiet chatty libraries by raising their logger level (e.g. `urllib3`)

QUICK SUMMARY

- `getLogger(__name__)` — per-module logger
- levels — DEBUG → CRITICAL
- `basicConfig()` — quick setup
- `log.exception()` — error + traceback

Regular Expressions

PATTERN MATCHING ON TEXT

The `re` module finds, extracts, splits, and replaces text using **regular expression** patterns. Write patterns as **raw strings** (`r"..."`) so backslashes reach the regex engine untouched. `search()` finds the first match anywhere, `match()` only at the start, and `fullmatch()` requires the whole string. Parentheses create **groups** you can extract from the match object.

NOTES

- Failed searches return `None`; check before `.group()`
- `findall()` returns groups if the pattern has them
- Quantifiers are greedy; add `?` to make them lazy
- Flags: `re.I` ignore case, `re.M` multiline, `re.S` dot-all
- `re.compile()` helps when reusing a pattern

RE FUNCTIONS

```
import re
text = "Order #123 shipped 2026-10-01; order #456 pending"
re.search(r"#(\d+)", text).group(1)        # '123'  first match
re.match(r"Order", text)                   # matches only at start
re.fullmatch(r"\d{4}", "2026")             # whole string must match
re.findall(r"#(\d+)", text)                # ['123', '456']
re.sub(r"\d", "X", "a1b22")                # 'aXbXX'
re.split(r"[;,]\s*", "a; b,c")             # ['a', 'b', 'c']
m = re.search(r"(?P<y>\d{4})-(?P<m>\d\d)-(?P<d>\d\d)", text)
m.group("y"), m.groups()                   # ('2026', ('2026', '10', '01'))
m.span()                                   # (19, 29)
re.findall(r"order", text, flags=re.I)     # ['Order', 'order']
pat = re.compile(r"\b[A-Z][a-z]+\b")
pat.findall("Ana met Bo in Lima")          # ['Ana', 'Bo', 'Lima']
```

CODE EXPLAINED

- → `(\d+)` — capture one or more digits
- → `(?P<y>...)` — named group, read with `group("y")`
- → `span()` — start and end positions
- → `sub()` — replacement can use `\1` or a function
- → `\b` — word boundary

PYTHON VERSIONS

- **3.6** — `m["name"]` indexing on match objects
- **3.11** — atomic groups `(?>...)` and possessive `*+`

THE PYTHONIC WAY

- ✗ ~~`"\\d+"`~~ → ✓ `r"\d+"`
- ✗ ~~`re.search("abc", s)` for plain text~~ → ✓ `"abc" in s`
- ✗ ~~`m.group(1)`~~ → ✓ **named groups:** `m["year"]`

PATTERN SYNTAX

PATTERN	MATCHES
`.`	Any char except newline
`\d` / `\D`	Digit / non-digit
`\w` / `\W`	Word char / non-word
`\s` / `\S`	Whitespace / non-space
`\b`	Word boundary
`^` / `$`	Start / end
`[abc]` / `[^abc]`	Set / negated set

QUANTIFIERS & GROUPS

PATTERN	MEANING
`*` / `+` / `?`	0+, 1+, 0 or 1
`{3}` / `{2,4}`	Exactly 3 / 2 to 4
`*?` / `+?`	Lazy versions
`(...)`	Capture group
`(?:...)`	Non-capturing group
`(?P<n>...)`	Named group
`a|b`	a or b

FUNCTIONS

FUNCTION	RETURNS
`search(p, s)`	First match or None
`match(p, s)`	Match at start or None
`fullmatch(p, s)`	Whole-string match
`findall(p, s)`	List of strings
`finditer(p, s)`	Iterator of matches
`sub(p, r, s)`	New string
`split(p, s)`	List of parts

⚠ COMMON MISTAKES

- Plain strings like `"\d"` — escapes get mangled; use `r"\d"`
- Using `match()` to search the middle of a string — use `search()`
- Greedy `.*` grabbing too much — use `.*?`
- Parsing HTML or JSON with regex — use a real parser

✓ TIPS & BEST PRACTICES

- Test patterns interactively (e.g. regex101.com, Python flavor)
- Use `re.VERBOSE` to comment complex patterns
- Prefer string methods (`in`, `startswith`) for simple checks
- Use `re.escape(user_text)` before inserting text into a pattern

QUICK SUMMARY

- `r"..."` — raw-string patterns
- `search` / `match` / `fullmatch`
- `findall` / `sub` / `split`
- groups — `( )` and `(?P<name>)`

Async/await & Coroutines

CONCURRENT I/O WITH ASYNCIO

`async def` defines a **coroutine**; calling it creates a coroutine object that does nothing until awaited. `await` pauses the coroutine until a result is ready, letting the **event loop** run other tasks meanwhile. That makes asyncio ideal for many simultaneous network or disk waits. `asyncio.run()` starts the loop, and `gather()`, `create_task()`, and `TaskGroup` run coroutines concurrently.

NOTES

- Only one coroutine runs at a time (single thread)
- `await` only works inside `async def`
- Blocking calls like `time.sleep()` freeze every task
- `asyncio.TimeoutError` is `TimeoutError` (3.11+)
- Use async libraries: `httpx`, `aiohttp`, `aiofiles`

UNDER THE HOOD

- Coroutines are suspendable frames, like generators
- The event loop runs a task until it hits `await`
- Waiting sockets are watched with epoll / kqueue

COROUTINES AND TASKS

```
import asyncio
async def fetch(name, delay):
    await asyncio.sleep(delay)          # yields to the event loop
    return f"{name} done"
async def main():
    results = await asyncio.gather(     # concurrent: ~2 s total
        fetch("b", 2), fetch("c", 1))
    print(results)                      # ['b done', 'c done']
    task = asyncio.create_task(fetch("d", 1))  # starts now
    async with asyncio.TaskGroup() as tg:      # 3.11+
        t1 = tg.create_task(fetch("e", 1))
        t2 = tg.create_task(fetch("f", 1))
    print(t1.result(), await task)      # e done d done
    try:
        await asyncio.wait_for(fetch("g", 5), timeout=1)
    except TimeoutError:
        print("too slow")
asyncio.run(main())                     # start the event loop
```

CODE EXPLAINED

- → `await asyncio.sleep()` — a non-blocking pause
- → `gather()` — run several and collect results in order
- → `create_task()` — schedule now, await later
- → `TaskGroup` — waits for all; cancels the rest if one fails
- → `wait_for(..., timeout=)` — give up after a limit

PYTHON VERSIONS

- **3.7** — `asyncio.run()` and `create_task()`
- **3.11** — `TaskGroup` and `asyncio.timeout()`

THE PYTHONIC WAY

- ✗ ~~`loop = get_event_loop()` + `run_until_complete`~~ → ✓ `asyncio.run(main())`
- ✗ ~~`await a(); await b()` when independent~~ → ✓ `TaskGroup` or `gather(a(), b())`
- ✗ ~~`time.sleep(1)` in a coroutine~~ → ✓ `await asyncio.sleep(1)`

CORE API

CALL	DOES
`asyncio.run(main())`	Start loop, run main
`await coro`	Wait for a result
`create_task(coro)`	Run in background
`gather(*coros)`	Run all, list results
`TaskGroup()`	Structured tasks (3.11+)
`asyncio.timeout(s)`	Timeout block (3.11+)
`to_thread(f, *a)`	Run blocking code in thread

WHICH MODEL?

WORKLOAD	USE
Many network calls	asyncio
Blocking I/O library	Threads
Heavy CPU math	Processes
Simple scripts	Plain sync code

ASYNC SYNTAX

SYNTAX	MEANING
`async def`	Coroutine function
`await x`	Pause until x is done
`async with`	Async context manager
`async for`	Async iteration
`asyncio.Queue`	Producer/consumer

⚠ COMMON MISTAKES

- Calling `fetch()` without `await` — "coroutine was never awaited"
- `time.sleep()` or `requests.get()` inside async code — blocks everything
- `asyncio.run()` inside an already running loop (e.g. Jupyter) — use `await`
- Expecting asyncio to speed up CPU-bound work

✓ TIPS & BEST PRACTICES

- Prefer `TaskGroup` over bare `gather()` for error handling
- Wrap unavoidable blocking calls with `asyncio.to_thread()`
- Limit concurrency with `asyncio.Semaphore(10)`
- Keep `asyncio.run()` as the single entry point

QUICK SUMMARY

- `async def` — coroutine
- `await` — pause without blocking
- `gather` / `TaskGroup` — concurrency
- `asyncio.run()` — entry point

Threading & Concurrency

RUNNING CODE IN THREADS

A **thread** runs a function concurrently inside the same process and shares its memory. In standard CPython, the **Global Interpreter Lock (GIL)** lets only one thread execute Python bytecode at a time, so threads help with **I/O-bound** waits (network, disk) but not CPU-heavy work. Protect shared data with a `Lock`, and prefer `ThreadPoolExecutor` for simple parallel jobs.

NOTES

- The GIL is released while threads wait on I/O
- `counter += 1` is not atomic across threads
- `join()` waits for a thread to finish
- Free-threaded builds (3.13+) can run without the GIL

UNDER THE HOOD

- The GIL lets one thread run bytecode at a time
- Threads switch every 5 ms: `sys.getswitchinterval()`
- Blocking I/O and many C extensions release the GIL

THREADS, LOCKS, EXECUTORS

```
import threading, time
from concurrent.futures import ThreadPoolExecutor
def download(url):
    time.sleep(1)                       # stands in for network I/O
    return f"got {url}"
counter, lock = 0, threading.Lock()
def bump():
    global counter
    for _ in range(100_000):
        with lock:                      # one thread at a time
            counter += 1
threads = [threading.Thread(target=bump) for _ in range(4)]
for th in threads: th.start()
for th in threads: th.join()
counter                                 # 400000, exact
with ThreadPoolExecutor(max_workers=5) as pool:
    results = list(pool.map(download, ["a", "b", "c"]))  # ~1 s
```

CODE EXPLAINED

- → `Thread(target=bump)` — function to run in the thread
- → `with lock:` — acquire and always release
- → `pool.map()` — parallel map that returns results in order

PYTHON VERSIONS

- **3.2** — `concurrent.futures` executors
- **3.13** — experimental free-threaded (no-GIL) build
- **3.14** — free-threaded build officially supported (PEP 779)

THE PYTHONIC WAY

- ✗ ~~Creating and joining~~ ~~`Thread`~~ ~~objects by hand~~ → ✓ `ThreadPoolExecutor().map()`
- ✗ ~~`lock.acquire()`~~ … ~~`lock.release()`~~ → ✓ `with lock:`
- ✗ ~~Shared list for results~~ → ✓ `queue.Queue` **or returned futures**

THREADING API

OBJECT	USE
`Thread(target, args)`	New thread
`t.start() / t.join()`	Run / wait
`Lock()`	Mutual exclusion
`RLock()`	Re-entrant lock
`Event()`	Signal between threads
`queue.Queue()`	Thread-safe hand-off

EXECUTOR API

CALL	DOES
`pool.submit(f, x)`	Returns a Future
`fut.result()`	Wait for value
`pool.map(f, xs)`	Ordered results
`as_completed(futs)`	Results as they finish
`max_workers=n`	Thread count

PICK A TOOL

JOB	USE
`Waiting on network/disk`	Threads or asyncio
`CPU-heavy math`	Processes (Sheet 86)
`Producer/consumer`	Thread + Queue
`Background timer`	threading.Timer

⚠ COMMON MISTAKES

- Updating shared data without a lock — lost updates
- Forgetting `join()` — the program moves on before results exist
- Expecting threads to speed up CPU-bound code under the GIL
- Calling `t.run()` instead of `t.start()` — runs in the current thread

✓ TIPS & BEST PRACTICES

- Use `ThreadPoolExecutor` instead of managing threads by hand
- Pass data with `queue.Queue` rather than shared variables
- Keep locked sections short
- Give threads names for clearer logs: `Thread(name="worker-1")`

QUICK SUMMARY

- `Thread` — concurrent function
- `Lock` — protect shared state
- `ThreadPoolExecutor` — easy pools
- GIL — threads help I/O, not CPU

Multiprocessing

TRUE PARALLELISM WITH PROCESSES

`multiprocessing` runs code in separate **processes**, each with its own interpreter and memory, so CPU-heavy work can use every core despite the GIL. Data moves between processes by **pickling**, which adds overhead. Use `Pool` or `concurrent.futures.ProcessPoolExecutor` for parallel maps, and always put startup code under `if __name__ == "__main__":`.

NOTES

- Each process has separate globals; nothing is shared by default
- Functions and arguments must be picklable (top-level defs)
- Windows and macOS use the "spawn" start method
- A `Process` discards its target's return value
- Best for large CPU-bound tasks, not tiny ones

UNDER THE HOOD

- Each worker is a separate interpreter with its own GIL
- Arguments and results are pickled through pipes
- `spawn` re-imports your main module in every child

PROCESS, POOL, QUEUE, EXECUTOR

```
from multiprocessing import Process, Pool, Queue, cpu_count
from concurrent.futures import ProcessPoolExecutor
def square(n):
    return n * n                        # runs in a worker process
def worker(q):
    q.put("hello from child")
if __name__ == "__main__":              # required with spawn
    p = Process(target=square, args=(5,))
    p.start(); p.join()                 # return value is discarded
    with Pool(processes=cpu_count()) as pool:
        pool.map(square, range(5))      # [0, 1, 4, 9, 16]
    q = Queue()
    Process(target=worker, args=(q,)).start()
    q.get()                             # 'hello from child'
    with ProcessPoolExecutor() as ex:
        list(ex.map(square, [10, 20])) # [100, 400]
        ex.submit(square, 7).result()  # 49
```

CODE EXPLAINED

→ `__main__` guard — child processes re-import the module

→ `Pool.map()` — splits work across cores

→ `Queue` — pass results back from a `Process`

→ `ProcessPoolExecutor` — same API as the thread version

→ `submit().result()` — one task, then wait for its value

PYTHON VERSIONS

- **3.8** — `multiprocessing.shared_memory`
- **3.14** — Linux default start method is `forkserver`

THE PYTHONIC WAY

✗ ~~`Process` + manual result plumbing~~ → ✓ `ProcessPoolExecutor().map()`

✗ ~~`lambda` as a worker~~ → ✓ **top-level `def` function**

✗ ~~Shared globals between processes~~ → ✓ **return values from workers**

API

OBJECT	USE
`Process(target, args)`	One child process
`Pool(n).map(f, xs)`	Parallel map
`pool.starmap(f, pairs)`	Multi-arg map
`Queue() / Pipe()`	Pass data
`Value / Array`	Shared memory values
`ProcessPoolExecutor`	Futures-based pool

THREADS VS PROCESSES

THREADS	PROCESSES
`Shared memory`	Separate memory
`Limited by GIL`	Use all cores
`Cheap to start`	Costly to start
`Pass objects directly`	Data is pickled
`Best for I/O`	Best for CPU

RULES OF THUMB

RULE	WHY
`Guard with __main__`	Prevents runaway spawning
`Top-level functions`	Lambdas can't pickle
`Use chunksize=`	Fewer, bigger hand-offs
`Send small data`	Pickling is slow

⚠ COMMON MISTAKES

- No `__main__` guard — `RuntimeError` or endless child processes
- Passing a lambda or nested function — `PicklingError`
- Parallelizing tiny tasks — overhead outweighs the gain
- Expecting a child to change the parent's globals

✓ TIPS & BEST PRACTICES

- Start with `ProcessPoolExecutor().map()`
- Set `chunksize` for long input lists
- Return results instead of sharing state
- Measure first; sometimes NumPy vectorization beats processes

QUICK SUMMARY

- processes — bypass the GIL
- `Pool` / `ProcessPoolExecutor` — parallel map
- `Queue` — communicate
- `__main__` guard — always

SQLite Database

BUILT-IN SQL DATABASE

`sqlite3` gives you a full SQL database stored in a single file, with no server to install. Connect, get a cursor, run SQL with `execute()`, and save changes with `commit()`. Always pass values with **? placeholders** rather than formatting them into the SQL string; this prevents SQL injection and handles quoting for you.

NOTES

- `":memory:"` creates a temporary in-RAM database
- Parameters must be a tuple or list: `(x,)`
- `with con:` commits or rolls back, but doesn't close
- `sqlite3.Row` allows access by column name
- Explore files with the `sqlite3` CLI or DB Browser

CRUD WITH SQLITE3

```
import sqlite3
con = sqlite3.connect("shop.db")        # file, or ":memory:"
con.row_factory = sqlite3.Row           # rows act like dicts
cur = con.cursor()
cur.execute("""CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY, name TEXT NOT NULL, price REAL)""")
cur.execute("INSERT INTO items (name, price) VALUES (?, ?)", ("pen", 1.5))
cur.executemany("INSERT INTO items (name, price) VALUES (?, ?)",
                [("ink", 4.0), ("pad", 2.25)])
con.commit()                            # save changes
cur.execute("SELECT * FROM items WHERE price > ?", (2,))
[dict(r) for r in cur.fetchall()]       # [{'id': 2, 'name': 'ink', ...}, ...]
cur.execute("UPDATE items SET price = ? WHERE name = ?", (1.75, "pen"))
cur.execute("DELETE FROM items WHERE id = ?", (3,))
with con:                               # commit, or rollback on error
    con.execute("INSERT INTO items (name) VALUES (?)", ("cap",))
con.close()
```

CODE EXPLAINED

- → `INTEGER PRIMARY KEY` — auto-assigned row id
- → `?` placeholders — values passed separately and safely
- → `executemany()` — one statement, many rows
- → `(2,)` — a one-item tuple, not `(2)`
- → `with con:` — transaction block

PYTHON VERSIONS

- **3.12** — `autocommit` connection attribute

THE PYTHONIC WAY

✗ ~~`f"... WHERE id = {i}"`~~ → ✓ `"... WHERE id = ?", (i,)`

✗ ~~`row[2]`~~ → ✓ `row_factory = sqlite3.Row` → `row["email"]`

✗ ~~Looped `execute` inserts~~ → ✓ `executemany()`

CONNECTION & CURSOR

CALL	DOES
`connect(path)`	Open / create DB
`con.cursor()`	New cursor
`cur.execute(sql, params)`	Run one statement
`cur.fetchone()`	Next row or None
`cur.fetchall()`	All rows
`con.commit() / rollback()`	Save / undo
`cur.lastrowid`	Id of last insert

TYPE MAPPING

SQLITE	PYTHON
`NULL`	None
`INTEGER`	int
`REAL`	float
`TEXT`	str
`BLOB`	bytes

PLACEHOLDER STYLES

SQL	PARAMS
`WHERE id = ?`	(5,)
`VALUES (?, ?)`	("pen", 1.5)
`WHERE name = :n`	{"n": "pen"}
`IN (?, ?, ?)`	One ? per value

⚠ COMMON MISTAKES

- `f"... WHERE name = '{name}'"` — SQL injection; use `?`
- Forgetting `commit()` — inserts vanish when the program ends
- Passing `(5)` instead of `(5,)` — not a tuple
- Assuming `with con:` closes the connection — call `close()`

✓ TIPS & BEST PRACTICES

- Set `row_factory = sqlite3.Row` for readable rows
- Use `executemany()` for bulk inserts
- Iterate the cursor directly for big result sets
- Move to SQLAlchemy or an ORM as the schema grows

QUICK SUMMARY

- `connect()` — open database
- `execute(sql, params)` — run SQL
- `?` — safe parameters
- `commit()` — persist changes

Serialization & Pickling

SAVING PYTHON OBJECTS

Serialization turns objects into bytes or text you can store or send, and **deserialization** rebuilds them. `pickle` handles almost any Python object, including sets, tuples, and class instances, but the format is Python-only and **unsafe to load from untrusted sources**, because unpickling can execute code. `shelve` is a dict-like store built on pickle. Prefer JSON for data exchange.

NOTES

- Pickle files are binary: open with `"wb"` / `"rb"`
- Unpickling needs the original class importable
- `marshal` is for Python's own `.pyc` files, not your data
- `copy.deepcopy()` uses the same machinery
- `pickle.HIGHEST_PROTOCOL` selects the newest format

PICKLE AND SHELVE

```
import pickle, shelve, json
data = {"user": "ana", "scores": {3, 5}, "when": (2026, 10, 1)}
blob = pickle.dumps(data)               # bytes; keeps set and tuple
pickle.loads(blob) == data              # True
with open("state.pkl", "wb") as f:
    pickle.dump(data, f)                # binary mode
with open("state.pkl", "rb") as f:
    restored = pickle.load(f)
json.dumps(data)                        # TypeError: set not serializable
with shelve.open("cache") as db:        # persistent dict
    db["config"] = {"theme": "dark"}
    db.get("config")                    # {'theme': 'dark'}
class Point:
    def __init__(self, x, y): self.x, self.y = x, y
p = pickle.loads(pickle.dumps(Point(1, 2)))
p.x                                     # 1
# NEVER pickle.load() data from an untrusted source
```

CODE EXPLAINED

- → `dumps` / `loads` — bytes in memory
- → `dump` / `load` — files opened in binary mode
- → pickle keeps Python types that JSON can't represent
- → `shelve` — keys are strings, values are pickled

PYTHON VERSIONS

- **3.8** — pickle protocol 5
- **3.14** — protocol 5 is the default

THE PYTHONIC WAY

- ✗ ~~`pickle` for config or data exchange~~ → ✓ `json`
- ✗ ~~`pickle.loads(untrusted_bytes)`~~ → ✓ **never; use `json`**
- ✗ ~~`shelve` as a database~~ → ✓ `sqlite3`

JSON VS PICKLE

JSON	PICKLE
Text	Binary
Any language	Python only
Basic types	Almost any object
Safe to parse	Can run code on load
Human-readable	Not readable

MODULES

MODULE	USE
`json`	Data exchange, configs
`pickle`	Python object snapshots
`shelve`	Disk-backed dict
`csv`	Tabular text
`sqlite3`	Queryable storage

SAFER CHOICES

NEED	USE
API / config data	json
Compact binary	msgpack
Typed schemas	protobuf
ML models	Library's own save()
Trusted cache only	pickle

⚠ COMMON MISTAKES

- Loading pickles from users or the internet — arbitrary code execution
- Opening pickle files in text mode — `TypeError` / corrupt data
- Renaming or moving a class — old pickles fail to load
- Using pickle for long-term storage across Python versions

✓ TIPS & BEST PRACTICES

- Use JSON whenever the data is plain dicts, lists, and strings
- Treat pickle files like executable code
- Pass `protocol=pickle.HIGHEST_PROTOCOL` for speed and size
- Sign or checksum pickles if you must move them between machines

QUICK SUMMARY

- `pickle` — any Python object
- `shelve` — persistent dict
- JSON — portable and safe
- untrusted pickle — never load

Type Hints & Typing Module

ANNOTATING YOUR CODE

Type hints annotate parameters, return values, and variables with expected types. Python itself **ignores them at runtime**; tools such as **mypy** and **pyright**, along with editors, use them to catch bugs before you run the code. Modern syntax uses built-in generics like `list[int]` (3.9+), `X | None` (3.10+), and the `type` statement and `[T]` generics (3.12+).

NOTES

- Hints don't convert or validate values
- `Optional[X]` means `X | None`, not "optional argument"
- `from __future__ import annotations` defers evaluation
- Use `Any` sparingly; it switches checking off
- Pydantic and dataclasses can use hints at runtime

HINTS IN PRACTICE

```
def greet(name: str, times: int = 1) -> str:
    return f"Hi {name}! " * times
scores: list[int] = [90, 85]            # built-in generics (3.9+)
lookup: dict[str, float] = {}
pair: tuple[int, str] = (1, "a")
maybe: int | None = None                # union syntax (3.10+)
from typing import Callable, TypedDict, Literal
def apply(f: Callable[[int], int], x: int) -> int:
    return f(x)
class Movie(TypedDict):
    title: str
    year: int
mode: Literal["r", "w"] = "r"
def first[T](items: list[T]) -> T:      # generic function (3.12+)
    return items[0]
type Vector = list[float]               # type alias (3.12+)
greet(42)                               # runs anyway; mypy reports it
```

CODE EXPLAINED

- → `-> str` — return type
- → `int | None` — value may be missing
- → `Callable[[int], int]` — takes an int, returns an int
- → `TypedDict` — dict with known keys
- → `[T]` — the return type follows the input type

PYTHON VERSIONS

- **3.9** — `list[int]` built-in generics
- **3.10** — `X | Y` unions
- **3.12** — `type` aliases and `def f[T]()` generics
- **3.14** — annotations evaluated lazily (PEP 649)

THE PYTHONIC WAY

- ✗ ~~`List[int]`~~, ~~`Dict[str, int]`~~ → ✓ `list[int]`, `dict[str, int]`
- ✗ ~~`Optional[str]`~~ / ~~`Union[a, b]`~~ → ✓ `str | None` / `a | b`
- ✗ ~~`Any`~~ ~~everywhere~~ → ✓ **a precise type or** `object`

COMMON HINTS

HINT	MEANS
`list[str]`	List of str
`dict[str, int]`	str → int
`tuple[int, ...]`	Any-length int tuple
`set[int]`	Set of int
`X | None`	X or None
`Iterable[int]`	Anything iterable

TYPING HELPERS

NAME	USE
`Any`	Opt out of checks
`Literal[...]`	Exact allowed values
`Final`	Don't reassign
`Protocol`	Structural interface
`Self`	Returns own class (3.11+)
`TypeVar / [T]`	Generics

TOOLS

TOOL	DOES
`mypy app.py`	Static type check
`pyright`	Fast checker (VS Code)
`reveal_type(x)`	Show inferred type
`get_type_hints(f)`	Read hints at runtime
`pydantic`	Validate at runtime

⚠ COMMON MISTAKES

- Expecting hints to reject bad values at runtime — they don't
- `def f(x: list = [])` — hints don't fix mutable defaults
- Overusing `Any` — hides the bugs hints are meant to catch
- Using `List`, `Dict` from `typing` in new code — use `list`, `dict`

✓ TIPS & BEST PRACTICES

- Annotate public functions first; locals are usually inferred
- Run mypy or pyright in CI
- Accept abstract types (`Iterable`), return concrete ones (`list`)
- Use `TypedDict` or dataclasses for structured data

QUICK SUMMARY

- `x: T` / `-> T` — annotations
- `list[int]`, `X | None` — modern syntax
- checkers — mypy, pyright
- runtime — hints ignored

GUI Frameworks Overview

DESKTOP APPS IN PYTHON

A GUI framework draws windows and widgets and runs an **event loop** that waits for clicks and keystrokes, then calls your **callbacks**. **Tkinter** ships with Python and suits small tools and learning; **Qt** (PySide6 / PyQt6) is the usual choice for full desktop apps. Every framework shares the same shape: build widgets, arrange them with a layout, connect events, start the loop.

NOTES

- Full coverage: the Tkinter Reference set (62 sheets)
- Tkinter needs no install; Linux may need `python3-tk`
- The event loop blocks — keep callbacks short
- GUI objects must only be touched from the main thread

SAME APP, TWO FRAMEWORKS

```
# Tkinter — standard library, no install
import tkinter as tk
from tkinter import ttk

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Counter")
        self.count = tk.IntVar(value=0)
        ttk.Label(self, textvariable=self.count).pack(padx=20, pady=8)
        ttk.Button(self, text="Add", command=self.add).pack(pady=8)

    def add(self):
        self.count.set(self.count.get() + 1)

App().mainloop()                        # event loop; blocks until closed

# PySide6 (Qt) — pip install pyside6
from PySide6.QtWidgets import QApplication, QPushButton
app = QApplication([])
btn = QPushButton("Add")
btn.clicked.connect(lambda: print("clicked"))    # signal → slot
btn.show()
app.exec()
```

CODE EXPLAINED

- → `class App(tk.Tk)` — window, state and widgets in one object
- → `IntVar` — control variable; the label updates itself
- → `command=` — Tkinter callback on click
- → `clicked.connect()` — Qt signal wired to a slot
- → `mainloop()` / `exec()` — run the event loop

THE PYTHONIC WAY

- ✗ ~~`from tkinter import *`~~ → ✓ `import tkinter as tk`
- ✗ ~~Classic~~ ~~`tk.Button`~~ → ✓ `ttk.Button` **(native look)**
- ✗ ~~`time.sleep()`~~ ~~in a callback~~ → ✓ `after()` / `QTimer`

GUI FRAMEWORKS

FRAMEWORK	BEST FOR
`Tkinter + ttk`	Built in; small tools
`PySide6`	Full Qt apps (LGPL)
`PyQt6`	Qt apps (GPL / commercial)
`wxPython`	Native OS widgets
`Kivy`	Touch & mobile
`Toga (BeeWare)`	Native, incl. mobile
`Dear PyGui`	GPU tools & dashboards

SHARED CONCEPTS

CONCEPT	TKINTER	QT
`Main window`	tk.Tk()	QMainWindow
`Event loop`	mainloop()	app.exec()
`Callback`	command= / bind()	signal.connect()
`Layout`	pack / grid / place	QVBoxLayout / QGridLayout
`Timer`	after(ms, f)	QTimer
`Dialogs`	messagebox / filedialog	QMessageBox / QFileDialog

TKINTER SET — WHERE TO LOOK

TOPIC	SHEETS
`Windows & widgets`	4–15
`Layout managers`	16–20
`Variables & events`	21–26
`Menus & dialogs`	27–32
`ttk themed widgets`	33–41
`Canvas`	42–46
`App patterns`	50–57

⚠ COMMON MISTAKES

- Forgetting `mainloop()` / `exec()` — the window flashes and the script ends
- Slow work in a callback — the window freezes; use a thread
- Updating widgets from a worker thread — crashes or glitches
- Creating a second `tk.Tk()` — use `Toplevel` for more windows

✓ TIPS & BEST PRACTICES

- Start with Tkinter; move to Qt when you need richer widgets
- Keep logic in plain functions so it is testable without the GUI
- Hand results back to the main thread (`after()` / signals)
- Ship apps with PyInstaller or Briefcase

QUICK SUMMARY

- Tkinter — built in, simple
- PySide6 — full-featured Qt
- Widgets → layout → events → loop
- Deep dive: Tkinter set

Web Frameworks Overview

SERVING THE WEB FROM PYTHON

A web framework maps a **URL route** to a Python function that takes a request and returns a response. **Django** is "batteries included" (ORM, admin, auth, templates); **Flask** is a small core you extend; **FastAPI** builds async JSON APIs from **type hints**. Apps run behind a **WSGI** (sync) or **ASGI** (async) server.

NOTES

- Full coverage: the Django Reference set (70 sheets)
- Install into a virtual environment (Sheet 02)
- Built-in dev servers are not for production
- Never commit secret keys — read them from env vars

ONE ROUTE, THREE FRAMEWORKS

```
# Flask — pip install flask            (WSGI, small core)
from flask import Flask
app = Flask(__name__)

@app.get("/hello/<name>")
def hello(name):
    return {"msg": f"Hello, {name}"}        # dict → JSON
# run: flask --app app run --debug

# FastAPI — pip install "fastapi[standard]"   (ASGI, async)
from fastapi import FastAPI
api = FastAPI()

@api.get("/hello/{name}")
async def hello(name: str, times: int = 1):   # hints validate input
    return {"msg": f"Hello, {name} " * times}
# run: fastapi dev main.py              → docs at /docs

# Django — pip install django             (full stack)
# django-admin startproject mysite; python manage.py startapp blog
from django.http import JsonResponse
def hello(request, name):                    # blog/views.py
    return JsonResponse({"msg": f"Hello, {name}"})
# urls.py: path("hello/<str:name>/", views.hello)
```

CODE EXPLAINED

- → `@app.get(...)` — register a route for GET
- → `<name>` / `{name}` — path parameter
- → `times: int = 1` — FastAPI query param, converted and checked
- → `JsonResponse` — Django views return response objects
- → `path()` — Django routes live in `urls.py`

THE PYTHONIC WAY

- ✗ ~~Hand-built JSON strings~~ → ✓ **return a dict / `JsonResponse`**
- ✗ ~~SQL built with f-strings~~ → ✓ **the ORM or `?` placeholders**
- ✗ ~~`DEBUG = True` in production~~ → ✓ **settings from env vars**

WEB FRAMEWORKS

FRAMEWORK	BEST FOR
Django	Full sites, admin, ORM
Flask	Small apps, pick your parts
FastAPI	Typed async JSON APIs
Starlette	ASGI toolkit (FastAPI's base)
Litestar	Typed ASGI APIs
Django REST Framework	APIs on top of Django

BUILT IN OR ADD-ON?

FEATURE	DJANGO	FLASK	FASTAPI
ORM	Built in	SQLAlchemy	SQLModel
Admin UI	Built in	Flask-Admin	—
Validation	Forms	WTForms	Pydantic
Templates	DTL	Jinja2	Jinja2
Auth	Built in	Flask-Login	Helpers
API docs	DRF	—	Auto /docs

SERVERS

SERVER	RUNS
Gunicorn	WSGI — Django, Flask
Uvicorn	ASGI — FastAPI, async Django
Hypercorn	ASGI, HTTP/2
Waitress	WSGI, works on Windows
runserver / flask run	Development only

⚠ COMMON MISTAKES

- Deploying with the dev server or `DEBUG` on
- Blocking calls inside `async def` routes — stalls every request
- Hard-coding secret keys and passwords in source
- Rebuilding Django's admin and auth by hand in Flask

✓ TIPS & BEST PRACTICES

- Pick Django for full sites, FastAPI for APIs, Flask for small apps
- Use plain `def` routes unless the code awaits I/O
- Run behind Gunicorn / Uvicorn and a reverse proxy
- Test with each framework's test client

QUICK SUMMARY

- Route → function → response
- Django — batteries included
- Flask — micro · FastAPI — typed async
- Deep dive: Django set

Sheet 92

Quick Reference — Syntax & Types

running · types & conversion · strings & f-strings · operators · control flow & match · comprehensions · functions · exceptions

RUNNING & BASICS

SYNTAX	DOES	#
`python3 app.py`	run a script	03
`python3 -m venv .venv`	virtual environment	02
`if __name__ == "__main__":`	entry point	04
`print(a, b, sep=", ", end="")`	output	05
`input("Name: ")`	read a line (str)	13
`# comment / """docstring"""`	comments	06

TYPES & CONVERSION

SYNTAX	DOES	#
`int / float / bool / str`	core types	08
`None`	no value; test with `is`	11
`int("42") / float("3.5")`	parse numbers	12
`str(x) / repr(x)`	to text	12
`type(x) / isinstance(x, int)`	check type	11
`bool([]) → False`	empty is falsy	10

STRINGS

SYNTAX	DOES	#
`f"{name!r} {x:.2f}"`	f-string + format spec	15
`f"{n:>8,}"`	align, thousands	15
`s.find / s.startswith`	search	16
`s.split(",") / ",".join(xs)`	split / join	17
`s.strip() / s.replace(a, b)`	transform	17
`s[1:4] / s[::-1]`	slice / reverse	18

OPERATORS

OPERATOR	DOES	#
`// and %`	floor div / modulo	19
`**`	power	19
`== vs is`	value vs identity	24
`in / not in`	membership	24
`and / or / not`	return an operand	21
`:=`	walrus (assign in expr)	23
`a if cond else b`	conditional expression	27

CONTROL FLOW

SYNTAX	DOES	#
`if / elif / else`	branch	25
`match x: case 1:`	pattern matching	26
`case {"type": t}:`	mapping pattern	26
`case Point(x=0):`	class pattern	26
`for i, x in enumerate(xs):`	index + value	28
`for a, b in zip(xs, ys):`	pairs	28
`while … else / break`	loop + else	30

COMPREHENSIONS

SYNTAX	BUILDS	#
`[x*2 for x in xs]`	list	31
`[x for x in xs if x > 0]`	filtered list	31
`{k: v for k, v in pairs}`	dict	32
`{x % 3 for x in xs}`	set	32
`(x*x for x in xs)`	lazy generator	33
`sum(x for x in xs)`	generator in a call	33

FUNCTIONS

SYNTAX	DOES	#
`def f(a, b=2):`	default arg	39
`def f(*args, **kwargs):`	variadic	38
`def f(a, /, b, *, c):`	positional / keyword-only	38
`lambda x: x + 1`	anonymous function	42
`nonlocal / global`	rebind outer names	40
`@decorator`	wrap a function	44
`yield x`	generator function	45

EXCEPTIONS

SYNTAX	DOES	#
`try / except ValueError as e:`	catch	34
`except (A, B):`	several types	34
`else / finally`	no error / always	34
`raise ValueError("bad")`	raise	35
`raise X from e`	chain	35
`class MyError(Exception):`	custom	36
`except* ValueError`	exception groups	35

⚠ COMMON MISTAKES

- Mutable default arguments (`def f(x=[])`) — use `None` and create inside
- `==` with `None` or `True` — use `is None`
- Shadowing built-ins like `list`, `dict`, `id` with variable names

✓ TIPS & BEST PRACTICES

- Use f-strings for all formatting
- Prefer `enumerate` and `zip` to index loops
- Catch the narrowest exception you can handle

QUICK SUMMARY

- Indentation is syntax; `:` opens a block
- f-strings, slicing and comprehensions are everyday tools
- `match` / `case` for structural patterns
- `try` / `except` / `else` / `finally`

Sheet 93

Quick Reference — Collections, OOP & Standard Library

lists, tuples, dicts, sets · collections & itertools · sorting · classes & dunders · files & paths · modules · testing · async & concurrency

LISTS & TUPLES

SYNTAX	DOES	#
`xs.append(x) / extend(ys)`	add	47
`xs.insert(i, x) / pop(i)`	insert / remove at	47
`xs.remove(x)`	remove first match	47
`xs.sort() vs sorted(xs)`	in place vs new	56
`a, *rest = xs`	unpacking	48
`t = (1,)`	one-item tuple	49

DICTS & SETS

SYNTAX	DOES	#
`d.get(k, default)`	safe lookup	51
`d.setdefault(k, [])`	get or insert	51
`d.items() / keys() / values()`	views	51
`d | other`	merge (3.9+)	52
`s | t / s & t / s - t`	union / inter / diff	53
`frozenset(xs)`	immutable set	53

COLLECTIONS & ITERTOOLS

NAME	DOES	#
`Counter(xs).most_common(3)`	count items	54
`defaultdict(list)`	auto default	54
`deque(maxlen=5)`	fast both ends	54
`namedtuple / dataclass`	records	54
`chain / groupby / product`	iteration tools	55
`islice / pairwise / batched`	slicing tools	55

SORTING & FUNCTOOLS

SYNTAX	DOES	#
`sorted(xs, key=len)`	sort by key	56
`key=lambda p: (p.age, p.name)`	multi-key	56
`reverse=True`	descending	56
`bisect.insort(xs, x)`	keep sorted	56
`@lru_cache / @cache`	memoize	79
`partial(f, x)`	preset args	79

CLASSES

SYNTAX	DOES	#
`class A(Base):`	inheritance	61
`def __init__(self, x):`	constructor	58
`@classmethod / @staticmethod`	class / static	59
`@property`	computed attribute	60
`super().__init__()`	call parent	61
`@dataclass`	generated `__init__`, `__repr__`	54
`ABC + @abstractmethod`	abstract class	64

DUNDER METHODS

METHOD	ENABLES	#
`__repr__ / __str__`	printing	65
`__eq__ / __hash__`	equality, dict keys	65
`__len__ / __iter__`	`len()`, `for`	66
`__getitem__`	`obj[k]`	66
`__add__ / __lt__`	operators	66
`__enter__ / __exit__`	`with` blocks	67

FILES & DATA

SYNTAX	DOES	#
`with open(p, encoding="utf-8") as f:`	open safely	68
`f.read() / for line in f`	read	68
`open(p, "a") / f.write(s)`	append / write	69
`Path("a") / "b.txt"`	pathlib paths	70
`json.load(f) / json.dumps(x)`	JSON	71
`csv.DictReader(f)`	CSV rows as dicts	72

MODULES & TOOLS

ITEM	DOES	#
`import x / from x import y`	imports	73
`__init__.py / pyproject.toml`	packages	74
`pip install / uv add`	dependencies	75
`pytest + assert`	tests	81
`logging.getLogger(__name__)`	logging	82
`re.search / re.sub`	regex	83

ASYNC & CONCURRENCY

API	DOES	#
`async def / await`	coroutine	84
`asyncio.run(main())`	event loop	84
`asyncio.gather(a, b)`	run concurrently	84
`ThreadPoolExecutor`	I/O-bound threads	85
`ProcessPoolExecutor`	CPU-bound processes	86
`sqlite3.connect(p)`	built-in SQL DB	87

⚠ COMMON MISTAKES

- Modifying a list while iterating over it
- `sort()` returns `None` — use `sorted()` when you need the result
- Opening files without `encoding="utf-8"` or without `with`

✓ TIPS & BEST PRACTICES

- Use `pathlib` for paths and `dataclasses` for plain records
- Reach for `Counter`, `defaultdict` and `itertools` before writing loops
- Add type hints and run a checker (mypy / pyright)

QUICK SUMMARY

- list / tuple / dict / set — know their methods
- `sorted(key=…)` and `functools` for ordering and caching
- Classes: `__init__`, `@property`, dunders, `@dataclass`
- `with open` + `pathlib` + `json` / `csv` for files